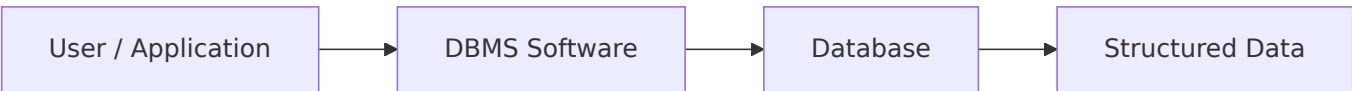


# Chapter 1: Database Fundamentals

## 1. What is DBMS?

A **Database Management System (DBMS)** is software that enables users to define, create, maintain, and control access to databases. It acts as an intermediary between the user/application and the physical data storage.

**Examples:** MySQL, PostgreSQL, Oracle, MongoDB, SQLite.

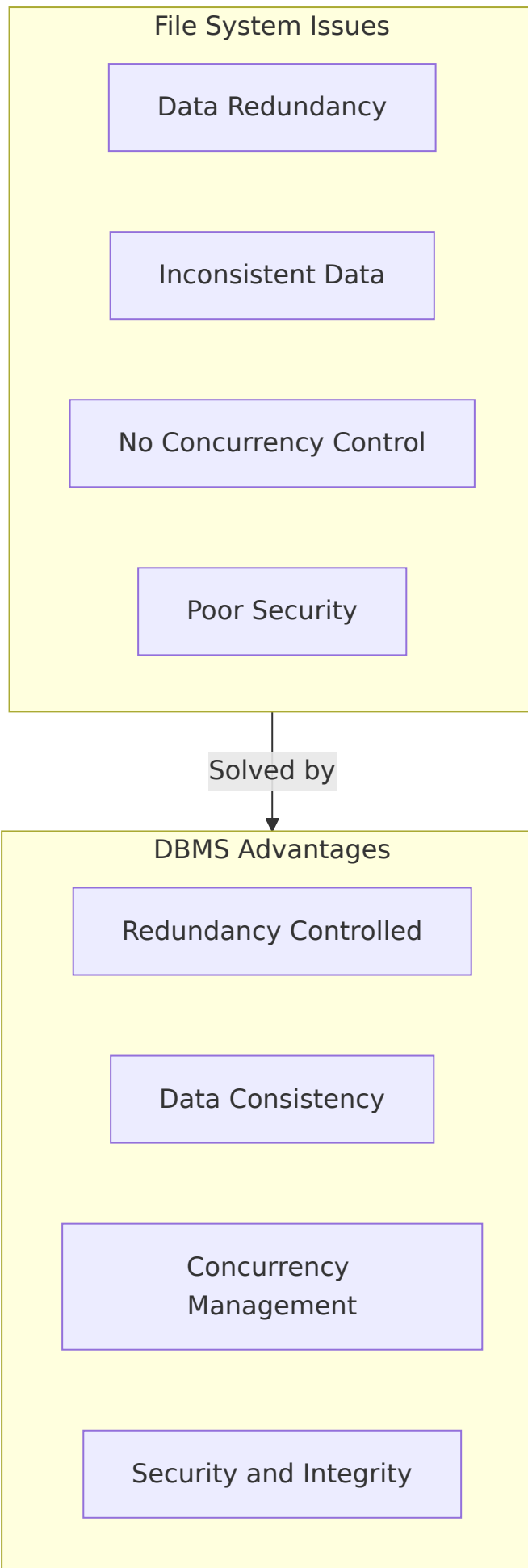


### Key functions of a DBMS:

- Data definition (creating tables, constraints)
- Data manipulation (insert, update, delete, query)
- Data security and integrity enforcement
- Concurrent access control
- Backup and recovery

## 2. Advantages of DBMS over File System

| Feature            | File System              | DBMS                            |
|--------------------|--------------------------|---------------------------------|
| Data redundancy    | High (data duplicated)   | Controlled, minimal             |
| Data inconsistency | Common                   | Avoided via constraints         |
| Data access        | Requires custom programs | Standard query language (SQL)   |
| Concurrent access  | No control, may corrupt  | Built-in concurrency control    |
| Security           | Limited                  | User authentication, privileges |
| Backup & recovery  | Manual                   | Automated, consistent           |
| Data integrity     | Application-dependent    | Enforced by DBMS                |



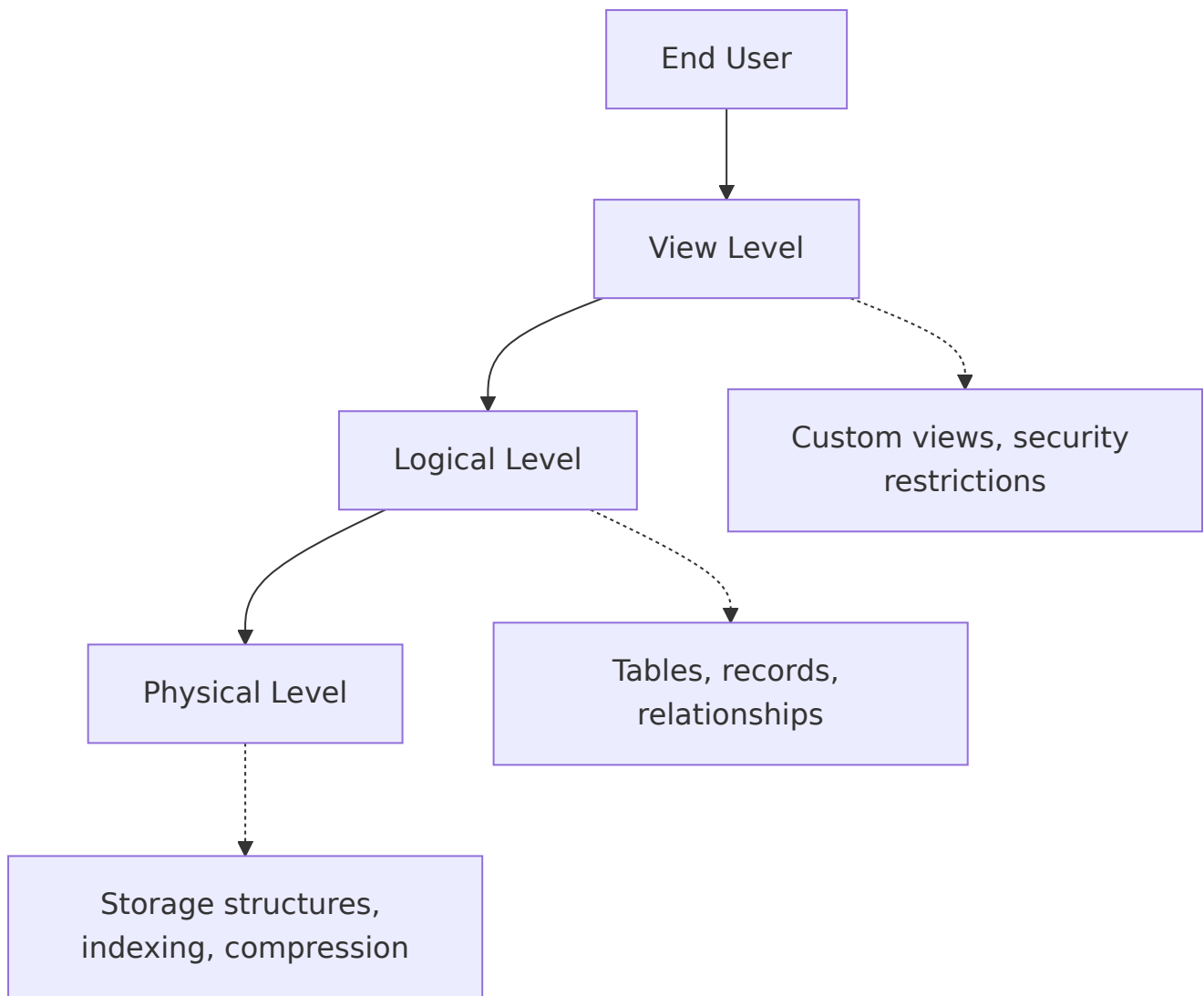
Detailed explanation:

- **Redundancy & Inconsistency:** In file systems, the same data may be stored in multiple files (redundancy). Updates in one file may not reflect in others (inconsistency). DBMS normalizes data and uses constraints to eliminate redundancy.
- **Concurrent access:** File systems allow multiple users to read/write same file – leads to lost updates or corrupted data. DBMS uses locking and transactions.
- **Security:** DBMS provides granular access control (user, role, table, column level).
- **Backup & Recovery:** DBMS automatically logs changes and can restore to a consistent state after a crash.

### 3. Data Abstraction

Data abstraction hides complex storage details and provides users with a simplified view. It has **three levels**:

| Level    | Description                                                                | Perspective              |
|----------|----------------------------------------------------------------------------|--------------------------|
| Physical | How data is actually stored (bits, blocks, indexes, compression)           | Low-level, DBA           |
| Logical  | What data is stored and the relationships (tables, records, data types)    | Database designer        |
| View     | Customised data presentation for specific users (subsets, derived columns) | End users / applications |



**Example:** A bank database

- **Physical:** Data stored on disk blocks with B-tree indexes on account numbers.
- **Logical:** Tables `Customer(custID, name)`, `Account(accNo, custID, balance)`.
- **View:** A teller sees only account balances; a manager sees all customer details.

## 4. Data Independence

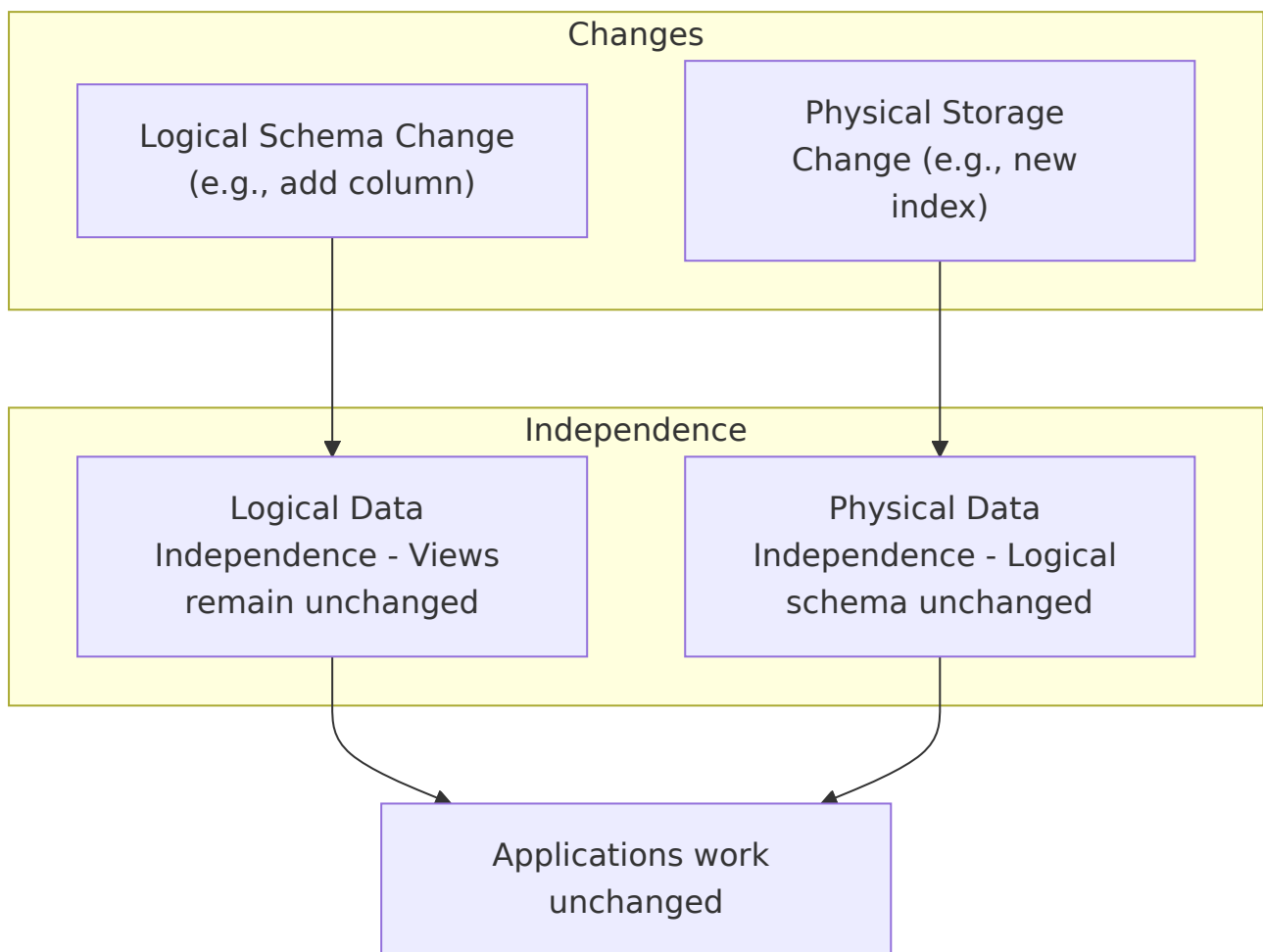
**Data independence** means changes at one level do not require changes at higher levels.

### 4.1 Logical Data Independence

- **Change:** Logical schema (adding a column, changing data type, merging tables).
- **Unaffected:** External views and application programs.

### 4.2 Physical Data Independence

- **Change:** Physical storage (switching from sequential to indexed file organisation, using different compression, adding indexes).
- **Unaffected:** Logical schema and views.



### Why it matters:

- Without logical independence, adding a single field would break every application.
- Without physical independence, optimising storage (e.g., adding an index) would require rewriting all queries.

## 5. Schema vs Instance

| Term          | Definition                                                                    | Analogy                 | Change Frequency           |
|---------------|-------------------------------------------------------------------------------|-------------------------|----------------------------|
| <b>Schema</b> | The overall design / blueprint of the database (tables, columns, constraints) | Class definition in OOP | Very rarely (months/years) |

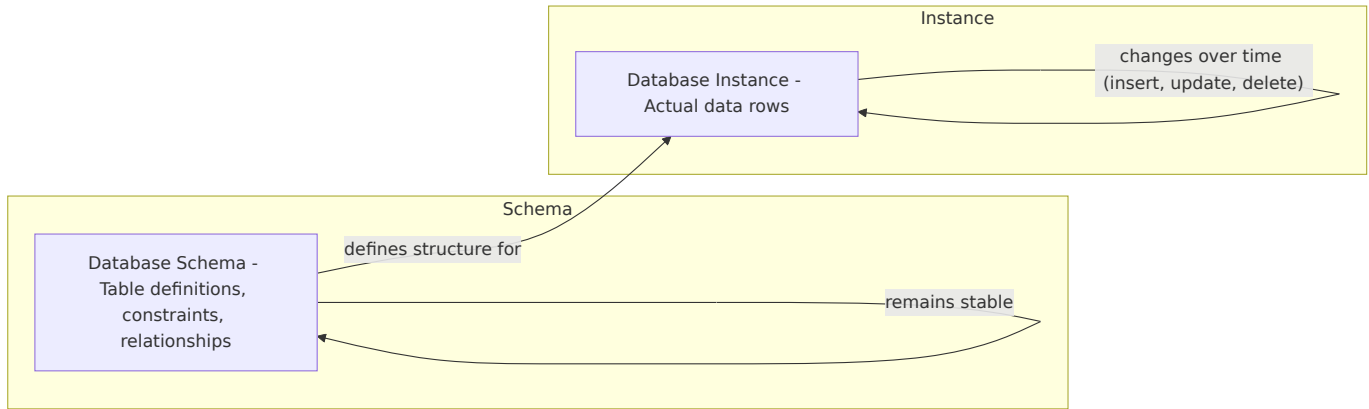
|          |                                                               |                      |                            |
|----------|---------------------------------------------------------------|----------------------|----------------------------|
| Instance | The actual data stored in the database at a particular moment | Objects of the class | Every insert/update/delete |
|----------|---------------------------------------------------------------|----------------------|----------------------------|

Example – A Student table schema:

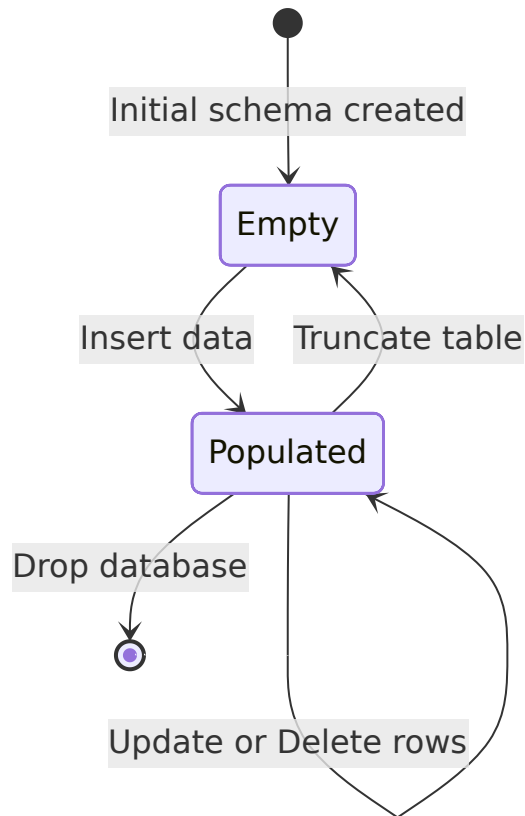
```
Student(rollNo: int, name: varchar(20), age: int)
```

An instance (snapshot of data at one moment):

| rollNo | name  | age |
|--------|-------|-----|
| 101    | Alice | 20  |
| 102    | Bob   | 22  |



State diagram for instance changes over time:



### Important:

- A database may have many instances over its lifetime, but only one schema (unless altered).
- Schema evolution (using `ALTER TABLE`) changes the blueprint but preserves existing data as much as possible.

## Summary Table

| Concept                            | Key Point                                                                              |
|------------------------------------|----------------------------------------------------------------------------------------|
| <b>DBMS</b>                        | Software for managing structured data with security, concurrency, and recovery         |
| <b>Advantages over file system</b> | No redundancy, consistency, concurrency, security, integrity                           |
| <b>Data abstraction levels</b>     | Physical (storage), Logical (tables), View (user-specific)                             |
| <b>Data independence</b>           | Logical (schema changes hide from views) & Physical (storage changes hide from schema) |

|                           |                                                               |
|---------------------------|---------------------------------------------------------------|
| <b>Schema vs Instance</b> | Schema = blueprint (stable); Instance = actual data (dynamic) |
|---------------------------|---------------------------------------------------------------|

---



# Chapter 2: Relational Model

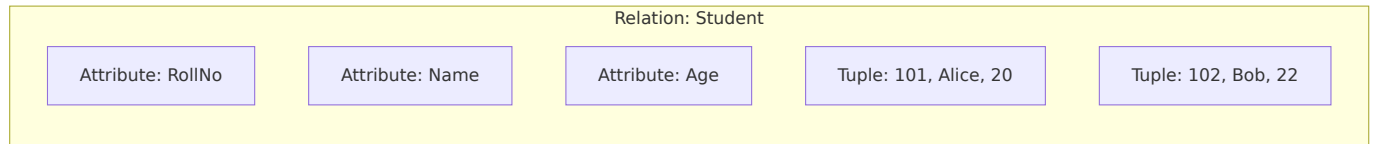
## 1. Relation, Tuple, Attribute

The relational model represents data as a **table** (relation).

| Term      | Definition                    | Analogy        |
|-----------|-------------------------------|----------------|
| Relation  | A table with rows and columns | A spreadsheet  |
| Tuple     | A row in the relation         | A record / row |
| Attribute | A column in the relation      | A field        |

**Example – Student** relation:

| RollNo (Attribute) | Name (Attribute) | Age (Attribute) |
|--------------------|------------------|-----------------|
| 101                | Alice            | 20              |
| 102                | Bob              | 22              |



### Properties of relations:

- Each tuple is unique (no duplicate rows).
- Order of tuples does not matter.
- Order of attributes does not matter (but usually fixed).
- Each attribute has a domain (data type).

## 2. Keys

Keys are attributes (or sets of attributes) that uniquely identify tuples.

### 2.1 Super Key

A **super key** is any set of attributes that uniquely identifies a tuple. It may contain extra attributes.

**Example:** In `Student(RollNo, Name, Age)`

- `{RollNo}` → super key (minimal)
- `{RollNo, Name}` → super key (redundant)
- `{Name, Age}` → not a super key (Name may duplicate)

## 2.2 Candidate Key

A **candidate key** is a minimal super key – no proper subset is a super key.

**Example:**

- `{RollNo}` is a candidate key (if RollNo is unique).
- `{Name}` may be a candidate key only if names are guaranteed unique (rare).

## 2.3 Primary Key

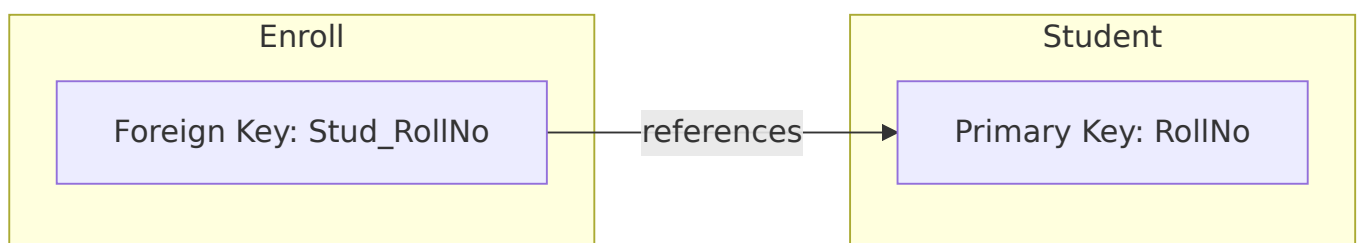
The **primary key** is one candidate key chosen by the database designer as the main identifier. It cannot be `NULL`.

**Example:** `RollNo` chosen as primary key.

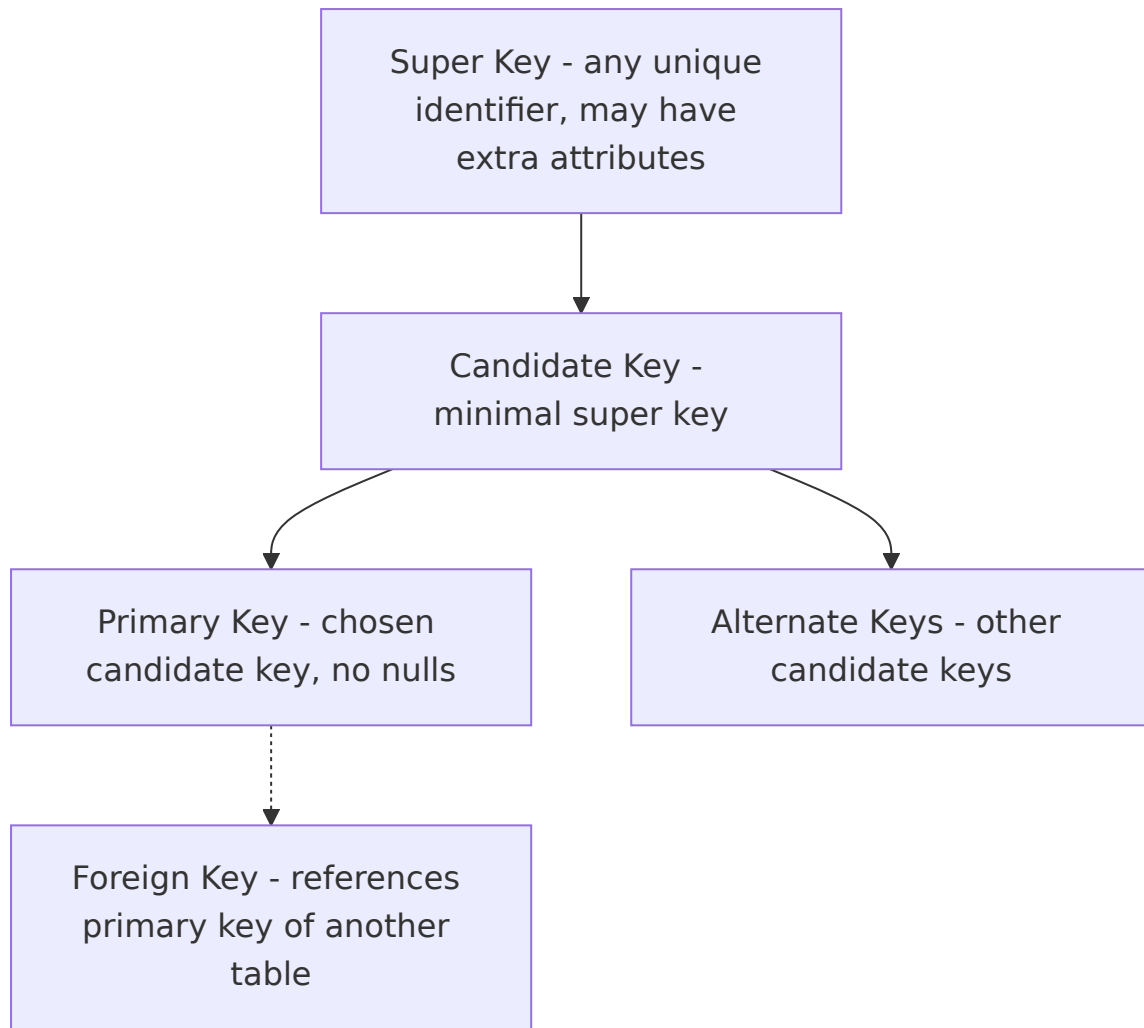
## 2.4 Foreign Key

A **foreign key** is an attribute (or set) in one relation that refers to the primary key of another relation. It establishes a relationship between tables.

**Example:** `Enroll` table has `Stud_RollNo` that references `Student(RollNo)`.



**Hierarchy of keys:**

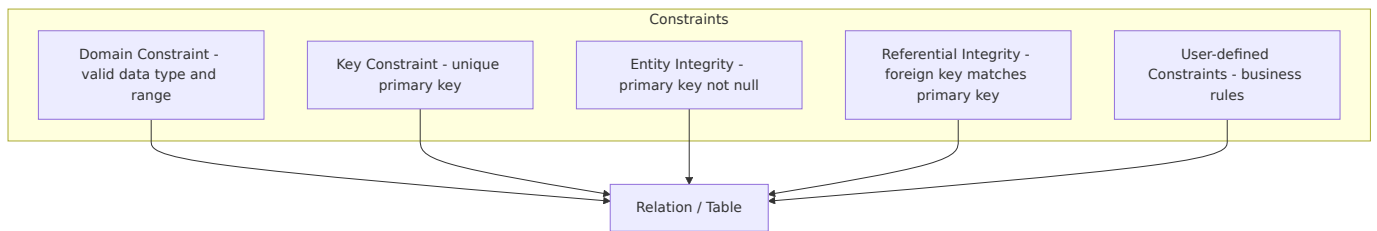


### 3. Integrity Constraints

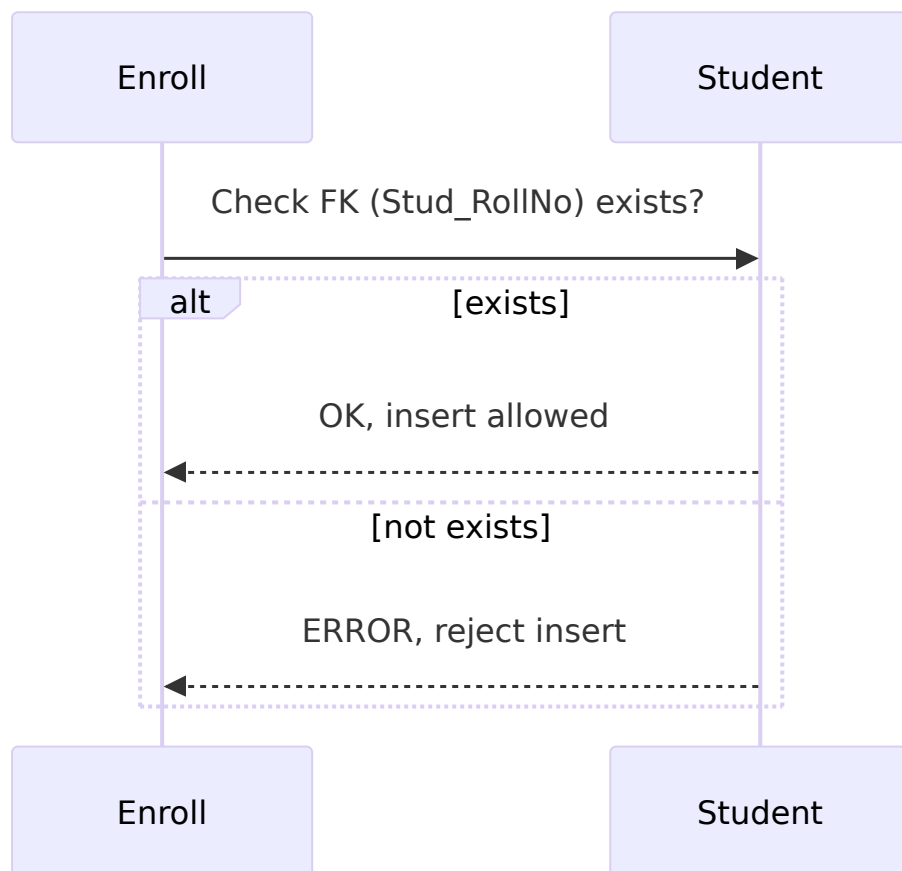
Integrity constraints ensure data accuracy and consistency.

| Constraint                   | Description                                                                                                       | Example                                                                                  |
|------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Domain constraint</b>     | Attribute values must be from a predefined domain (data type, range)                                              | Age between 0 and 120                                                                    |
| <b>Key constraint</b>        | No two tuples have the same primary key value                                                                     | RollNo unique                                                                            |
| <b>Entity integrity</b>      | Primary key cannot be <code>NULL</code>                                                                           | RollNo always present                                                                    |
| <b>Referential integrity</b> | Foreign key value must match an existing primary key in the referenced table (or be <code>NULL</code> if allowed) | Every <code>Stud_RollNo</code> in <code>Enroll</code> must exist in <code>Student</code> |

|                                |                                      |                          |
|--------------------------------|--------------------------------------|--------------------------|
| <b>User-defined constraint</b> | Business rules not covered by others | Salary > 0, Email format |
|--------------------------------|--------------------------------------|--------------------------|



**Referential integrity diagram** (using sequence diagram style):



## 4. Relational Schema

A **relational schema** describes the structure of a relation – its name, attributes, domains, and constraints.

**Notation:**

- Relation name followed by list of attributes.
- Primary key underlined.

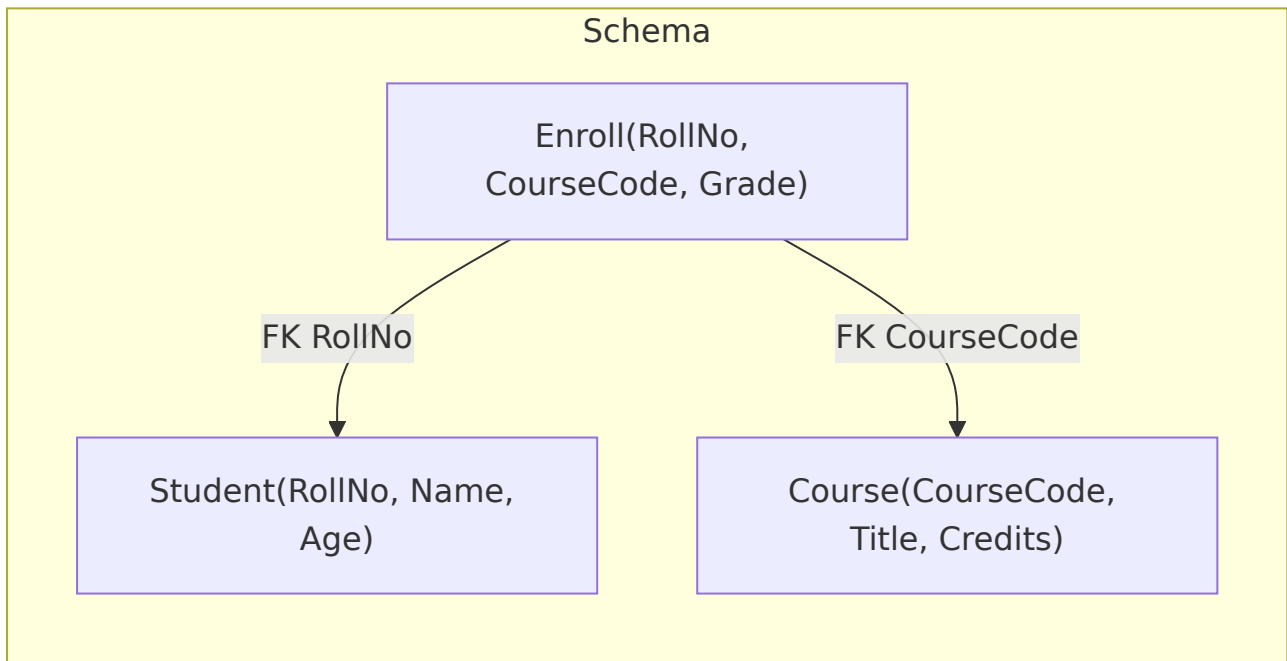
- Foreign key indicated with arrow.

**Example** – University database:

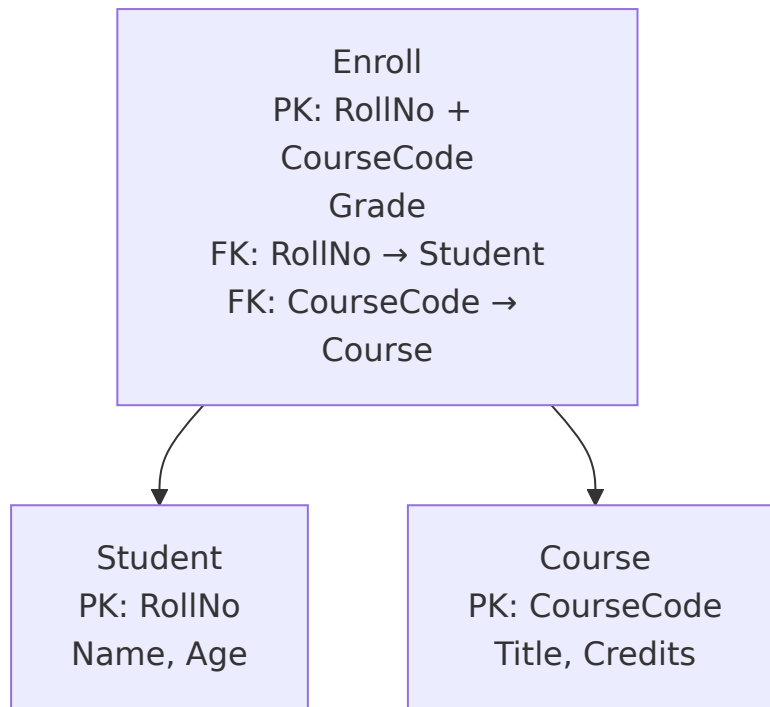
```
Student( RollNo, Name, Age )  
Enroll( RollNo, CourseCode, Grade )  
Course( CourseCode, Title, Credits )
```

with:

- Primary keys: RollNo in Student, CourseCode in Course, (RollNo, CourseCode) in Enroll.
- Foreign key: Enroll.RollNo references Student.RollNo; Enroll.CourseCode references Course.CourseCode.



**Schema diagram** (detailed):



## Summary Table

| Concept               | Definition                                         |
|-----------------------|----------------------------------------------------|
| Relation              | Table with rows and columns                        |
| Tuple                 | Row of a relation                                  |
| Attribute             | Column of a relation                               |
| Super key             | Set of attributes that uniquely identifies a tuple |
| Candidate key         | Minimal super key                                  |
| Primary key           | Chosen candidate key, not null                     |
| Foreign key           | References primary key of another relation         |
| Domain constraint     | Attribute values must be from allowed set          |
| Key constraint        | No duplicate primary keys                          |
| Entity integrity      | Primary key cannot be null                         |
| Referential integrity | Foreign key must match existing primary key        |

| Relational schema | Description of relation structure (name, attributes, keys) |
|-------------------|------------------------------------------------------------|
|-------------------|------------------------------------------------------------|

---

# Chapter 3: Relational Algebra

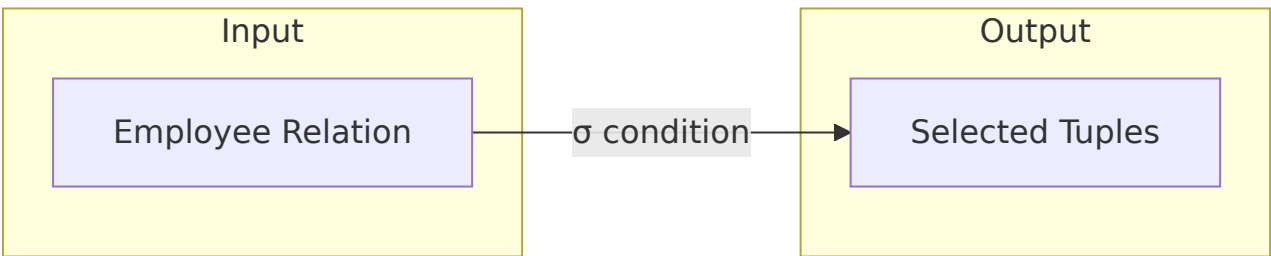
Relational algebra is a formal query language for the relational data model. It provides a set of operators that take one or two relations as input and produce a new relation as output. The operations are closed under the relational model, meaning the result is always a relation. Relational algebra forms the theoretical foundation for SQL and other relational query languages.

## 3.1 Selection ( $\sigma$ )

The selection operator extracts tuples that satisfy a given predicate from a relation. It is a unary operation denoted by  $\sigma_{\text{condition}}(R)$ , where  $R$  is a relation and condition is a logical expression composed of attributes, constants, comparison operators ( $=, \neq, <, \leq, >, \geq$ ), and logical connectives ( $\wedge, \vee, \neg$ ).

**Example:**  $\sigma_{\text{salary} > 50000 \wedge \text{department} = \text{'Sales'}}(\text{Employee})$  returns all employees whose salary exceeds 50,000 and who work in the Sales department.

Diagram:



Sample data:

| emp_id | name  | department | salary |
|--------|-------|------------|--------|
| 101    | Alice | Sales      | 60000  |
| 102    | Bob   | Sales      | 45000  |
| 103    | Carol | IT         | 70000  |

$\sigma_{\text{salary} > 50000 \wedge \text{department} = \text{'Sales'}}(\text{Employee}) \rightarrow$

| emp_id | name | department | salary |
|--------|------|------------|--------|
|--------|------|------------|--------|



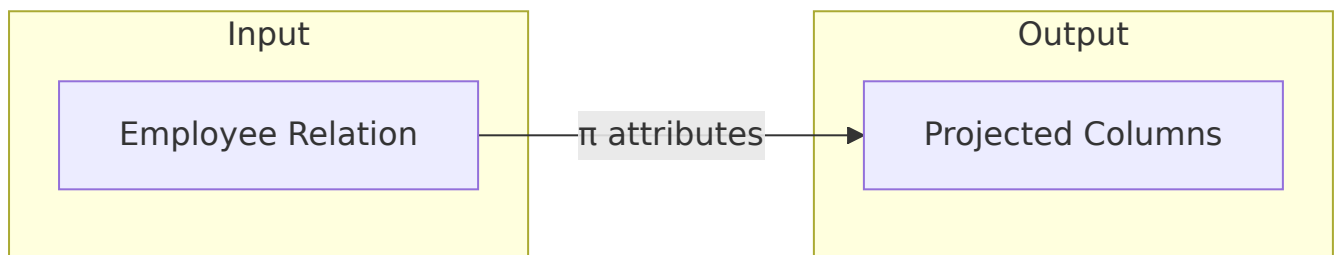
|     |       |       |       |
|-----|-------|-------|-------|
| 101 | Alice | Sales | 60000 |
|-----|-------|-------|-------|

## 3.2 Projection ( $\pi$ )

The projection operator selects a specified subset of attributes from a relation, eliminating duplicate tuples from the result. It is denoted by  $\pi_{A_1, A_2, \dots, A_k}(R)$ .

**Example:**  $\pi_{\text{name, salary}}(\text{Employee})$  returns only name and salary columns.

**Diagram:**



**Sample:**

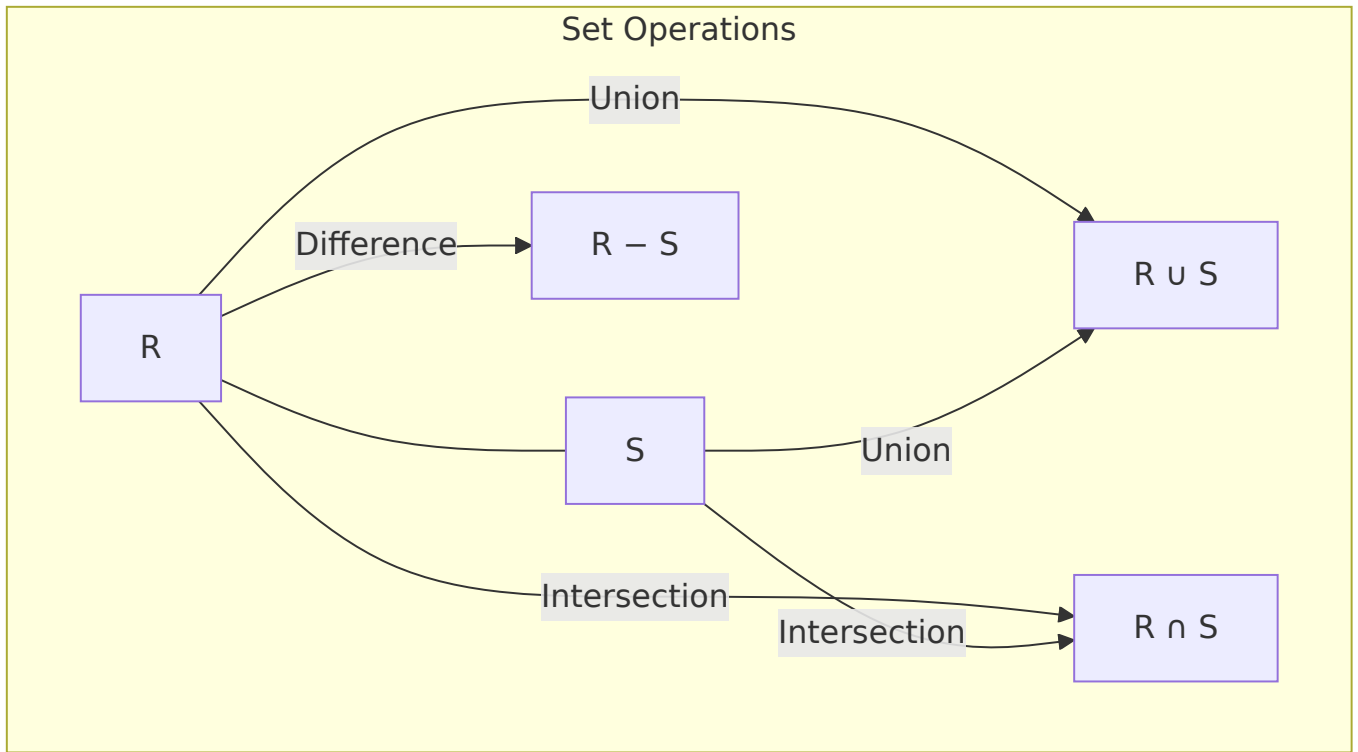
| name  | salary |
|-------|--------|
| Alice | 60000  |
| Bob   | 45000  |
| Carol | 70000  |

## 3.3 Union ( $\cup$ ), Intersection ( $\cap$ ), Difference ( $-$ )

These binary set operations require union-compatible relations (same arity and corresponding domains).

- **Union ( $R \cup S$ ):** Tuples in R or S.
- **Intersection ( $R \cap S$ ):** Tuples in both R and S.
- **Difference ( $R - S$ ):** Tuples in R but not in S.

**Diagram:**



**Example:** Let  $R$  = Sales employees,  $S$  = employees with bonus > 5000.

| emp_id | name  | dept  |
|--------|-------|-------|
| 101    | Alice | Sales |
| 102    | Bob   | Sales |
| 104    | David | Sales |

| emp_id | name  | bonus |
|--------|-------|-------|
| 101    | Alice | 6000  |
| 103    | Carol | 7000  |

After projecting onto common attributes (emp\_id, name):

$R \cup S \rightarrow$  Alice, Bob, David, Carol

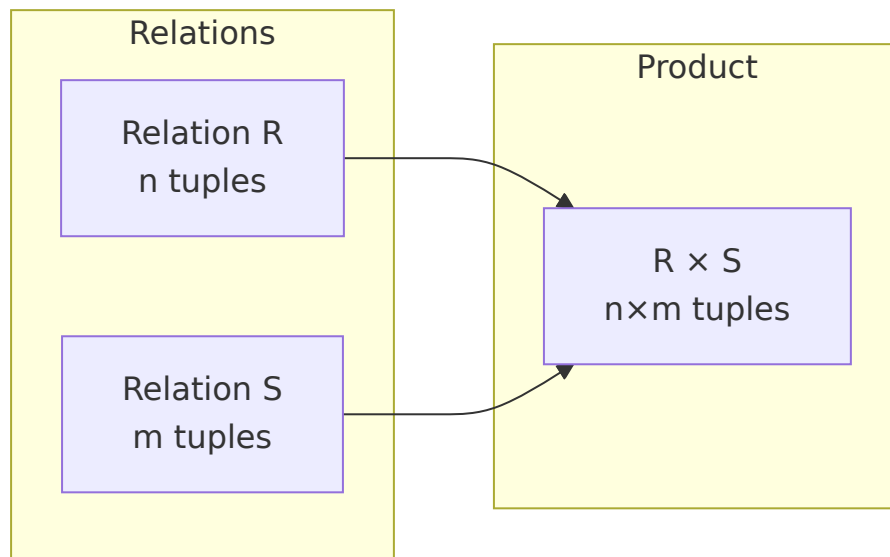
$R \cap S \rightarrow$  Alice

$R - S \rightarrow$  Bob, David

## 3.4 Cartesian Product ( $\times$ )

$R \times S$  combines every tuple of R with every tuple of S. Arity =  $\text{arity}(R) + \text{arity}(S)$ , cardinality =  $|R| \times |S|$ .

**Diagram:**



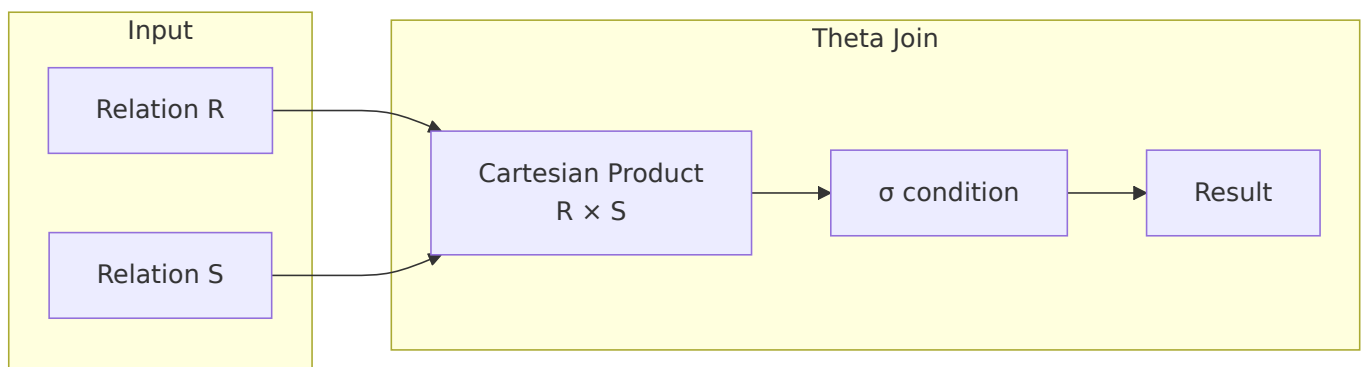
**Example:** Employee (2 tuples)  $\times$  Department (2 tuples) yields 4 tuples.

## 3.5 Joins

### 3.5.1 Theta Join ( $\bowtie_{\theta}$ )

$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$ . The condition  $\theta$  may involve any comparison operators.

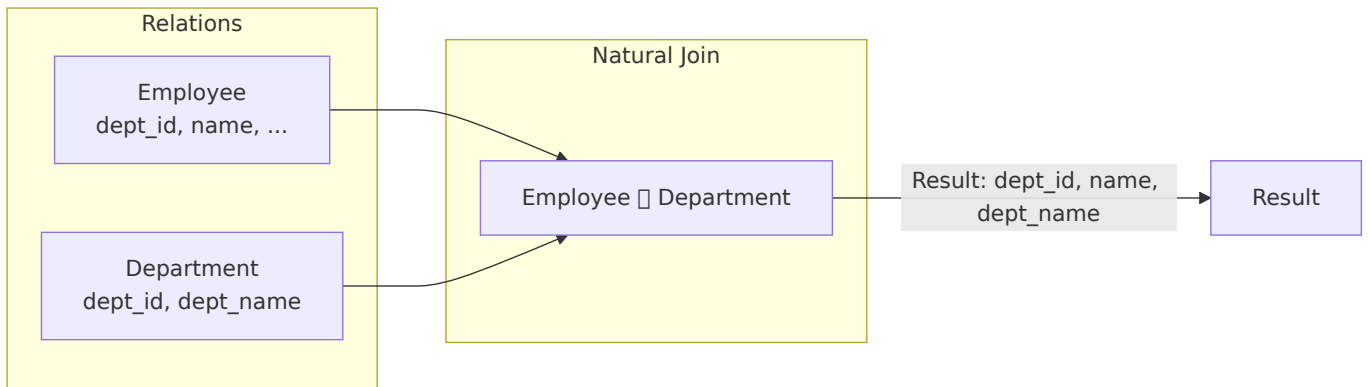
**Diagram:**



### 3.5.2 Natural Join ( $\Join$ )

Natural join equates all attributes with the same name and removes duplicate columns. It is a special case of theta join with equality on common attributes.

**Diagram:**

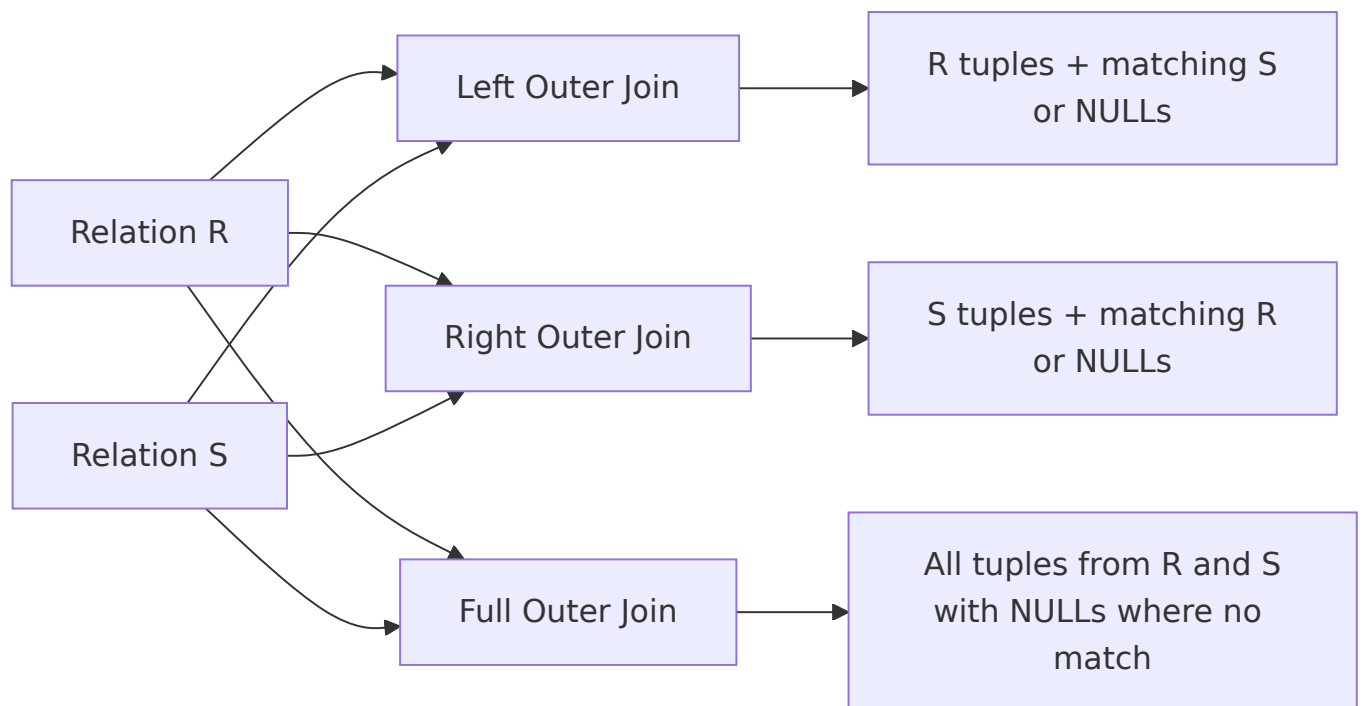


### 3.5.3 Outer Join

Outer joins preserve non-matching tuples by padding with NULLs.

- **Left Outer Join ( $R \bowtie S$ ):** All tuples from R.
- **Right Outer Join ( $R \bowtie S$ ):** All tuples from S.
- **Full Outer Join ( $R \bowtie S$ ):** All tuples from both.

**Diagram:**



**Example** (Left Outer Join):

Employee (dept\_id may be NULL)  $\bowtie$  Department:

| emp_id | name  | dept_id | dept_name |
|--------|-------|---------|-----------|
| 101    | Alice | D1      | Sales     |

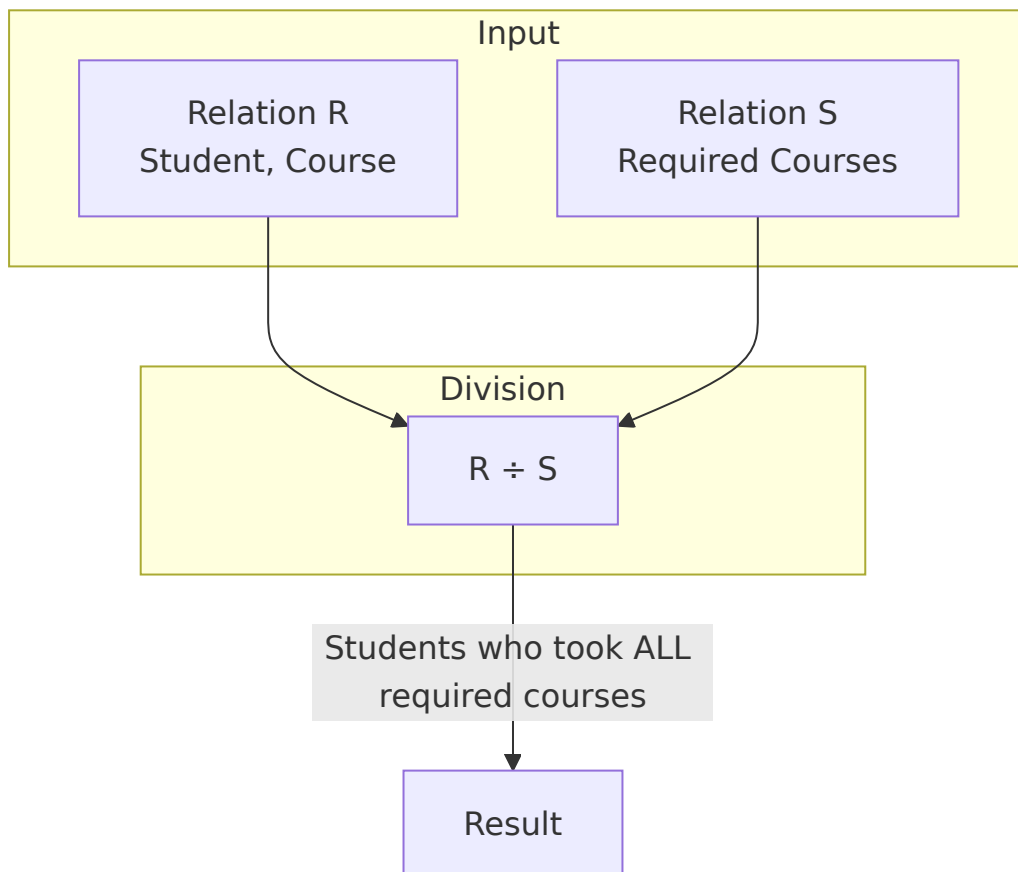
|     |       |      |      |
|-----|-------|------|------|
| 102 | Bob   | NULL | NULL |
| 103 | Carol | D2   | IT   |

## 3.6 Division Operator ( $\div$ )

$R \div S$  returns tuples from R (excluding the attributes of S) that are associated with **every** tuple in S. It answers “for all” queries.

**Formal definition:** Given  $R(A, B)$  and  $S(B)$ ,  
 $R \div S = \{ a \in \pi_A(R) \mid \forall s \in S, (a, s) \in R \}$

**Diagram:**



**Example:**

$R$  (Student, Course):

| Student | Course |
|---------|--------|
| Alice   | Math   |

|       |           |
|-------|-----------|
| Alice | Physics   |
| Bob   | Math      |
| Carol | Math      |
| Carol | Physics   |
| Carol | Chemistry |

S (Course):

| Course  |
|---------|
| Math    |
| Physics |

$R \div S \rightarrow$  Students who took both Math and Physics:

| Student |
|---------|
| Alice   |
| Carol   |

## 3.7 Summary

Relational algebra provides a minimal, complete set of operations for manipulating relations. The fundamental operations are selection, projection, Cartesian product, union, and difference. Intersection, joins, and division can be derived from these primitives but are included for convenience. Understanding relational algebra is essential for query optimization and for formally reasoning about database queries.

---

# Chapter 4: Structured Query Language (SQL)

SQL is the standard language for defining, manipulating, and querying relational databases. It comprises several sublanguages: Data Definition Language (DDL) for schema management, Data Manipulation Language (DML) for data operations, and Data Control Language (DCL) for access control. This chapter focuses on DDL and DML, along with advanced query features.

## 4.1 Data Definition Language (DDL)

DDL statements define and modify the database schema. The primary DDL commands are CREATE, ALTER, and DROP.

### 4.1.1 CREATE

The CREATE statement creates databases, tables, indexes, and other objects.

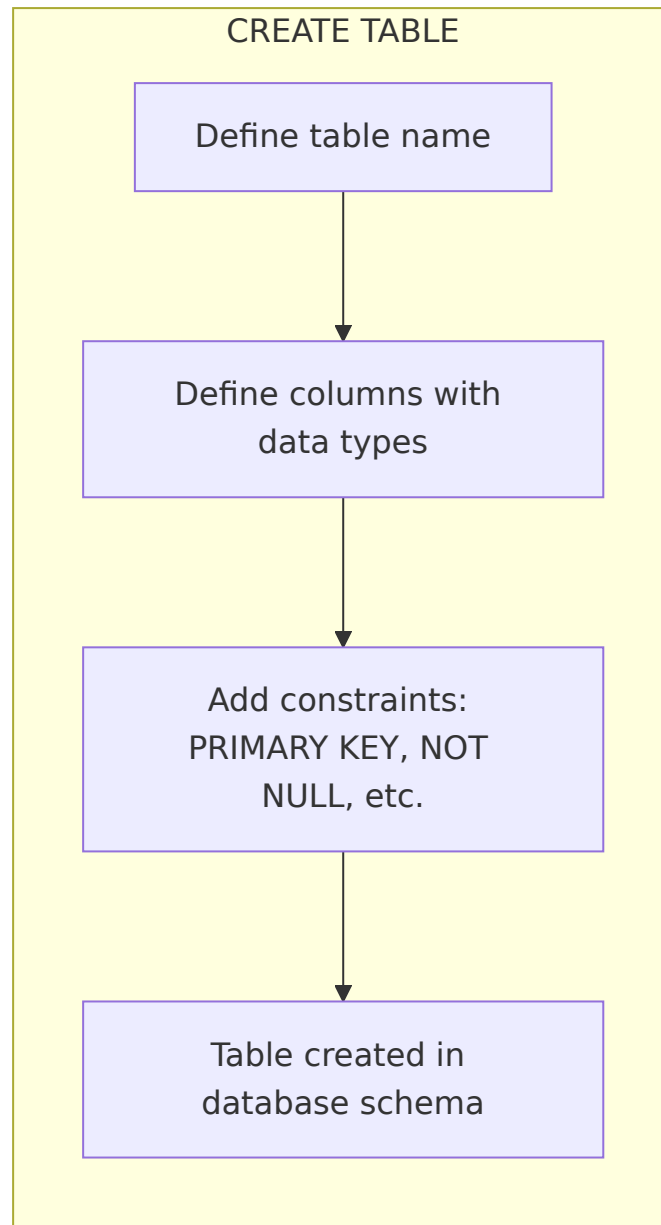
**Syntax for a table:**

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    ...  
    table_constraint  
);
```

**Example:**

```
CREATE TABLE Employee (  
    emp_id INTEGER PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    department_id INTEGER,  
    salary DECIMAL(10,2),  
    hire_date DATE  
);
```

## Diagram:



### 4.1.2 ALTER

ALTER modifies an existing table structure: add, drop, or modify columns; add or drop constraints.

#### Examples:



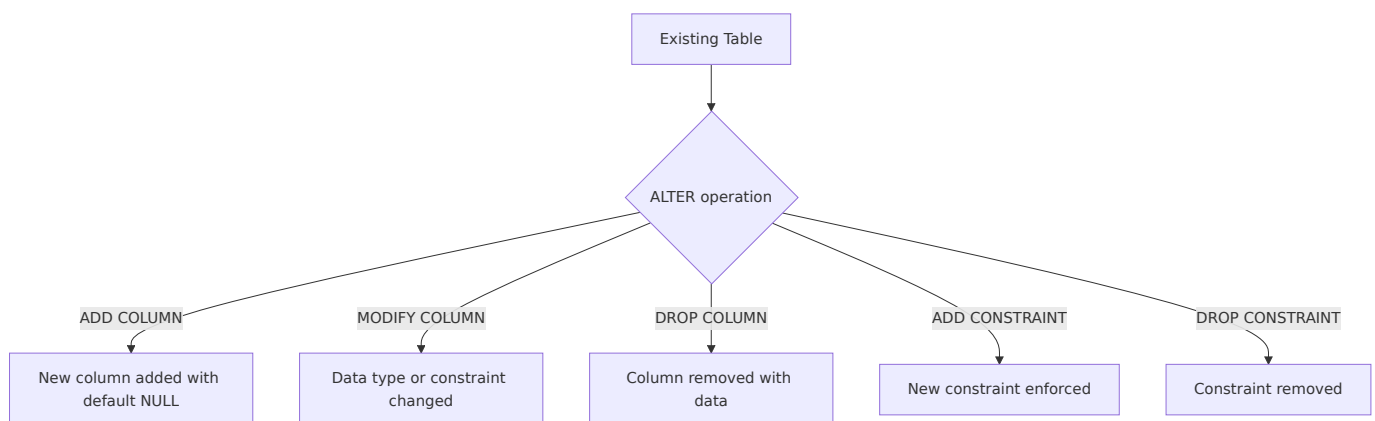
```
-- Add a new column
ALTER TABLE Employee ADD email VARCHAR(100);

-- Modify a column data type
ALTER TABLE Employee MODIFY COLUMN salary NUMERIC(12,2);

-- Drop a column
ALTER TABLE Employee DROP COLUMN hire_date;

-- Add a foreign key constraint
ALTER TABLE Employee ADD CONSTRAINT fk_dept
    FOREIGN KEY (department_id) REFERENCES Department(dept_id);
```

Diagram:



### 4.1.3 DROP

DROP removes tables, databases, or other objects entirely.

Example:

```
DROP TABLE Employee; -- Removes table and all data
```

**Caution:** DROP is irreversible unless a backup exists. Use TRUNCATE to remove data only while retaining the table structure.

## 4.2 Data Manipulation Language (DML)

DML commands handle data within tables: SELECT (query), INSERT (add), UPDATE (modify), DELETE (remove).

## 4.2.1 SELECT

SELECT retrieves data from one or more tables. Basic form:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
ORDER BY column;
```

**Example:**

```
SELECT name, salary  
FROM Employee  
WHERE department_id = 101  
ORDER BY salary DESC;
```

**Diagram:**



## 4.2.2 INSERT

INSERT adds new rows to a table.

**Full row insert:**

```
INSERT INTO Employee (emp_id, name, department_id, salary)  
VALUES (201, 'John Doe', 101, 55000);
```

**Insert from a query:**

```
INSERT INTO HighEarners (emp_id, name, salary)  
SELECT emp_id, name, salary FROM Employee WHERE salary > 70000;
```

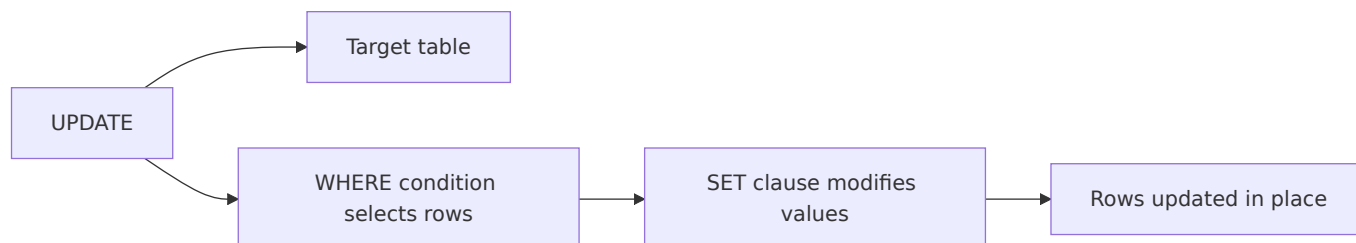
## 4.2.3 UPDATE

UPDATE modifies existing rows based on a condition.

**Example:**

```
UPDATE Employee
SET salary = salary * 1.10
WHERE department_id = 101;
```

**Diagram:**



## 4.2.4 DELETE

DELETE removes rows that satisfy a condition. Without a WHERE clause, all rows are deleted.

**Example:**

```
DELETE FROM Employee
WHERE hire_date < '2020-01-01';
```

## 4.3 Joins

Joins combine rows from two or more tables based on a related column. SQL supports inner joins and outer joins (left, right, full).

### 4.3.1 Inner Join

Returns only rows where the join condition matches in both tables.

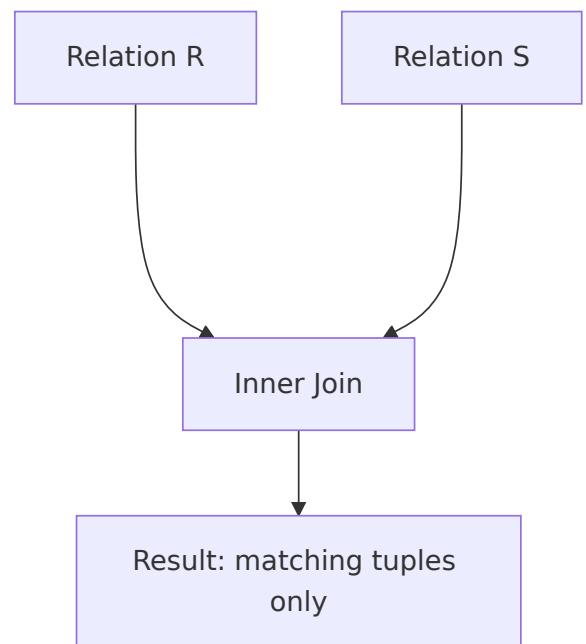
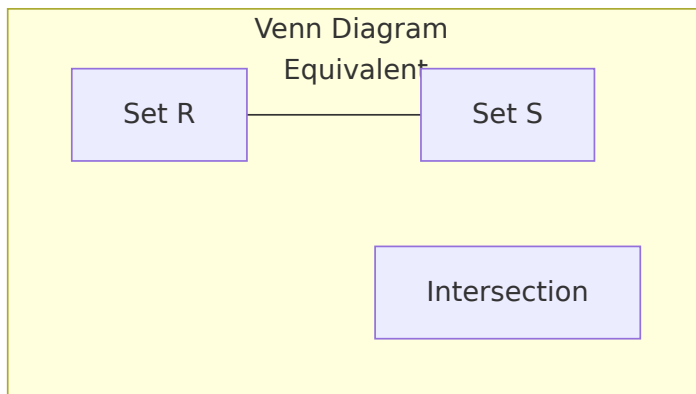
**Syntax:**

```
SELECT columns
FROM table1
INNER JOIN table2 ON table1.key = table2.key;
```

**Example:**

```
SELECT Employee.name, Department.dept_name
FROM Employee
INNER JOIN Department ON Employee.dept_id = Department.dept_id;
```

**Diagram:**



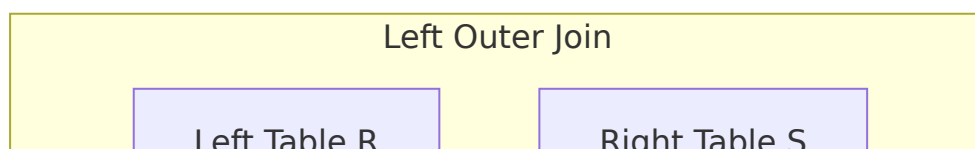
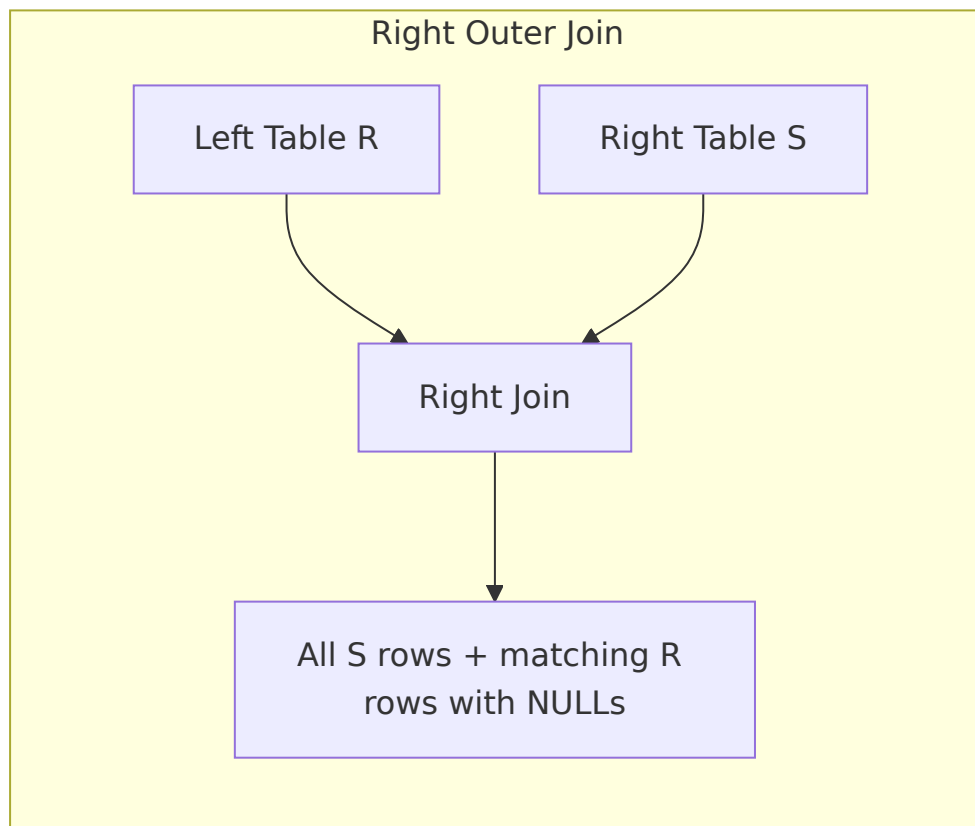
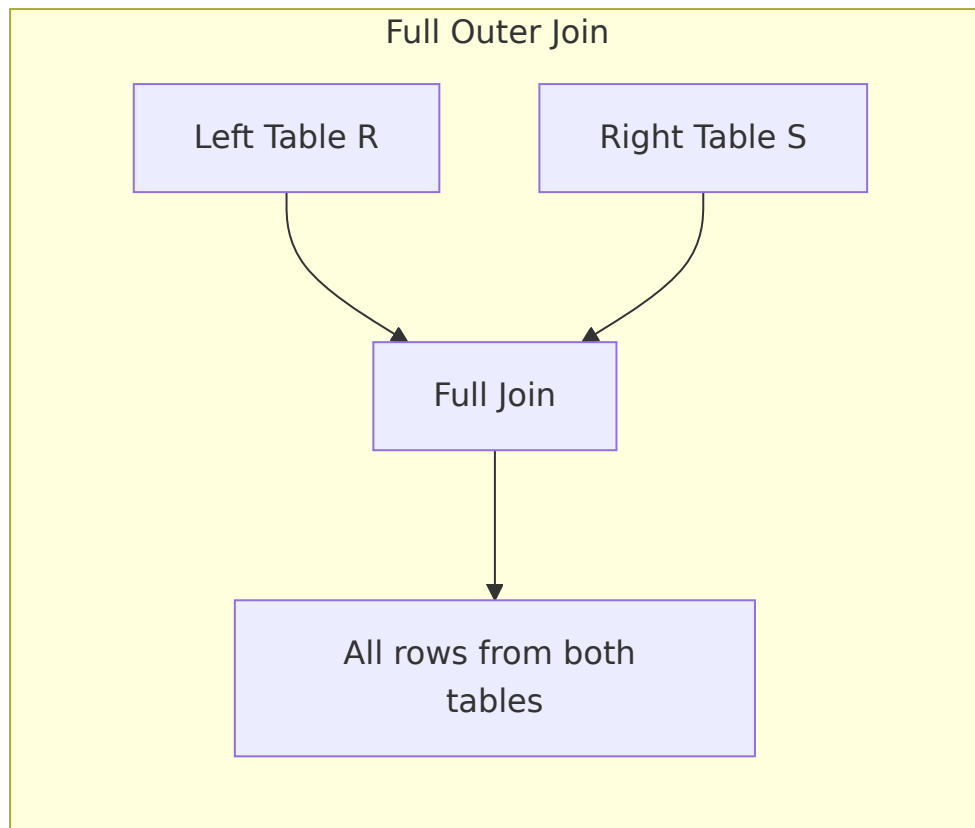
### 4.3.2 Outer Joins

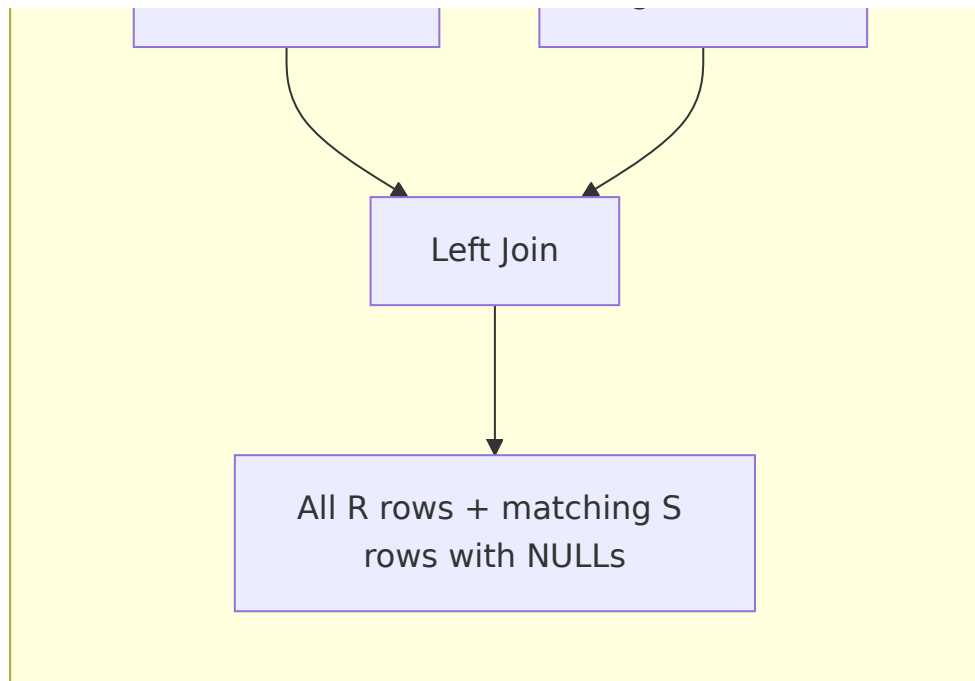
- **Left Outer Join:** All rows from the left table; unmatched rows from the right table contain NULLs.
- **Right Outer Join:** All rows from the right table; unmatched rows from the left table contain NULLs.
- **Full Outer Join:** All rows from both tables; NULLs where no match exists.

**Left Outer Join Example:**

```
SELECT Employee.name, Department.dept_name
FROM Employee
LEFT OUTER JOIN Department ON Employee.dept_id = Department.dept_id;
```

**Diagram:**





## 4.4 Nested Queries (Subqueries)

A subquery is a query nested inside another SQL statement. Subqueries can appear in SELECT, FROM, WHERE, or HAVING clauses. They may be correlated (reference outer query) or non-correlated.

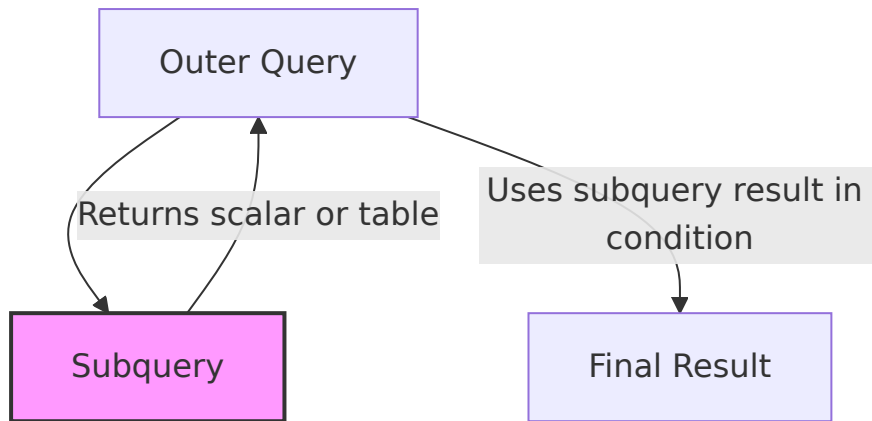
**Example: Subquery in WHERE clause:**

```
SELECT name, salary
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

**Example: Correlated subquery** (finds employees earning above average in their own department):

```
SELECT e1.name, e1.salary, e1.department_id
FROM Employee e1
WHERE e1.salary > (SELECT AVG(e2.salary)
                  FROM Employee e2
                  WHERE e2.department_id = e1.department_id);
```

**Diagram:**



## 4.5 Aggregation

Aggregate functions (COUNT, SUM, AVG, MIN, MAX) compute summary values. GROUP BY groups rows that share attribute values, and HAVING filters groups.

### 4.5.1 GROUP BY

Groups rows by one or more columns. Aggregate functions apply per group.

**Example:**

```
SELECT department_id, COUNT(*) AS num_employees, AVG(salary) AS
avg_salary
FROM Employee
GROUP BY department_id;
```

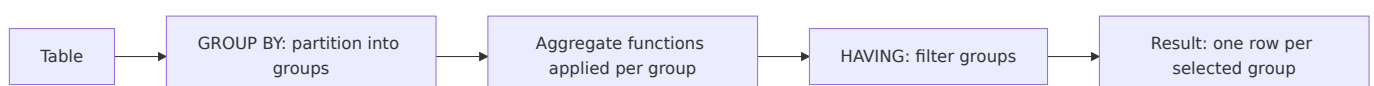
### 4.5.2 HAVING

HAVING filters groups after aggregation, similar to WHERE for rows before grouping.

**Example:**

```
SELECT department_id, AVG(salary) AS avg_salary
FROM Employee
GROUP BY department_id
HAVING AVG(salary) > 60000;
```

**Diagram:**



## 4.6 Views

A view is a virtual table defined by a stored query. It does not store data physically but presents a customized perspective of the underlying tables.

**Create a view:**

```
CREATE VIEW HighEarners AS  
SELECT emp_id, name, salary  
FROM Employee  
WHERE salary > 70000;
```

**Using a view:**

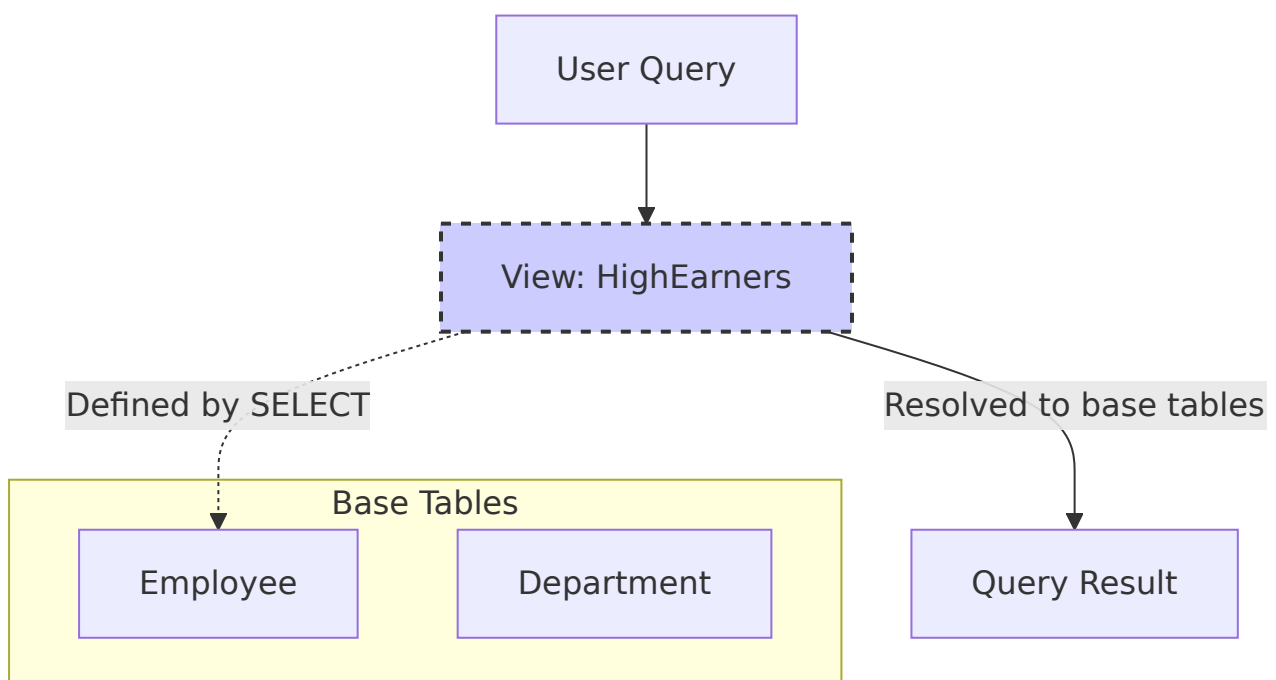
```
SELECT * FROM HighEarners WHERE department_id = 101;
```

**Updateable views:** Some views are updateable (e.g., simple views on a single table without aggregation). Others are read-only.

**Drop a view:**

```
DROP VIEW HighEarners;
```

**Diagram:**





## 4.7 Constraints

Constraints enforce data integrity rules at the table or column level. They are part of DDL but can also be added via ALTER.

| Constraint  | Description                                                                       |
|-------------|-----------------------------------------------------------------------------------|
| PRIMARY KEY | Uniquely identifies each row; automatically NOT NULL and unique.                  |
| FOREIGN KEY | Enforces referential integrity; values must match a primary key in another table. |
| UNIQUE      | Ensures all values in a column (or combination) are distinct.                     |
| NOT NULL    | Prevents NULL values.                                                             |
| CHECK       | Validates that values satisfy a Boolean expression.                               |
| DEFAULT     | Provides a default value when none is supplied.                                   |

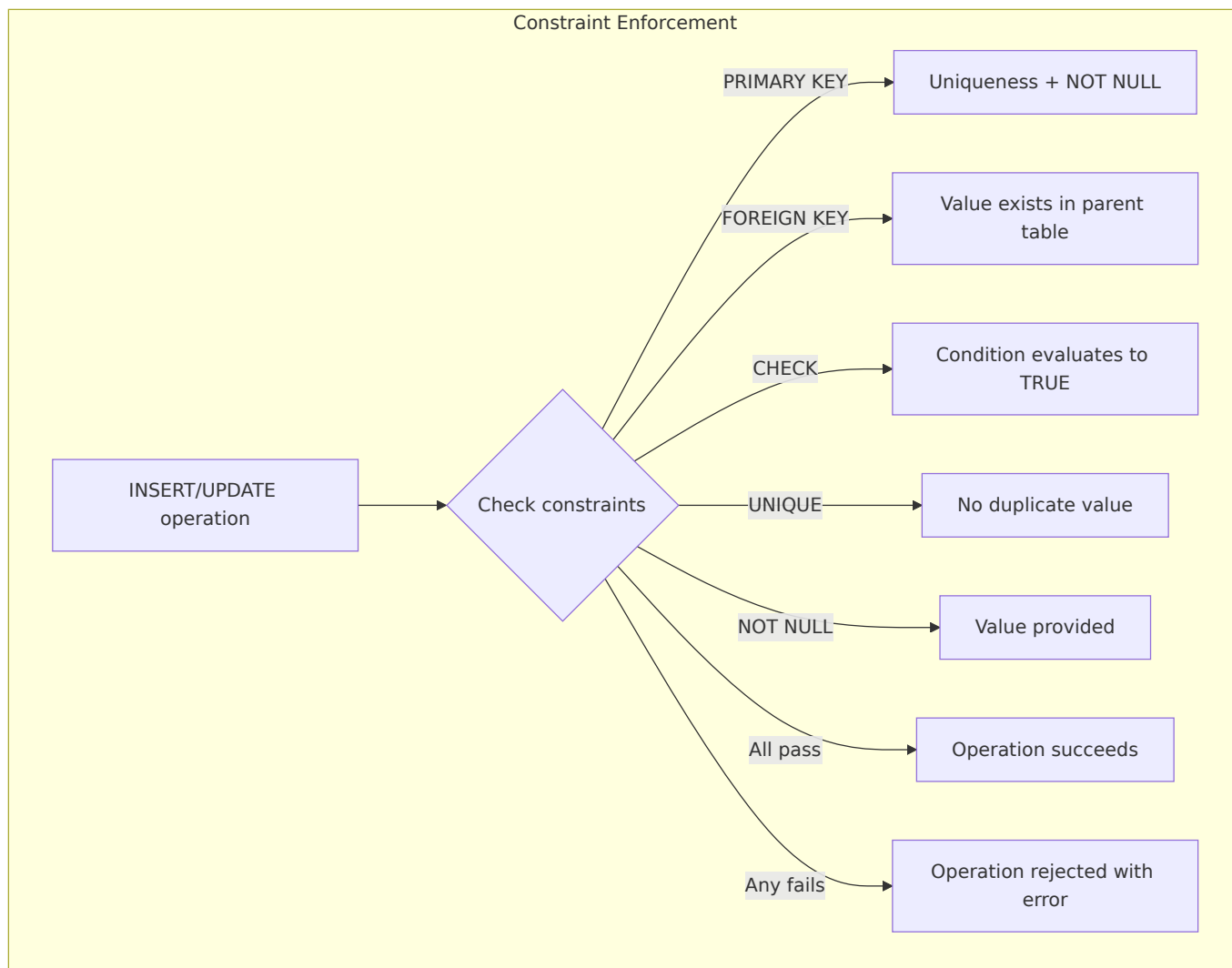
**Example with multiple constraints:**

```
CREATE TABLE Orders (  
    order_id INTEGER PRIMARY KEY,  
    customer_id INTEGER NOT NULL,  
    order_date DATE DEFAULT CURRENT_DATE,  
    amount DECIMAL(10,2) CHECK (amount > 0),  
    status VARCHAR(20) CHECK (status IN ('Pending', 'Shipped',  
    'Delivered')),  
    FOREIGN KEY (customer_id) REFERENCES Customers(cust_id)  
);
```

**Adding a constraint via ALTER:**

```
ALTER TABLE Employee ADD CONSTRAINT unique_email UNIQUE (email);
```

**Diagram:**



## 4.8 Summary

SQL provides comprehensive capabilities for defining schema (DDL) and manipulating data (DML). Key constructs include:

- **DDL:** CREATE, ALTER, DROP for tables and constraints.
- **DML:** SELECT, INSERT, UPDATE, DELETE for data.
- **Joins:** Inner, left outer, right outer, full outer.
- **Nested queries:** Subqueries in WHERE, FROM, SELECT clauses.
- **Aggregation:** GROUP BY with HAVING for grouped filtering.
- **Views:** Virtual tables for security and abstraction.
- **Constraints:** Enforce integrity (PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE, NOT NULL, DEFAULT).

# Chapter 5: Entity-Relationship (ER) Model

The Entity-Relationship model is a high-level conceptual data model used to describe the structure of a database. It provides a graphical notation for representing entities, attributes, and relationships, facilitating communication between database designers and stakeholders. The ER model serves as a blueprint for implementing a relational database.

## 5.1 Entity and Attribute

### 5.1.1 Entity

An entity is a real-world object or concept that is distinguishable from other objects. Entities are represented as rectangles in ER diagrams. A set of entities of the same type is called an entity set.

**Example:** Employee, Department, Project.

### 5.1.2 Attribute

Attributes describe properties of entities. They are represented as ovals connected to their entity. Common attribute types:

- **Simple attribute:** Indivisible (e.g., age, salary).
- **Composite attribute:** Can be subdivided (e.g., address into street, city, zip).
- **Derived attribute:** Computed from other attributes (e.g., age from birth\_date).
- **Multi-valued attribute:** May have multiple values (e.g., phone\_numbers).

**Diagram:**

| EMPLOYEE |               |    |              |
|----------|---------------|----|--------------|
| int      | emp_id        | PK |              |
| string   | name          |    |              |
| date     | birth_date    |    |              |
| int      | age           |    | derived      |
| string   | phone_numbers |    | multi-valued |

### Explanation of notation:

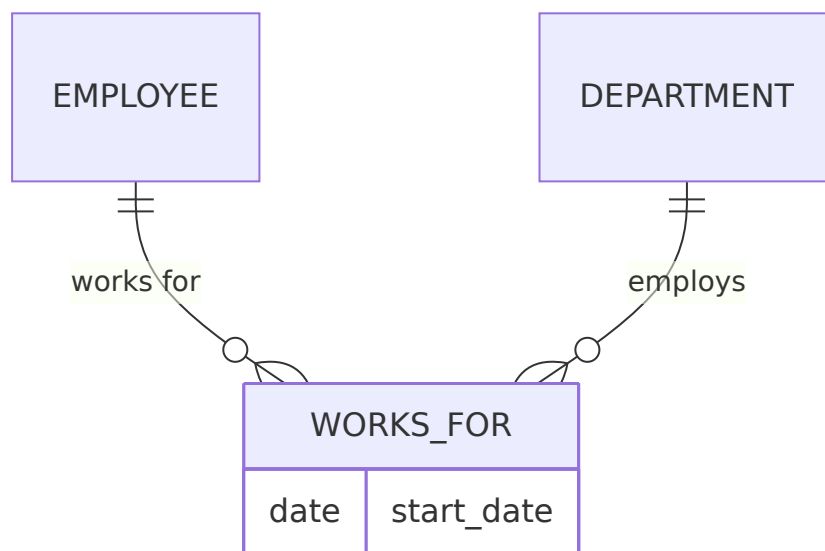
- **PK** indicates primary key (underlined in traditional ER, here shown as comment).
- Derived attributes are often shown with dashed ovals; multi-valued with double ovals.

## 5.2 Relationship

A relationship is an association among two or more entities. It is represented as a diamond in ER diagrams. The degree of a relationship is the number of participating entity sets (binary, ternary, etc.).

**Example:** Employee **works\_for** Department.

**Diagram:**



This shows a binary relationship with an attribute `start_date` on the relationship.

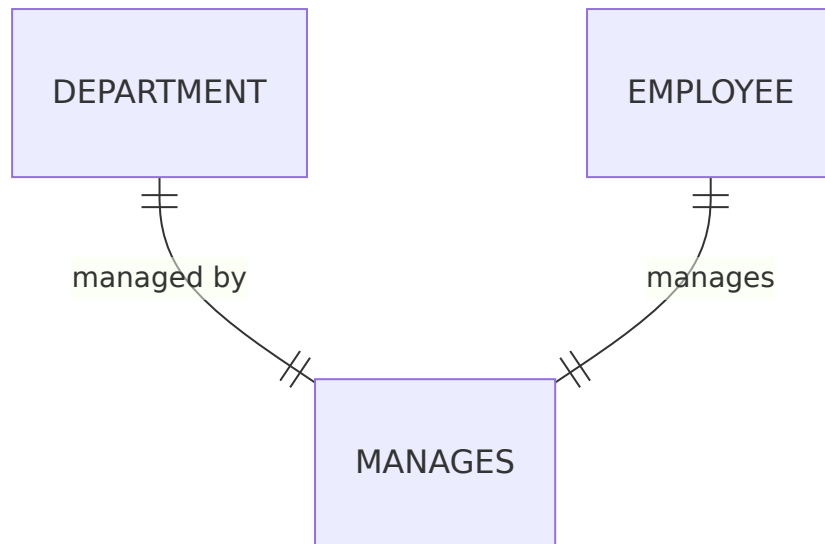
## 5.3 Cardinality

Cardinality specifies the number of instances of one entity that can associate with instances of another entity through a relationship. Common cardinalities: 1:1, 1:N, M:N.

### 5.3.1 One-to-One (1:1)

Each entity in set A is associated with at most one entity in set B, and vice versa.

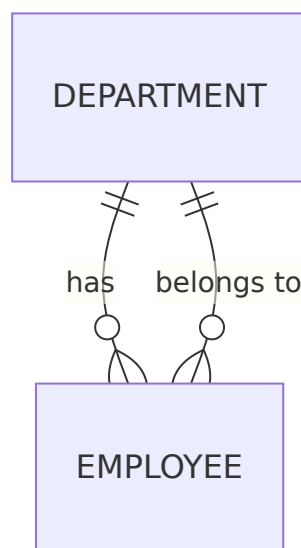
**Example:** Each department has one manager, and each manager manages exactly one department (assuming managers manage only one department).



### 5.3.2 One-to-Many (1:N)

An entity in A can be associated with many entities in B, but an entity in B is associated with at most one entity in A.

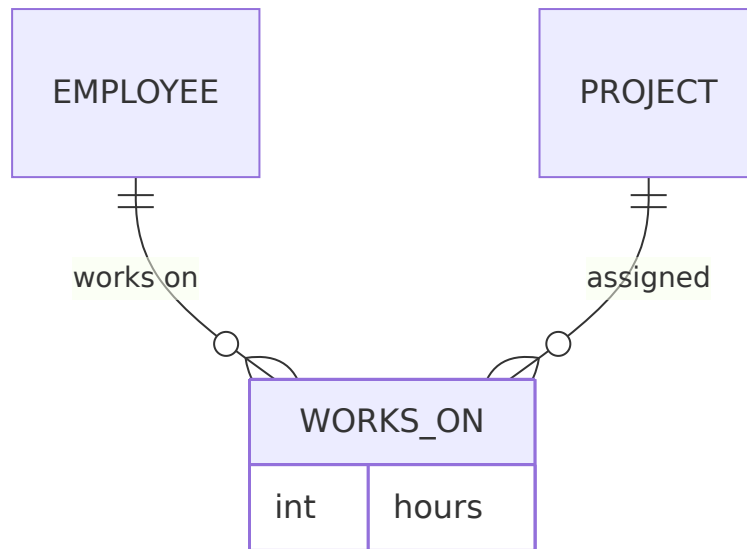
**Example:** A department has many employees, but each employee works for exactly one department.



### 5.3.3 Many-to-Many (M:N)

Entities in A can be associated with many entities in B, and vice versa.

**Example:** Employees work on many projects, and projects have many employees.



**Cardinality notation summary** (using Mermaid's crow's foot notation):

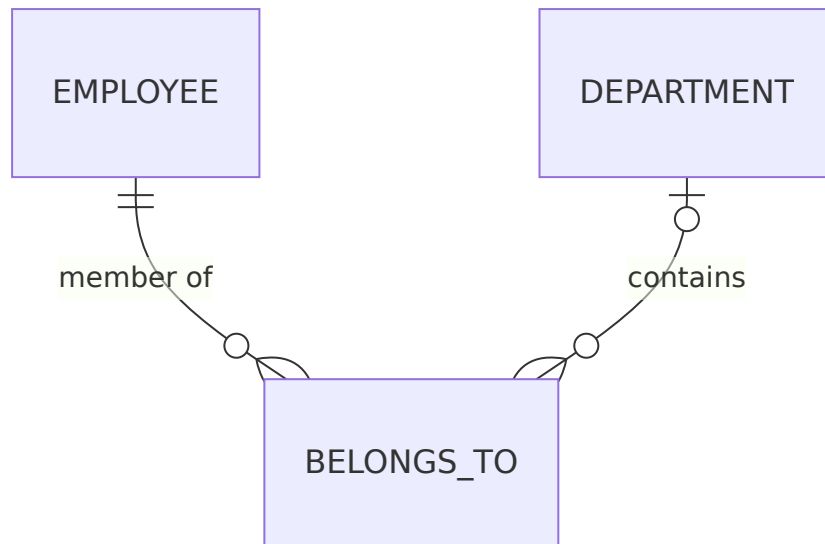
| Symbol | Meaning      |
|--------|--------------|
|        | exactly one  |
| o      | zero or one  |
| }      | one or more  |
| o{     | zero or more |

## 5.4 Participation Constraints

Participation constraints specify whether every entity in a set must participate in a relationship.

- **Total participation (mandatory):** Every entity in the set appears in at least one relationship instance. Represented by a double line.
- **Partial participation (optional):** Some entities may not participate. Represented by a single line.

**Example:** Every employee must belong to a department (total participation of Employee in `belongs_to`). A department may have no employees (partial participation of Department).

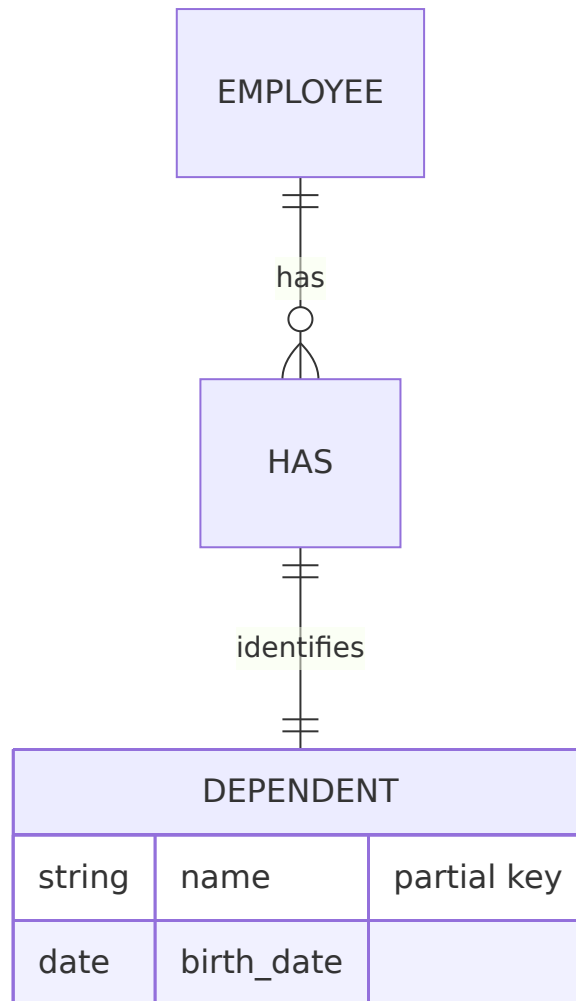


Here, the double line ( || ) next to EMPLOYEE indicates total participation; the single line ( | o ) next to DEPARTMENT indicates partial participation.

## 5.5 Weak Entity

A weak entity is an entity that cannot be uniquely identified by its own attributes alone; it depends on a **identifying relationship** with a **strong entity** (owner). Weak entities are represented as double rectangles. The identifying relationship is shown as a double diamond. The weak entity's partial key (discriminator) is underlined with a dashed line.

**Example:** A `Dependent` cannot exist without an `Employee`. The combination of employee's primary key and dependent's name identifies the dependent.



In traditional ER notation, the weak entity and identifying relationship are drawn with double borders. Mermaid does not directly support double borders, but the concept is explained.

## 5.6 Mapping ER to Relational Schema

The process of converting an ER diagram into a set of relational tables (relations) involves rules for each component.

### 5.6.1 Mapping Strong Entities

Create a relation for each strong entity set. Include all simple attributes as columns. Choose a primary key from the entity's key attributes.

**Example:** `Employee(emp_id, name, birth_date)` – `emp_id` as primary key.

### 5.6.2 Mapping Weak Entities

Create a relation for the weak entity. Include its partial key attributes and the primary key of the owner entity as a foreign key. The composite primary key is the combination of the



owner' s primary key and the weak entity' s partial key.

**Example:** `Dependent(emp_id, dependent_name, birth_date)` where `(emp_id, dependent_name)` is the primary key, and `emp_id` references `Employee(emp_id)`.

### 5.6.3 Mapping Relationships

The mapping depends on cardinality and participation.

#### Binary 1:1 Relationship:

- Choose one entity' s relation and add the other entity' s primary key as a foreign key. Include any relationship attributes.
- Alternatively, create a separate relation for the relationship.

#### Binary 1:N Relationship:

- Add the primary key of the entity on the “1” side as a foreign key in the relation of the entity on the “N” side. Include any relationship attributes.

#### Binary M:N Relationship:

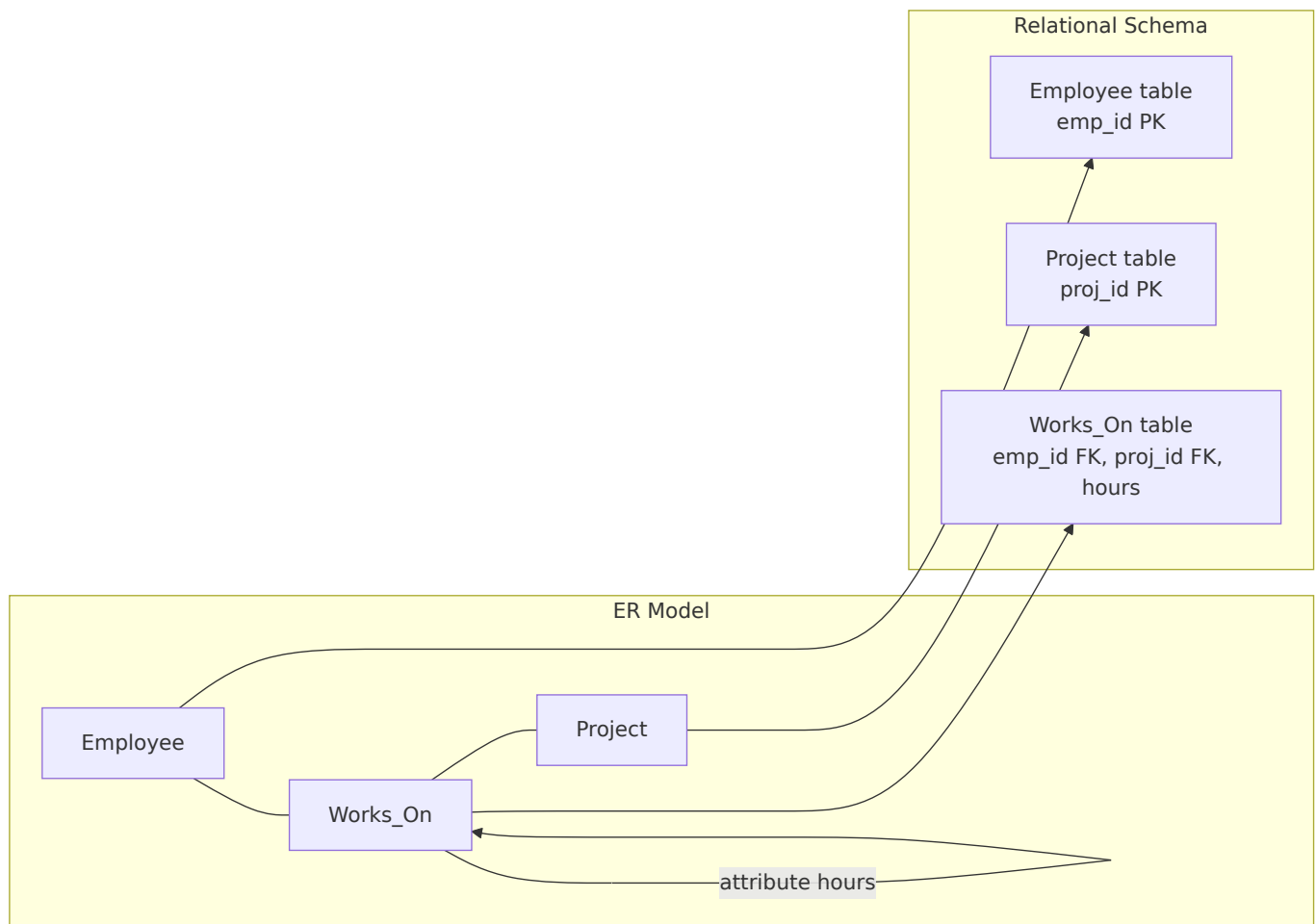
- Create a new relation representing the relationship. Include the primary keys of both participating entities as foreign keys. Their combination forms the primary key. Include any relationship attributes.

#### Example:

ER: `Employee (emp_id)` – `Works_On (hours)` – `Project (proj_id)`. Mapping:

```
Employee(emp_id, name, ...)
Project(proj_id, title, ...)
Works_On(emp_id, proj_id, hours)
```

#### Diagram of mapping process:



## 5.6.4 Mapping Multi-valued Attributes

Create a separate relation for each multi-valued attribute. Include the primary key of the owner entity as a foreign key. The primary key is the combination of the owner key and the attribute value.

**Example:** Employee has multi-valued attribute `phone_numbers`. Map to:

```
Employee_Phone(emp_id, phone_number)
```

## 5.6.5 Mapping Composite Attributes

Flatten composite attributes into their constituent simple attributes within the owner's relation. No separate table is needed.

**Example:** Address(street, city, zip) becomes columns `street`, `city`, `zip` in the entity's table.

## 5.6.6 Mapping Derived Attributes

Derived attributes are not stored in the relational schema. They are computed when needed via queries or views.

## 5.6.7 Complete Mapping Example

**ER Diagram** (conceptual):

Entities: Employee(emp\_id, name, birth\_date), Department(dept\_id, dept\_name, budget).  
Relationship: Employee works\_for Department (1:N, total participation of Employee). Multi-valued: Employee has phone\_numbers.

**Mapping result:**

```
CREATE TABLE Employee (  
    emp_id INTEGER PRIMARY KEY,  
    name VARCHAR(100),  
    birth_date DATE,  
    dept_id INTEGER NOT NULL,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);  
  
CREATE TABLE Department (  
    dept_id INTEGER PRIMARY KEY,  
    dept_name VARCHAR(100),  
    budget DECIMAL(12,2)  
);  
  
CREATE TABLE Employee_Phone (  
    emp_id INTEGER,  
    phone_number VARCHAR(20),  
    PRIMARY KEY (emp_id, phone_number),  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)  
);
```

## 5.7 Summary

The ER model provides a clear, visual way to design databases before implementation. Key concepts:

- **Entities** (strong and weak) and **attributes** (simple, composite, derived, multi-valued).

- **Relationships** with cardinality (1:1, 1:N, M:N) and participation (total/partial).
- **Mapping rules** convert ER diagrams into relational schemas: tables for strong entities, separate tables for M:N relationships and multi-valued attributes, foreign keys for 1:N and 1:1 relationships, and composite primary keys for weak entities.

# Chapter 6: Functional Dependencies

Functional dependencies are a fundamental concept in relational database design. They describe constraints between attributes and form the basis for normalization. Understanding functional dependencies helps identify candidate keys, detect redundancy, and guide schema decomposition.

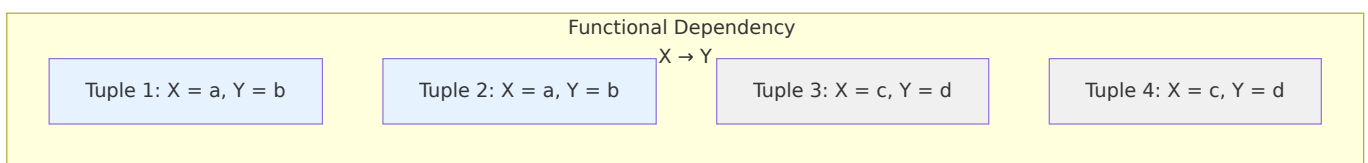
## 6.1 Functional Dependency (FD)

A functional dependency is a constraint between two sets of attributes in a relation. Given a relation  $R$  with attribute sets  $X$  and  $Y$  (subsets of the schema of  $R$ ), we say that  **$X$  functionally determines  $Y$** , denoted  $X \rightarrow Y$ , if for every valid instance of  $R$ , whenever two tuples agree on all attributes in  $X$ , they must also agree on all attributes in  $Y$ .

In other words, the value of  $X$  uniquely determines the value of  $Y$ .

**Example:** In a relation `Employee(emp_id, name, department, salary)`, the dependency `emp_id → name` holds because each employee ID corresponds to exactly one name. Similarly, `emp_id → department` and `emp_id → salary` hold. However, `department → salary` may not hold because multiple employees in the same department can have different salaries.

**Diagram:**



The diagram illustrates that for a given  $X$  value, all tuples have the same  $Y$  value.

## 6.2 Trivial and Non-trivial Functional Dependencies

### 6.2.1 Trivial FD

A functional dependency  $X \rightarrow Y$  is **trivial** if  $Y \subseteq X$ . It holds for any relation because if two tuples agree on all attributes in  $X$ , they automatically agree on any subset of  $X$ .

**Examples:**

- `{emp_id, name} → emp_id` (trivial)

- $\{\text{emp\_id}\} \rightarrow \text{emp\_id}$  (trivial)
- $\{\text{emp\_id}, \text{name}\} \rightarrow \text{name}$  (trivial)

Trivial FDs provide no useful information for database design.

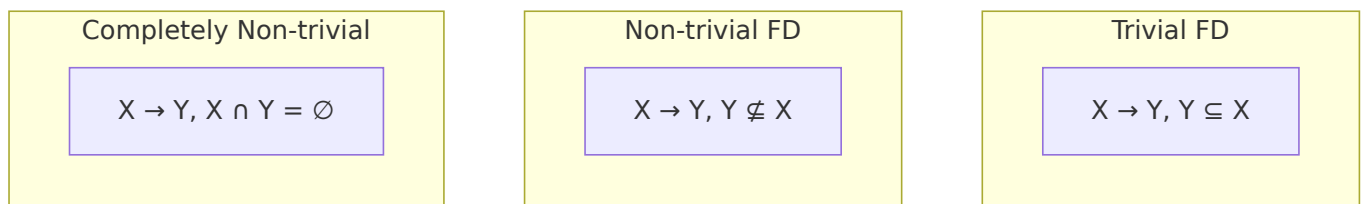
## 6.2.2 Non-trivial FD

An FD  $X \rightarrow Y$  is **non-trivial** if  $Y \not\subseteq X$  (i.e., at least one attribute in  $Y$  is not in  $X$ ). It is **completely non-trivial** if  $X \cap Y = \emptyset$ .

**Examples:**

- $\text{emp\_id} \rightarrow \text{name}$  (non-trivial, and completely non-trivial if  $\text{emp\_id}$  and  $\text{name}$  are disjoint)
- $\text{emp\_id}, \text{department} \rightarrow \text{salary}$  (non-trivial if  $\text{salary}$  is not part of the left side)

**Diagram:**



## 6.3 Closure of Attributes

The **closure** of a set of attributes  $X$  under a set of functional dependencies  $F$ , denoted  $X^+$ , is the set of all attributes that are functionally determined by  $X$ , i.e., all attributes  $A$  such that  $X \rightarrow A$  can be derived from  $F$  using inference rules.

The closure algorithm:

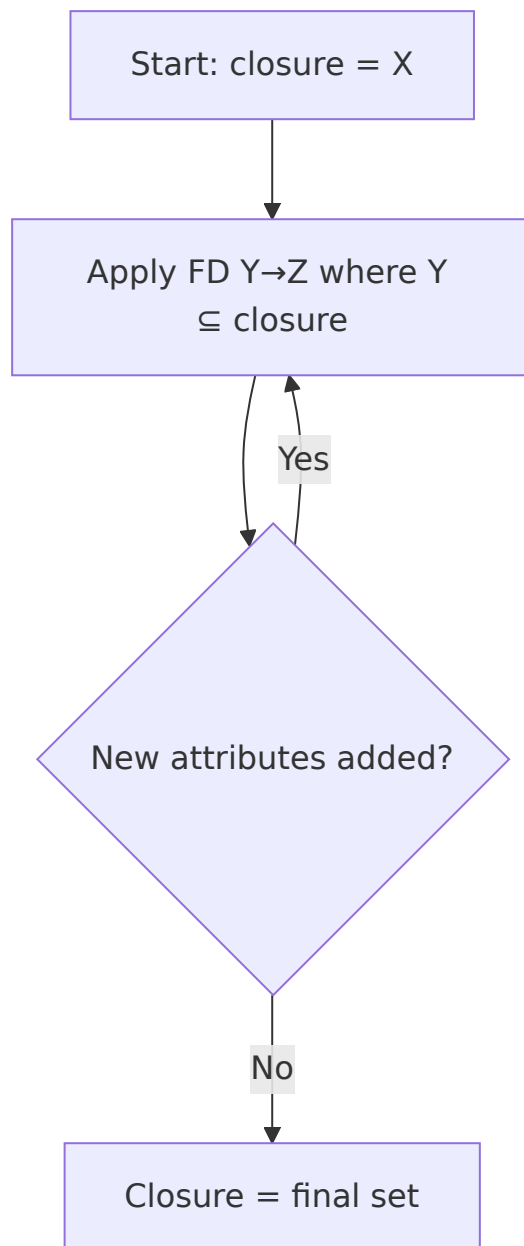
1. Initialize closure =  $X$ .
2. Repeat:
  - For each FD  $Y \rightarrow Z$  in  $F$ , if  $Y \subseteq \text{closure}$ , then add  $Z$  to closure.
3. Continue until no more attributes can be added.

**Example:** Let  $R(A, B, C, D, E, F)$  and  $F = \{A \rightarrow B, B \rightarrow C, AD \rightarrow E, C \rightarrow D\}$ . Compute  $(A)^+$ .

- Start: closure =  $\{A\}$
- $A \rightarrow B$  adds  $B \rightarrow$  closure =  $\{A, B\}$
- $B \rightarrow C$  adds  $C \rightarrow$  closure =  $\{A, B, C\}$
- $C \rightarrow D$  adds  $D \rightarrow$  closure =  $\{A, B, C, D\}$

- $AD \rightarrow E$ : A and D are in closure, so add E  $\rightarrow$  closure = {A, B, C, D, E}
- No more FDs apply. Result:  $(A)^+ = \{A, B, C, D, E\}$

**Diagram:**



## 6.4 Finding Keys Using Attribute Closure

A **candidate key** is a minimal set of attributes that functionally determines all attributes of the relation. To determine if a set  $X$  is a superkey, compute  $X^+$ . If  $X^+$  contains all attributes of the relation, then  $X$  is a superkey.  $X$  is a candidate key if it is a superkey and no proper subset of  $X$  is a superkey.

**Algorithm to find all candidate keys** (for small schemas):

1. Start with all single attributes; compute their closures.

2. If a single attribute's closure contains all attributes, it is a candidate key.
3. Otherwise, consider pairs of attributes, then triples, etc., but only those not containing a previously found candidate key (minimality).
4. Stop when no larger set can be a candidate key (since any superset of a candidate key is a superkey but not minimal).

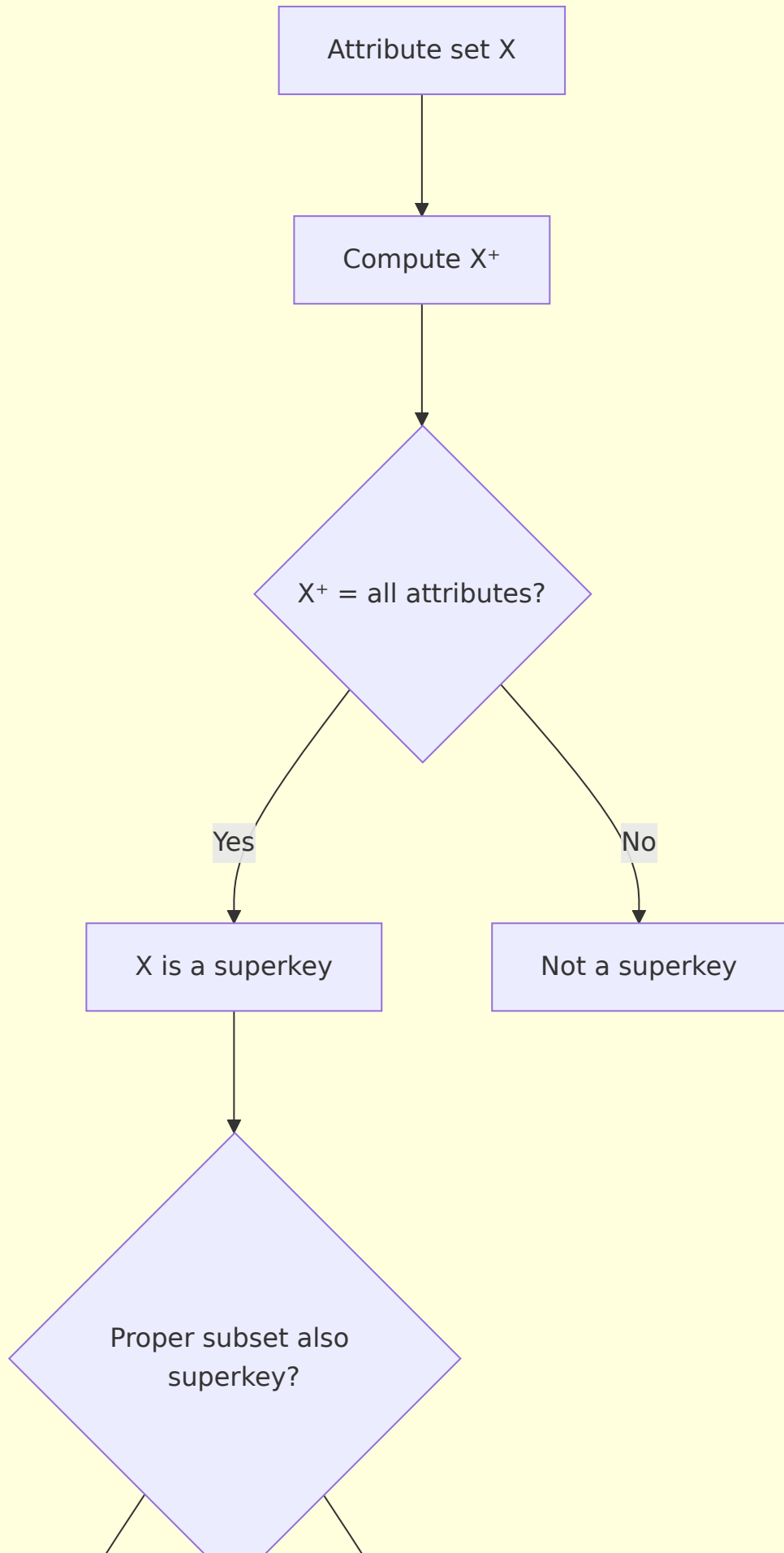
**Example:**  $R(A, B, C, D)$  with  $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$ . Compute closures:

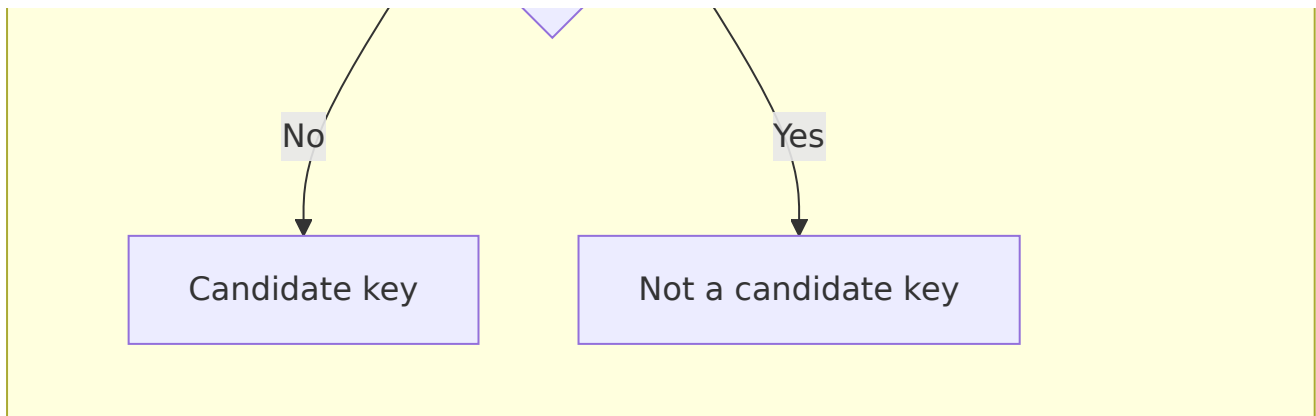
- $(A)^+ = \{A, B, C, D\} \rightarrow A$  is a candidate key.
- $(B)^+ = \{B, C, D\}$  (missing  $A$ )  $\rightarrow$  not a superkey.
- $(C)^+ = \{C, D\} \rightarrow$  no.
- $(D)^+ = \{D\} \rightarrow$  no.
- $(B, C)^+ = \{B, C, D\} \rightarrow$  no.
- Since  $A$  is already a key, no need to consider sets containing  $A$  (they are superkeys but not minimal). Thus only candidate key is  $A$ .

**Diagram:**



## Key Determination





## 6.5 Armstrong's Axioms

Armstrong's axioms are a set of inference rules used to derive all functional dependencies implied by a given set  $F$ . They are sound (generate only implied FDs) and complete (can derive all implied FDs).

### 6.5.1 Primary Axioms

1. **Reflexivity:** If  $Y \subseteq X$ , then  $X \rightarrow Y$ . (Trivial dependencies)
2. **Augmentation:** If  $X \rightarrow Y$ , then  $XZ \rightarrow YZ$  for any  $Z$ . (Adding attributes to both sides preserves dependency)
3. **Transitivity:** If  $X \rightarrow Y$  and  $Y \rightarrow Z$ , then  $X \rightarrow Z$ .

### 6.5.2 Additional Derived Rules

From the primary axioms, we can derive:

- **Union:** If  $X \rightarrow Y$  and  $X \rightarrow Z$ , then  $X \rightarrow YZ$ .
- **Decomposition:** If  $X \rightarrow YZ$ , then  $X \rightarrow Y$  and  $X \rightarrow Z$ .
- **Pseudo-transitivity:** If  $X \rightarrow Y$  and  $YW \rightarrow Z$ , then  $XW \rightarrow Z$ .

### 6.5.3 Example Proof Using Armstrong's Axioms

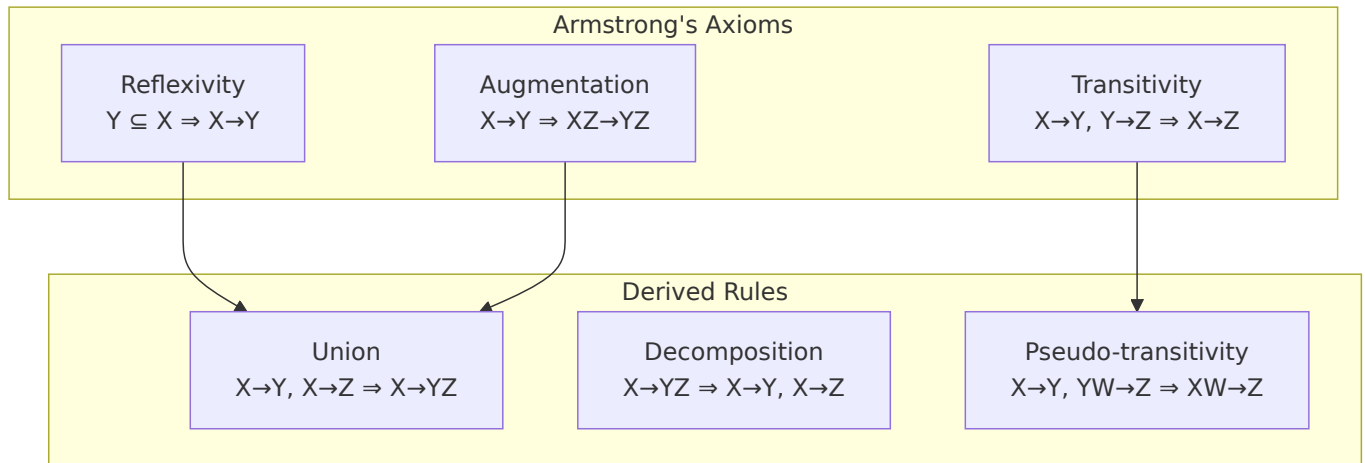
Given  $F = \{A \rightarrow B, B \rightarrow C\}$ . Prove  $A \rightarrow AC$ .

1.  $A \rightarrow B$  (given)
2. Augment (1) with  $A$ :  $AA \rightarrow AB \rightarrow A \rightarrow AB$  (since  $AA = A$ )
3. Decompose  $A \rightarrow AB$  to  $A \rightarrow A$  (trivial) and  $A \rightarrow B$  (already have)
4. From  $A \rightarrow B$  and  $B \rightarrow C$ , transitivity gives  $A \rightarrow C$ .
5. Union  $A \rightarrow A$  (trivial) and  $A \rightarrow C$  gives  $A \rightarrow AC$ .

### 6.5.4 Closure of Functional Dependencies ( $F^+$ )

The closure of a set  $F$  (denoted  $F^+$ ) is the set of all FDs that can be derived from  $F$  using Armstrong's axioms. Computing  $F^+$  is exponential, but for practical purposes we compute attribute closures to test membership (e.g., whether  $X \rightarrow Y \in F^+$ : check if  $Y \subseteq X^+$  under  $F$ ).

**Diagram:**



## 6.6 Summary

Functional dependencies are essential for relational database design and normalization. Key points:

- An FD  $X \rightarrow Y$  means that the value of  $X$  uniquely determines  $Y$ .
- **Trivial FDs** have  $Y \subseteq X$ ; **non-trivial FDs** provide useful constraints.
- **Attribute closure ( $X^+$ )** is the set of all attributes determined by  $X$  under a given FD set.
- **Keys** are found by checking closures: a candidate key is a minimal superkey.
- **Armstrong's axioms** (reflexivity, augmentation, transitivity) allow systematic derivation of all implied FDs.

# Chapter 7: Normalization

Normalization is the process of decomposing relational schemas to eliminate redundancy and avoid update anomalies (insertion, deletion, and modification anomalies). It is based on the theory of functional dependencies and, for higher normal forms, multi-valued dependencies. This chapter covers the first four normal forms, the conditions for lossless decomposition, and the concept of dependency preservation.

## 7.1 First Normal Form (1NF)

### Definition

A relation schema R is in **First Normal Form (1NF)** if the domain of every attribute is **atomic** (indivisible). No attribute can contain sets, lists, composite values, or repeating groups. Every tuple contains exactly one value (possibly NULL) per attribute.

### Violations and Resolution

Violations occur in two common forms:

- **Repeating columns:** e.g., phone1 , phone2 , phone3 .
- **Set-valued attributes:** e.g., phone\_numbers containing "555-1111,555-2222".

**Resolution:** Flatten the relation by creating a separate tuple for each value in the multi-valued attribute.

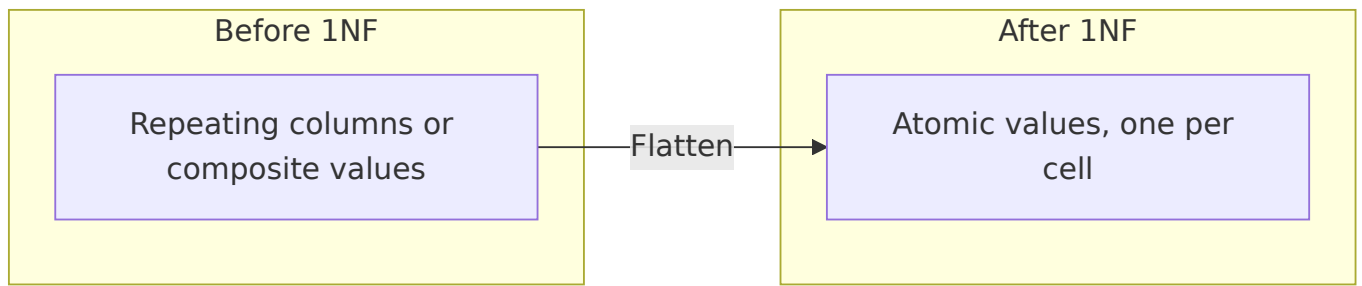
**Example (repeating columns):**

| emp_id | name  | phone1   | phone2   |
|--------|-------|----------|----------|
| 101    | Alice | 555-1111 | 555-2222 |

After 1NF:

| emp_id | name  | phone_number |
|--------|-------|--------------|
| 101    | Alice | 555-1111     |
| 101    | Alice | 555-2222     |

**Diagram:**



## Importance

1NF is mandatory for all relational database systems. It eliminates nested structures and ensures that each row-column intersection contains a single value.

## 7.2 Second Normal Form (2NF)

### Prerequisite

The relation must already be in 1NF.

### Definition

A relation is in **2NF** if it has **no partial dependency**: no non-prime attribute (an attribute that is not part of any candidate key) is functionally dependent on only a **proper subset** of any candidate key. In other words, every non-prime attribute must be **fully functionally dependent** on every candidate key.

### Partial Dependency Explained

Given a candidate key that is composite (e.g., `(student_id, course_id)`), a partial dependency occurs when a non-prime attribute depends on only one part of the key (e.g., `student_id → student_name`).

### Example Violation

Consider `Enroll(student_id, course_id, student_name, grade)`.

- Candidate key: `(student_id, course_id)`.
- Non-prime attributes: `student_name, grade`.
- FD: `student_id → student_name` exists. Here, `student_name` depends on a proper subset of the key (`student_id` alone). This is a partial dependency → violates 2NF.

### Decomposition to 2NF

Decompose into:

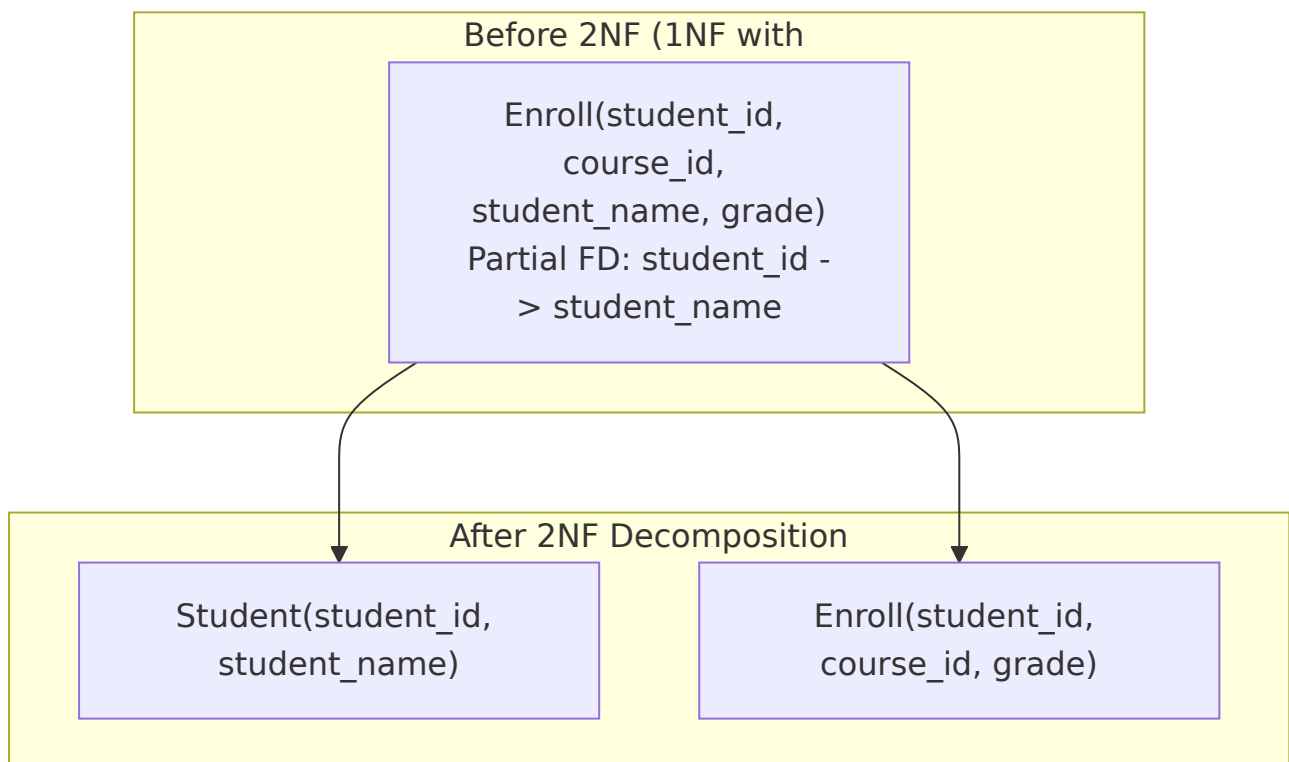
- `Student(student_id, student_name)` – all attributes depend on the full key.
- `Enroll(student_id, course_id, grade)` – the remaining attributes depend on the full key.

## Formal Algorithm for 2NF

Given a relation R with candidate keys and a set of FDs F:

1. Identify all partial dependencies: for each candidate key K that is composite, if there exists an FD  $X \rightarrow Y$  where  $X \subset K$ , X is non-empty, and Y contains non-prime attributes.
2. For each such  $X \rightarrow Y$ , create a new relation  $(X \cup Y)$  and remove Y from the original relation.
3. Ensure each relation is in 1NF and has no remaining partial dependencies.

**Diagram:**



## 7.3 Third Normal Form (3NF)

### Prerequisite

The relation must be in 2NF.

### Definition

A relation is in **3NF** if it has **no transitive dependency**: for every non-trivial functional dependency  $X \rightarrow Y$ , either:

- $X$  is a superkey, **or**
- $Y$  is prime (i.e., every attribute in  $Y$  is part of some candidate key).

Equivalently, no non-prime attribute is functionally dependent on another non-prime attribute.

## Transitive Dependency Explained

A transitive dependency occurs when  $X \rightarrow Y$  and  $Y \rightarrow Z$ , with  $Y$  not being a candidate key and  $Z$  being non-prime. Then  $X \rightarrow Z$  is transitive and undesirable.

## Example Violation

Consider `Employee(emp_id, dept_id, dept_location)` with FDs:

- `emp_id → dept_id`
- `dept_id → dept_location`

Here, `emp_id → dept_location` is transitive via `dept_id`. `dept_location` is non-prime and depends on `dept_id` (a non-superkey). This violates 3NF.

## Decomposition to 3NF

Decompose into:

- `Employee(emp_id, dept_id)`
- `Department(dept_id, dept_location)`

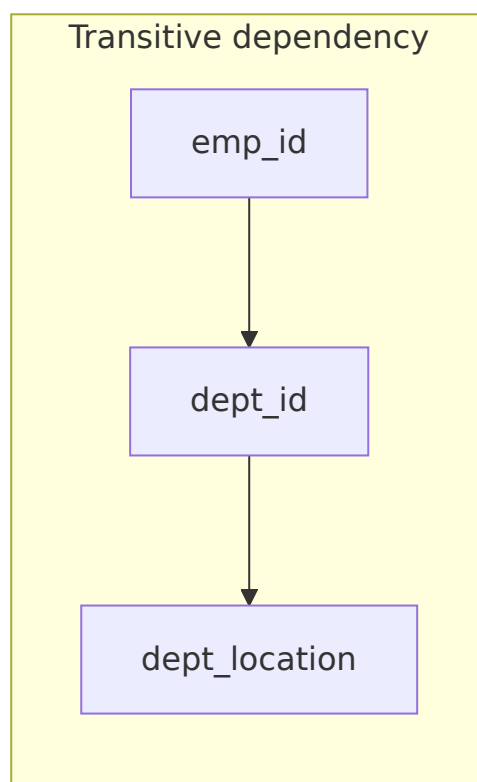
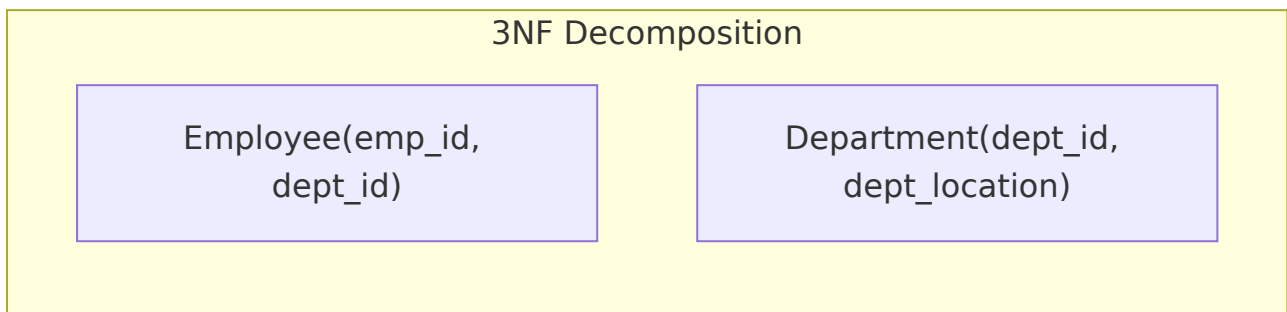
## Synthesis Algorithm for 3NF (Lossless, Dependency-Preserving)

Given a relation  $R$  and a set of FDs  $F$ :

1. Compute a minimal cover  $F_c$  of  $F$ .
2. For each FD  $X \rightarrow Y$  in  $F_c$ , create a relation schema  $(X \cup Y)$ .
3. If none of the created schemas contains a candidate key of  $R$ , add an additional relation that contains a candidate key.
4. Eliminate any relation that is a subset of another.

This algorithm guarantees a 3NF decomposition that is both lossless and dependency-preserving.

**Diagram:**



## 7.4 Boyce-Codd Normal Form (BCNF)

### Definition

A relation is in **BCNF** if for every non-trivial functional dependency  $X \rightarrow Y$  (where  $Y$  is not a subset of  $X$ ),  $X$  is a **superkey**. BCNF is a stronger version of 3NF: it removes all redundancy based on functional dependencies, but may sacrifice dependency preservation.

### Difference from 3NF



In 3NF, a dependency  $X \rightarrow Y$  is allowed if  $Y$  is prime (i.e., part of a candidate key) even when  $X$  is not a superkey. BCNF forbids this. Thus BCNF is stricter.

## Example Violation of BCNF but Not 3NF

Consider `Instructor(course_id, instructor_id, office)` with FDs:

- `course_id → instructor_id` (each course has one instructor)
- `instructor_id → office` (each instructor has one office)

Candidate keys: `(course_id)` only? Let's examine: `course_id` determines `instructor_id`, and `instructor_id` determines `office`. So `course_id` alone determines all attributes? Yes, `course_id → instructor_id, office`. So candidate key = `{course_id}`. Now FD `instructor_id → office` violates BCNF because `instructor_id` is not a superkey. However, is it allowed in 3NF? `office` is non-prime, and `instructor_id` is not a superkey, so it also violates 3NF? Actually, in this example it violates both. Let's construct a classic BCNF/3NF difference:

Classic example: `R(A, B, C)` with FDs: `AB → C` and `C → B`. Candidate keys: `AB` and `AC`. Here, `C → B` has `C` not a superkey, but `B` is prime (part of candidate key `AB`). So the dependency is allowed in 3NF but violates BCNF. That is the distinction.

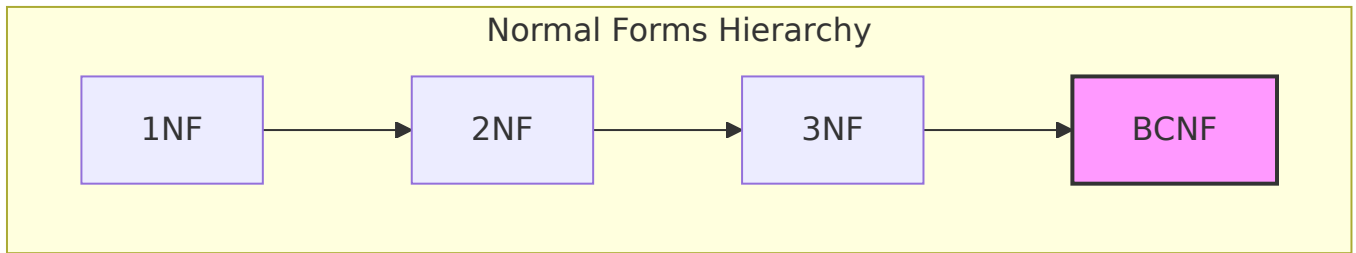
**BCNF Decomposition Algorithm** (lossless, may lose dependencies):

1. Initialize result = `{R}`.
2. While there is a relation `S` in result that is not in BCNF:
  - Choose a non-trivial FD  $X \rightarrow Y$  in `S` that violates BCNF ( $X$  is not a superkey of `S`).
  - Replace `S` with `(X ∪ Y)` and `(S \ Y)`.
3. Continue until all relations are in BCNF.

**Example:** `R(A, B, C)` with FDs `AB → C`, `C → B`. Violation: `C → B`. Decompose:

- First decomposition: `R1(C, B)`, `R2(A, C)`. Now `R1` has `C → B` (`C` is superkey in `R1`, so BCNF). `R2` has no non-trivial FDs except trivial, so BCNF. The decomposition is lossless but the FD `AB → C` is lost because `R2(A, C)` alone does not imply `AB → C`.

**Diagram:**



## 7.5 Fourth Normal Form (4NF) – Basic Idea

### Motivation

BCNF deals only with functional dependencies. However, there exist **multi-valued dependencies (MVDs)** that can cause redundancy even in BCNF relations.

### Multi-valued Dependency (MVD)

An MVD  $X \twoheadrightarrow Y$  (read as "X multi-determines Y") means that for a given X value, the set of Y values is independent of the set of Z values, where Z is the set of all remaining attributes. Formally, if  $(x, y_1, z_1)$  and  $(x, y_2, z_2)$  are tuples, then  $(x, y_1, z_2)$  and  $(x, y_2, z_1)$  must also be tuples.

### Definition of 4NF

A relation is in **4NF** if it is in BCNF and has **no non-trivial multi-valued dependencies** except those that are implied by candidate keys (i.e., an MVD  $X \twoheadrightarrow Y$  where X is a superkey is allowed).

### Example Violation

Consider `Employee_Skills_Languages(emp_id, skill, language)`. Assume each employee can have multiple skills and multiple languages independently. The relation is in BCNF because the only key is all three attributes, and there are no FDs. However, there is an MVD: `emp_id  $\twoheadrightarrow$  skill` and `emp_id  $\twoheadrightarrow$  language`. This causes redundancy: for each skill, every language is repeated.

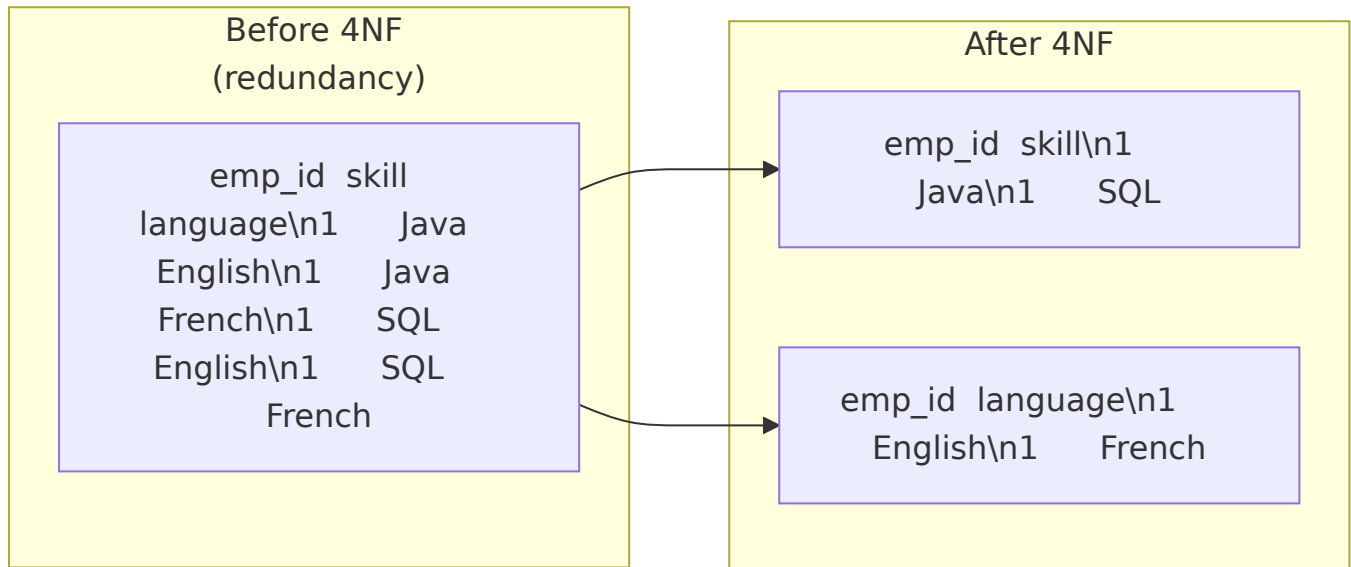
| emp_id | skill | language |
|--------|-------|----------|
| 1      | Java  | English  |
| 1      | Java  | French   |
| 1      | SQL   | English  |

|   |     |        |
|---|-----|--------|
| 1 | SQL | French |
|---|-----|--------|

The repetition is eliminated by decomposing into:

- `Employee_Skill(emp_id, skill)`
- `Employee_Language(emp_id, language)`

**Diagram:**



## 7.6 Lossless Decomposition

### Definition

A decomposition of a relation  $R$  into  $R_1, R_2, \dots, R_k$  is **lossless** (or lossless-join) if the natural join of all  $R_i$  yields exactly the original relation  $R$  (no spurious tuples and no missing tuples). Losslessness is **mandatory** for any practical decomposition.

### Testing for Binary Decomposition

For a decomposition into  $R_1$  and  $R_2$ , the join is lossless if and only if:

- $(R_1 \cap R_2) \rightarrow R_1$  **or**  $(R_1 \cap R_2) \rightarrow R_2$  (i.e., the common attributes form a superkey for at least one of the relations).

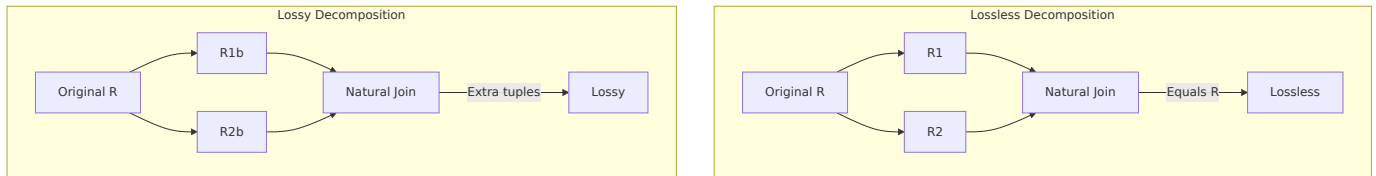
### Example

$R(A, B, C)$  with  $F = \{A \rightarrow B\}$ . Decompose into  $R_1(A, B)$  and  $R_2(A, C)$ . Intersection =  $\{A\}$ . Since  $A \rightarrow B$  holds,  $A$  is a superkey of  $R_1$ . Hence lossless.

## Lossy Decomposition Counterexample

$R(A, B, C)$  with no FDs (or with  $A \rightarrow B$ ). Decompose into  $R_1(A, B)$  and  $R_2(B, C)$ . Intersection =  $\{B\}$ .  $B$  is not a superkey of either (no FD  $B \rightarrow A$  or  $B \rightarrow C$ ). The join may produce spurious tuples.

**Diagram:**



## Algorithm for Lossless Decomposition into BCNF

The BCNF decomposition algorithm described earlier always produces a lossless decomposition because each step replaces a relation with two relations whose common attributes include the left-hand side of a violating FD, which is a superkey of the new relation.

## 7.7 Dependency Preservation

### Definition

A decomposition is **dependency-preserving** if the union of the projections of the original functional dependencies onto each decomposed relation implies all original dependencies. Formally, let  $F$  be the set of FDs on  $R$ . For each  $R_i$ , compute  $\pi_{R_i}(F)$  = all FDs  $X \rightarrow Y$  such that  $X \cup Y \subseteq R_i$  and  $X \rightarrow Y$  is in  $F^+$ . The decomposition is dependency-preserving if  $(\pi_{R_1}(F) \cup \pi_{R_2}(F) \cup \dots \cup \pi_{R_k}(F))^+ = F^+$ .

In simpler terms: every FD in  $F$  should be enforceable by checking individual relations without performing joins.

### Example of Preservation

$R(A, B, C)$  with  $F = \{A \rightarrow B, B \rightarrow C\}$ . Decompose into  $R_1(A, B)$  and  $R_2(B, C)$ . Projected FDs:  $R_1$  gives  $A \rightarrow B$ ;  $R_2$  gives  $B \rightarrow C$ . Their union implies  $A \rightarrow C$  (transitivity), which was implied by  $F$ . So dependencies are preserved.

### Example of Non-Preservation

$R(A, B, C)$  with  $F = \{AB \rightarrow C, C \rightarrow B\}$ . Decompose into  $R_1(A, C)$  and  $R_2(B, C)$  (or into BCNF as earlier:  $R_1(C, B)$ ,  $R_2(A, C)$ ). Projected FDs:  $R_1$  gives  $C \rightarrow B$ ;  $R_2$  gives no non-trivial FDs. The FD  $AB \rightarrow C$  cannot be derived from these projections because it involves both  $A$  and  $B$  together, which are never in the same relation. Hence the decomposition is **not** dependency-preserving.

## Importance

Dependency preservation allows efficient integrity checking: each FD can be enforced on a single relation. Without it, checking an FD may require a join, which is expensive. However, BCNF sometimes forces the loss of dependencies; 3NF can always preserve dependencies.

**Diagram:**

### Non-Preserving

FD:  $AB \rightarrow C$

No single  $R_i$  contains  
 $A, B, C$

Missing  $AB \rightarrow C$

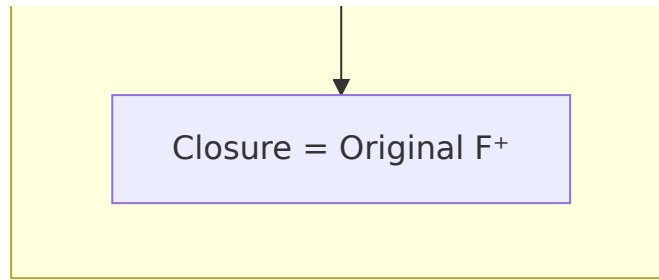
Closure  $\neq$  Original

### Dependency Preservation

Original FD:  $X \rightarrow Y$

Project onto  $R_i$   
containing  $XuY$

Union of projected FDs



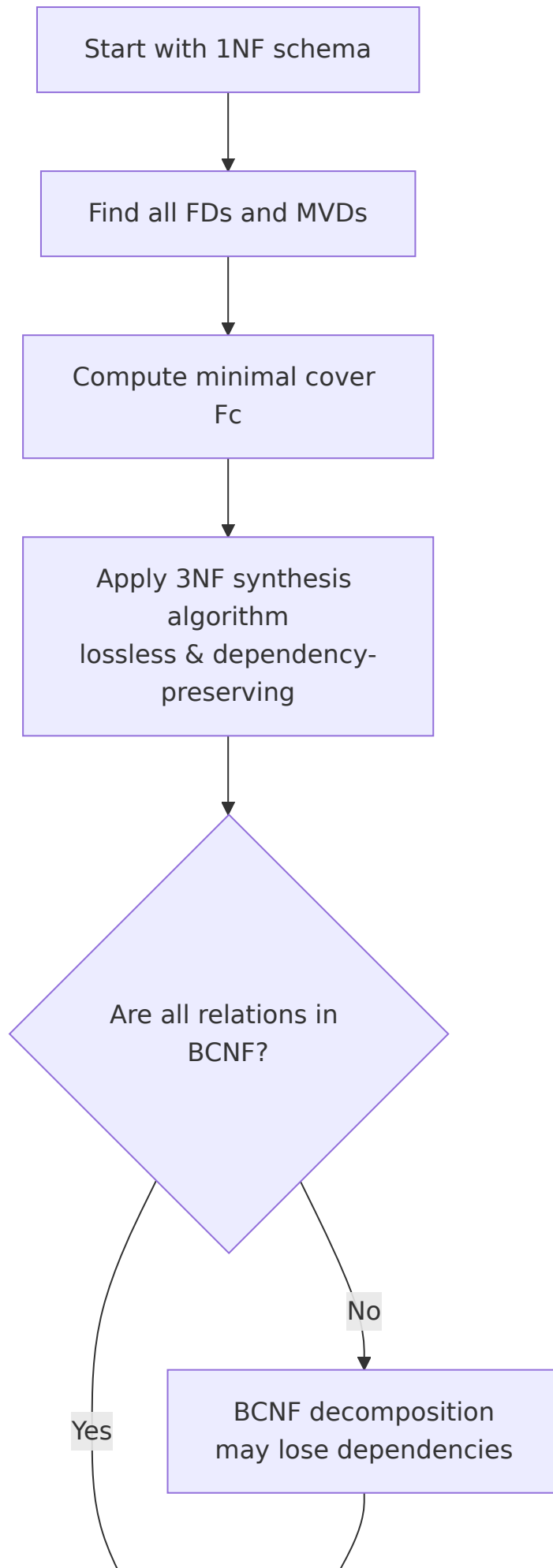
## 7.8 Summary Table of Normal Forms

| Normal Form | Condition                                        | Redundancy Eliminated                | Dependency Preservation Possible? |
|-------------|--------------------------------------------------|--------------------------------------|-----------------------------------|
| 1NF         | Atomic attributes                                | Repeating groups                     | N/A                               |
| 2NF         | No partial dependencies                          | Redundancy from composite keys       | Yes                               |
| 3NF         | No transitive dependencies                       | Redundancy from non-key dependencies | Yes (synthesis algorithm)         |
| BCNF        | Every FD's left-hand side is a superkey          | All FD-based redundancy              | Not always                        |
| 4NF         | No non-trivial MVDs except those implied by keys | MVD-based redundancy                 | N/A (handles MVDs)                |

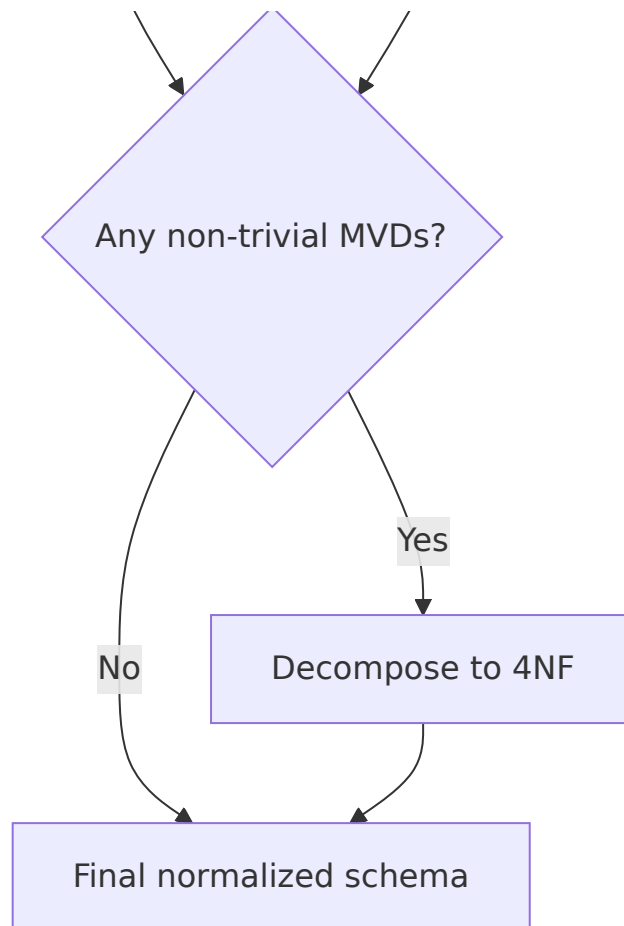
## 7.9 Practical Design Guidelines

1. **Start from a minimal cover** of functional dependencies.
2. **Decompose to 3NF** using the synthesis algorithm to achieve losslessness and dependency preservation.
3. **Check for BCNF violations**; if present and dependency loss is acceptable, further decompose to BCNF.
4. **Always ensure lossless join** – it is non-negotiable.
5. **Test for MVDs** only if redundancy remains in BCNF; then normalize to 4NF.

**Algorithmic Flowchart:**







## 7.10 Conclusion

Normalization is a rigorous approach to database design that reduces redundancy and prevents anomalies. First normal form ensures atomicity. Second and third normal forms remove partial and transitive dependencies, respectively. Boyce-Codd normal form provides a stronger condition but may sacrifice dependency preservation. Fourth normal form addresses multi-valued dependencies. Lossless decomposition is always required, while dependency preservation is highly desirable. Understanding these concepts enables the creation of robust, maintainable relational databases.

# Chapter 8: Transactions

A transaction is a logical unit of work that consists of one or more database operations (reads, writes, inserts, updates, deletes). Transactions ensure that the database remains in a consistent state even in the presence of concurrent accesses or system failures. This chapter introduces the fundamental concepts of transactions, the ACID properties that guarantee reliability, and the various states a transaction can pass through during its execution.

## 8.1 Transaction Concept

### Definition

A **transaction** is a sequence of operations performed as a single logical unit of work. A transaction must be executed in its entirety or not at all. Typical operations within a transaction include reading data from the database (read) and writing data to the database (write).

### Example

Consider a bank transfer of \$100 from account A to account B. This operation consists of two steps:

1. Read balance of A, subtract \$100, write new balance to A.
2. Read balance of B, add \$100, write new balance to B.

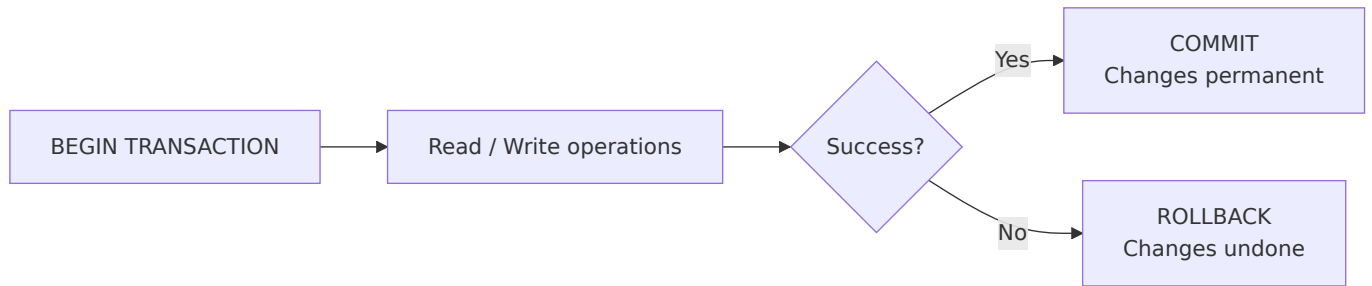
Both steps together form a transaction. If only step 1 executes and then a failure occurs, the database would be inconsistent. Transactions prevent such partial updates.

### Transaction Boundaries

In SQL, transactions are defined using:

- `BEGIN TRANSACTION` (or `START TRANSACTION`) – marks the start.
- `COMMIT` – makes all changes permanent.
- `ROLLBACK` – undoes all changes made since the last commit.

**Diagram:**



## 8.2 ACID Properties

ACID is an acronym for Atomicity, Consistency, Isolation, Durability. These properties guarantee that database transactions are processed reliably.

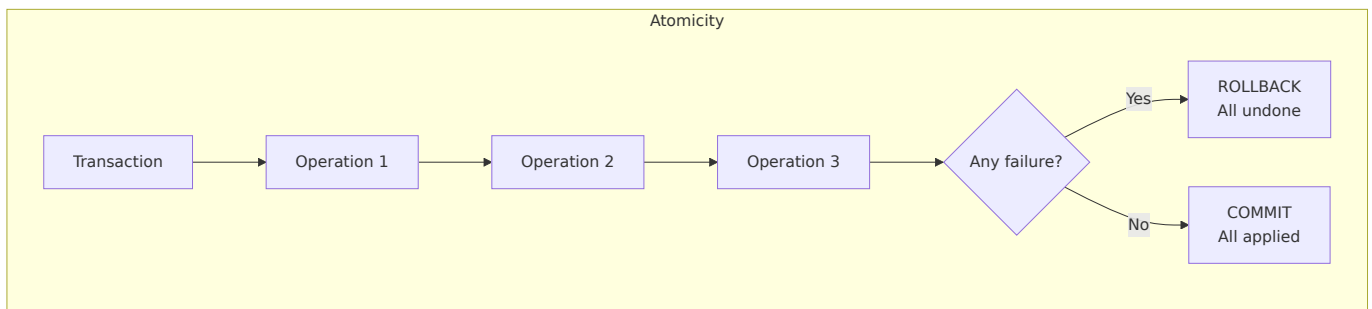
### 8.2.1 Atomicity

**Definition:** A transaction is an atomic unit of work. Either all of its operations are executed successfully, or none of them are applied. Partial execution is not allowed.

**Mechanism:** The database maintains a log (or write-ahead log) that records all changes. In case of failure before commit, the log is used to undo (roll back) partial changes.

**Example:** In the bank transfer, if the system crashes after debiting A but before crediting B, atomicity ensures that the debit is also undone.

**Diagram:**



### 8.2.2 Consistency

**Definition:** A transaction transforms the database from one consistent state to another consistent state. Consistency constraints (e.g., foreign keys, unique constraints, check constraints, application-specific invariants) must be preserved before and after the transaction.

**Example:** In the bank transfer, a consistency constraint might be that the sum of all account balances remains constant. After the transfer, this invariant still holds (total = old total). Also, no account balance becomes negative if sufficient funds exist.

**Note:** Consistency is partly enforced by the database (integrity constraints) and partly by the application logic.

### 8.2.3 Isolation

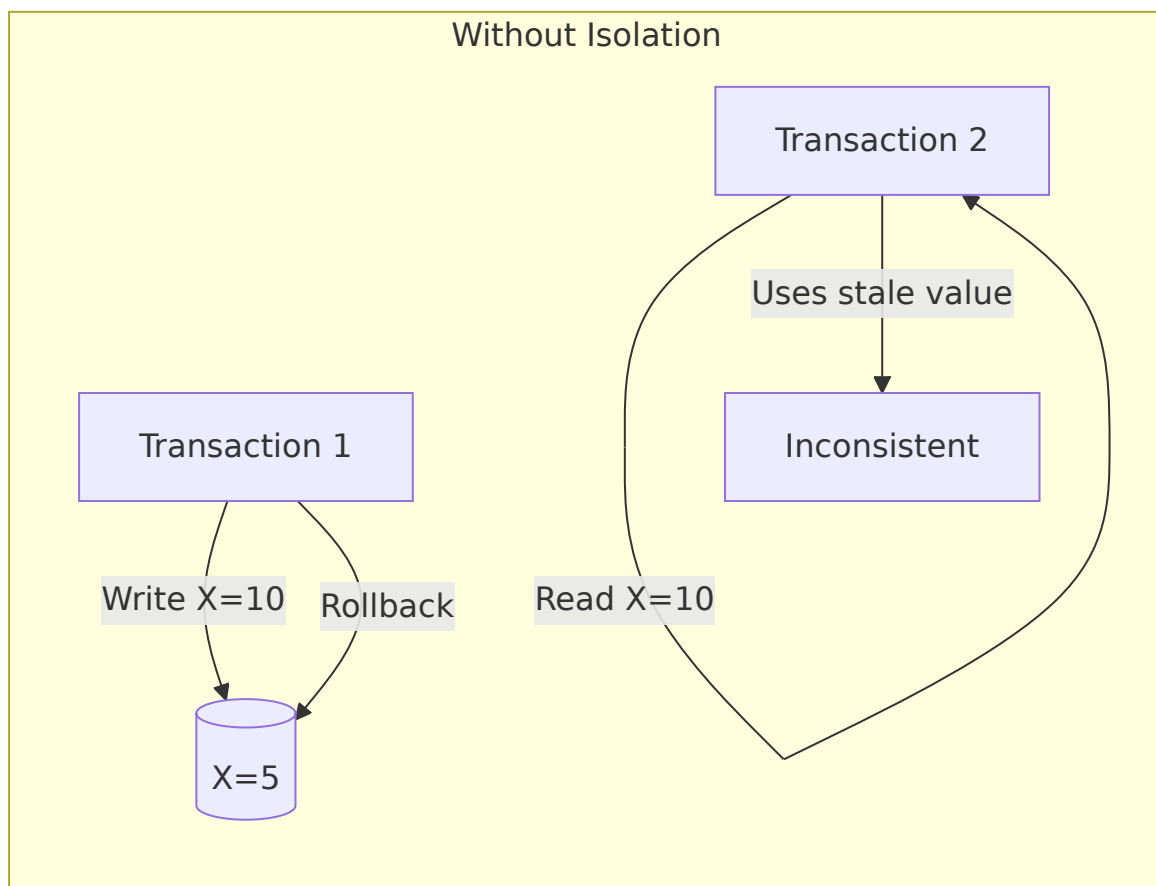
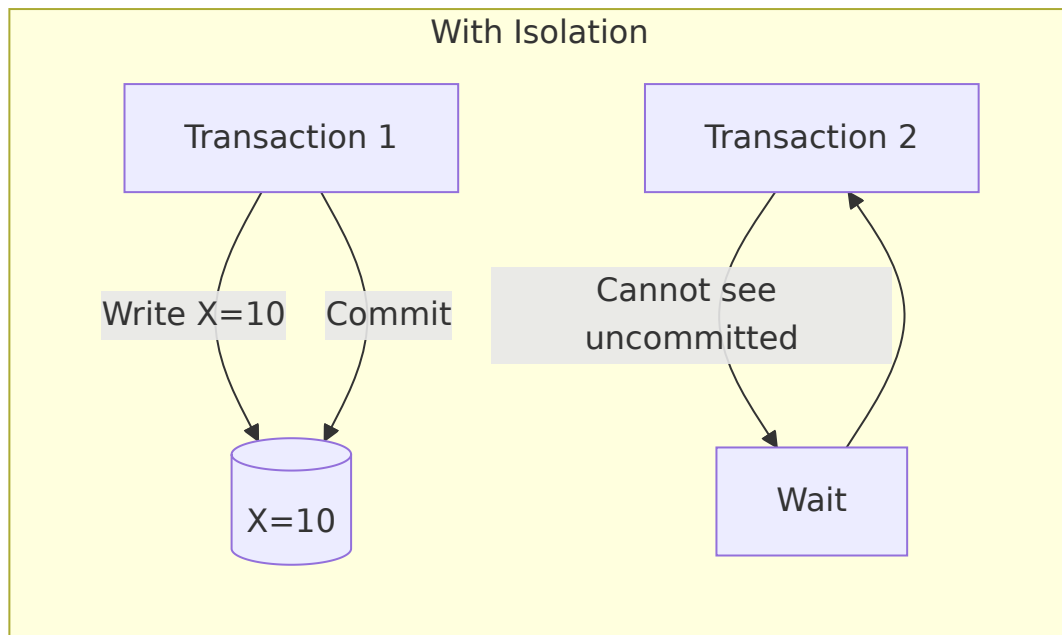
**Definition:** Transactions executing concurrently appear to be executed serially (one after another). Intermediate states of a transaction are not visible to other transactions. This prevents issues such as dirty reads, non-repeatable reads, and phantom reads.

**Levels of Isolation** (from weakest to strongest):

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

**Example:** If two concurrent transfers try to update the same account, isolation ensures that the final result is as if one transfer happened after the other, not an interleaved mixture that could lose updates.

**Diagram:**



## 8.2.4 Durability

**Definition:** Once a transaction has been committed, its changes persist permanently, even in the event of a system failure (power loss, crash, etc.). The database ensures that committed updates are stored on non-volatile storage (e.g., disk).

**Mechanism:** Write-ahead logging ensures that before a transaction is committed, all its log records are flushed to disk. On recovery, the log is replayed to redo committed transactions.

**Example:** After the bank transfer commits, even if the database crashes immediately, the new balances are preserved.

## Summary Table of ACID

| Property    | Guarantee                             | Enforced by                         |
|-------------|---------------------------------------|-------------------------------------|
| Atomicity   | All or nothing execution              | Transaction log + recovery manager  |
| Consistency | Preserves database invariants         | Application + DB constraints        |
| Isolation   | Concurrent transactions appear serial | Concurrency control (locking, MVCC) |
| Durability  | Committed changes survive failures    | Logging + checkpointing             |

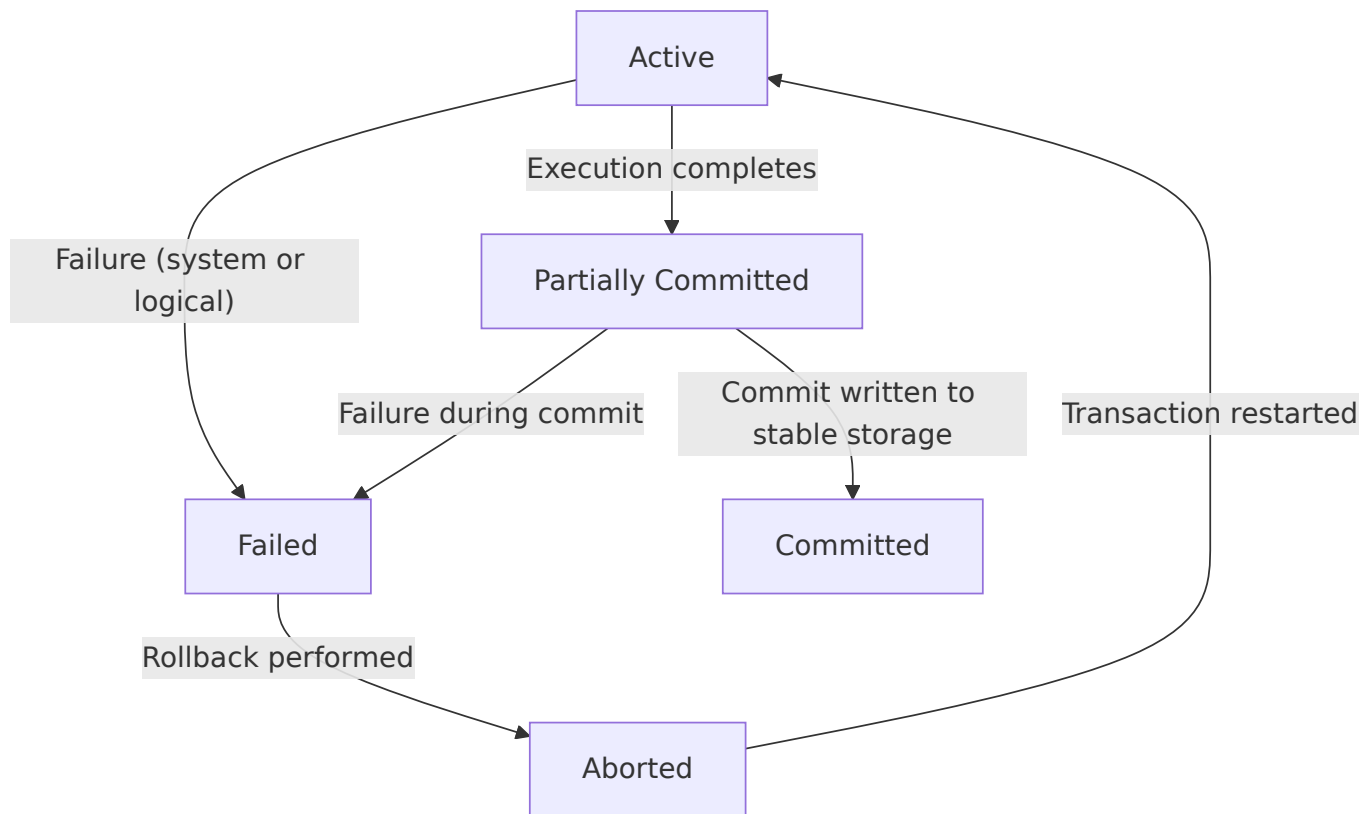
## 8.3 Transaction States

A transaction passes through several states during its lifetime. The state diagram captures the possible transitions.

### State Definitions

1. **Active:** The initial state. The transaction is executing its read/write operations. It remains active until the last operation is reached.
2. **Partially Committed:** After the final operation is executed, but before the commit decision is made permanent. The transaction has completed its work, but changes may still be in memory buffers.
3. **Committed:** After successful commit and all changes have been permanently written to disk. The transaction is complete.
4. **Failed:** If the transaction cannot complete normally (e.g., due to a system error, constraint violation, or user rollback), it enters the failed state.
5. **Aborted:** After the failed state, the transaction is rolled back (undo operations). The database is restored to the state before the transaction began. The aborted transaction may be restarted later.

### State Transition Diagram



## Explanation of Transitions

- **Active → Partially Committed**: Occurs when the transaction executes its last operation.
- **Active → Failed**: Occurs if the transaction cannot continue due to hardware failure, software bug, deadlock, or constraint violation.
- **Partially Committed → Committed**: After all log records are forced to disk and the commit record is written.
- **Partially Committed → Failed**: If the system crashes while writing the commit record.
- **Failed → Aborted**: The recovery manager performs a rollback, undoing all changes made by the transaction.
- **Aborted → Active**: The transaction can be restarted (e.g., by the application) after the cause of failure is resolved.

## Example Timeline

Consider a transaction that updates two accounts:

1. **Active**: Begin; read A, write A; read B, write B (last operation).
2. **Partially Committed**: Last operation done, but changes not yet forced to disk.
3. **Committed**: Log buffer flushed; commit record written. Transaction ends.

If a power failure occurs before the commit record is written, the transaction goes to Failed and then Aborted after recovery.

## System Log and State Recovery

The log contains records: `<START T>`, `<WRITE T, X, old_value, new_value>`, `<COMMIT T>`, `<ABORT T>`. On restart:

- Transactions with `<START>` but no `<COMMIT>` are undone (aborted).
- Transactions with `<COMMIT>` are redone (if necessary) for durability.

## 8.4 Summary

Transactions are fundamental to reliable database systems. The key concepts are:

- **Transaction:** A logical unit of work with atomic execution.
- **ACID properties:**
  - Atomicity: All or nothing.
  - Consistency: Preserves database invariants.
  - Isolation: Concurrent execution appears serial.
  - Durability: Committed changes survive failures.
- **Transaction states:** Active → Partially Committed → Committed (normal path), or Active → Failed → Aborted (failure path), with possible restart.



# Chapter 9: Concurrency Control

Concurrency control manages simultaneous execution of transactions to ensure database consistency and isolation. Without proper concurrency control, interleaved operations can lead to anomalies such as lost updates, dirty reads, and inconsistent retrievals. This chapter introduces schedules, serializability, locking protocols, two-phase locking, and deadlock handling.

## 9.1 Schedules

A **schedule** is the order in which operations of multiple transactions are executed. Operations include read (R), write (W), commit (C), and abort (A).

### 9.1.1 Serial Schedule

A schedule is **serial** if the operations of one transaction are executed consecutively without interleaving with operations of other transactions. Serial schedules always maintain consistency, but they limit concurrency.

**Example:** Transactions T1 and T2.

Serial schedule S1: T1 then T2

| Time | T1     | T2     |
|------|--------|--------|
| 1    | R(A)   |        |
| 2    | W(A)   |        |
| 3    | R(B)   |        |
| 4    | W(B)   |        |
| 5    | Commit |        |
| 6    |        | R(A)   |
| 7    |        | W(A)   |
| 8    |        | Commit |

### 9.1.2 Non-Serial Schedule

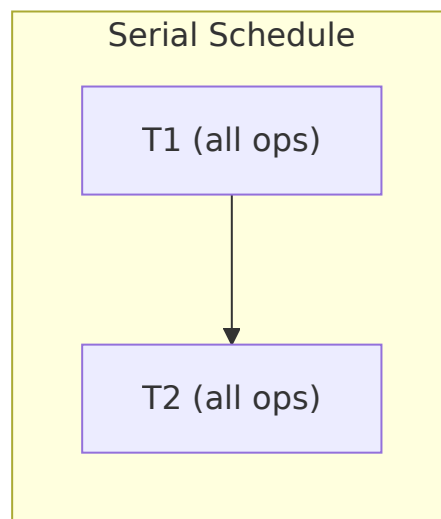
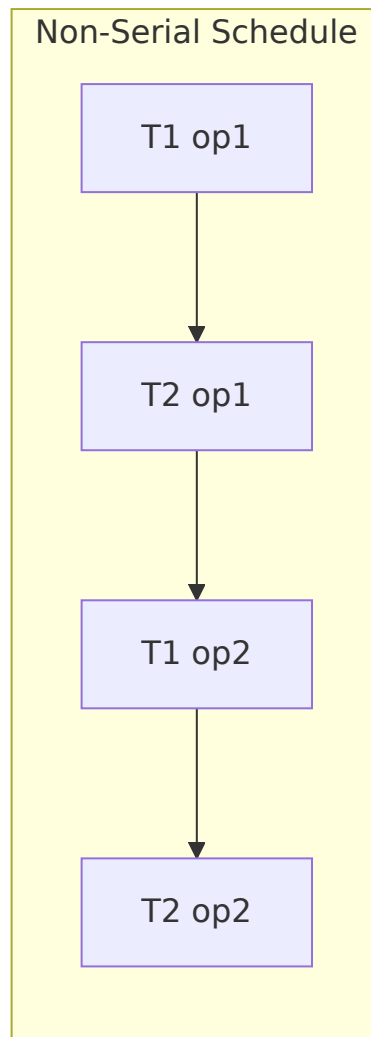
A **non-serial** schedule interleaves operations from multiple transactions. Such schedules may be correct (if they are equivalent to some serial schedule) or incorrect (if they lead to anomalies).

**Example:** Interleaved schedule

| Time | T1     | T2     |
|------|--------|--------|
| 1    | R(A)   |        |
| 2    |        | R(A)   |
| 3    | W(A)   |        |
| 4    |        | W(A)   |
| 5    | Commit |        |
| 6    |        | Commit |

This schedule may produce a lost update.

**Diagram:**



## 9.2 Serializability

A schedule is **serializable** if its effect on the database is equivalent to some serial schedule. Serializability is the standard correctness criterion for concurrent transactions. Two main types: conflict serializability and view serializability.

## 9.2.1 Conflict Serializability

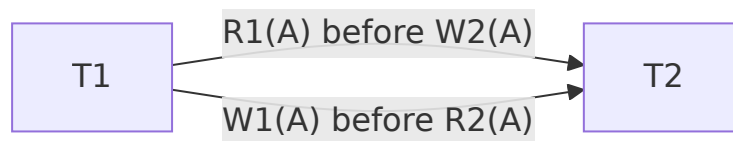
Two operations **conflict** if they belong to different transactions, access the same data item, and at least one of them is a write. Conflict pairs: R-W, W-R, W-W.

A schedule is **conflict serializable** if it can be transformed into a serial schedule by swapping adjacent non-conflicting operations (i.e., operations that do not conflict). The **precedence graph** is used to test conflict serializability:

- Nodes: transactions.
- Edge  $T_i \rightarrow T_j$  if an operation of  $T_i$  conflicts with and precedes an operation of  $T_j$ .
- If the graph is acyclic, the schedule is conflict serializable. The topological order gives a serial schedule.

**Example:** Schedule with  $T_1: R(A), W(A)$ ;  $T_2: R(A), W(A)$ . Conflicts:  $R_1(A)$  before  $W_2(A)$  gives edge  $T_1 \rightarrow T_2$ ;  $W_1(A)$  before  $R_2(A)$  gives  $T_1 \rightarrow T_2$ ; etc. Acyclic  $\rightarrow$  conflict serializable.

**Precedence Graph Diagram:**



## 9.2.2 View Serializability

View serializability is less restrictive than conflict serializability. A schedule is **view serializable** if it is view-equivalent to a serial schedule. Two schedules are view-equivalent if:

1. For each data item  $X$ , the initial read operations read from the same initial database state.
2. For each data item  $X$ , the write-read dependencies (i.e., which transaction writes  $X$  that is later read) are the same.
3. The final write operations on  $X$  are performed by the same transaction.

Every conflict-serializable schedule is view-serializable, but not vice versa. Testing view serializability is NP-complete, so practical concurrency control (e.g., 2PL) uses conflict serializability.

**Example of view serializable but not conflict serializable:**

$T_1: R(A), W(A)$ ;  $T_2: W(A)$ ;  $T_3: W(A)$ . A schedule with no conflicts but same initial/final writes.

## 9.3 Lock-Based Protocols

Locks are the most common mechanism for enforcing serializability. A transaction must acquire a lock before accessing a data item and release it afterward.

## Lock Types

- **Shared lock (S):** For read operations. Multiple transactions can hold shared locks on the same item simultaneously.
- **Exclusive lock (X):** For write operations. Only one transaction can hold an exclusive lock; no other lock (shared or exclusive) can coexist.

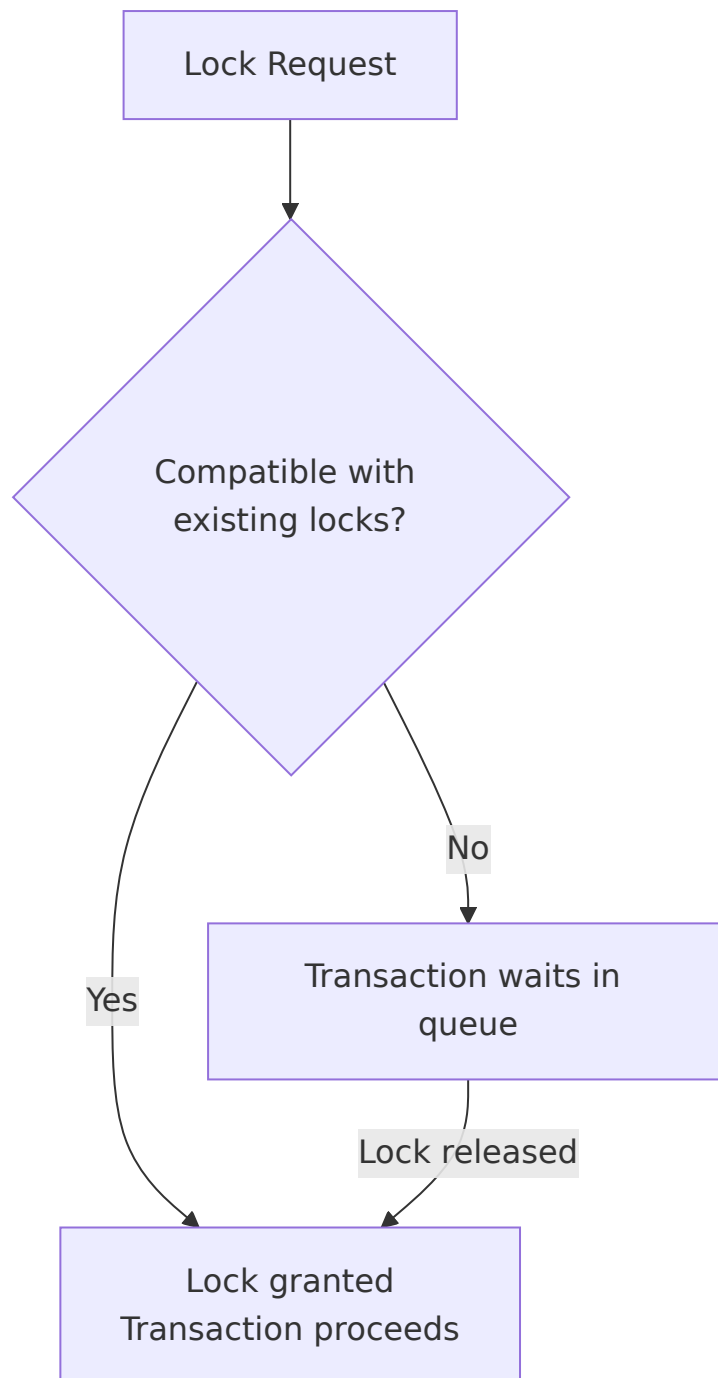
## Lock Compatibility Matrix

|               | Shared (S) | Exclusive (X) |
|---------------|------------|---------------|
| Shared (S)    | Yes        | No            |
| Exclusive (X) | No         | No            |

## Lock Manager

The lock manager grants or denies lock requests. If a lock cannot be granted, the requesting transaction is blocked until the lock becomes available (waiting queue).

**Diagram:**



## Basic Locking Protocol

A transaction must:

- Acquire a shared lock before reading an item.
- Acquire an exclusive lock before writing an item.
- Release locks after use.

However, basic locking does not guarantee serializability because lock release can occur early, leading to cascading aborts or non-serializable schedules.

## 9.4 Two-Phase Locking (2PL)

Two-phase locking is a concurrency control protocol that guarantees conflict serializability. It defines two phases:

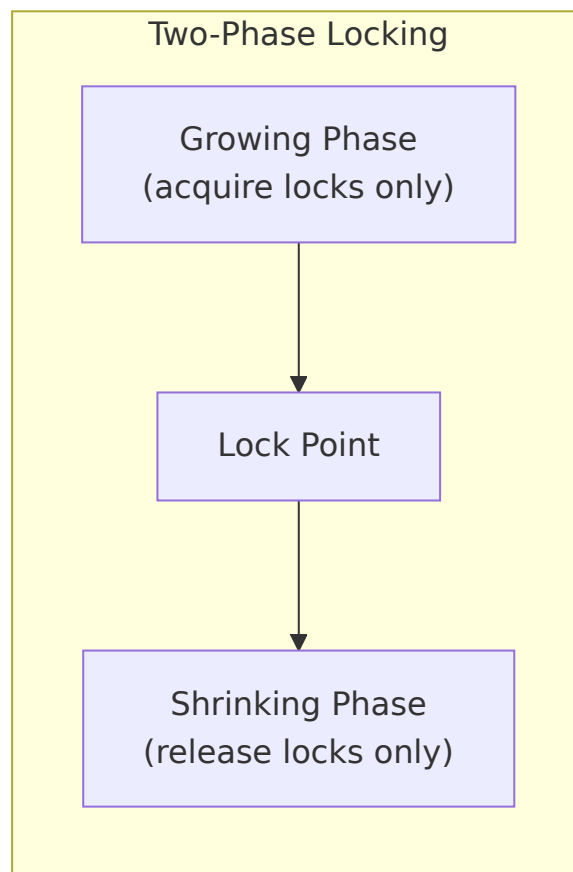
1. **Growing phase:** Transaction acquires locks but cannot release any lock.
2. **Shrinking phase:** Transaction releases locks but cannot acquire any new lock.

The point where a transaction transitions from growing to shrinking is called the **lock point**.

## Strict 2PL

A stricter variant: exclusive locks are held until commit (or abort). This prevents cascading aborts and ensures recoverability. Most DBMS implement strict 2PL.

**Diagram:**



## Example

Transaction T1: Lock-X(A), read(A), write(A), Lock-S(B), read(B), unlock(A), unlock(B). This violates 2PL because it releases A before acquiring B (releasing in growing phase). Correct: acquire all locks before any release.

## Properties of 2PL

- **Guarantees conflict serializability.**

- May cause deadlocks.
- Does not guarantee freedom from cascading aborts unless strict 2PL is used.
- Prevents unrepeatable reads and dirty writes.

## 9.5 Deadlocks

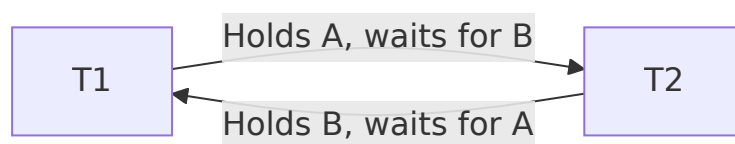
A **deadlock** occurs when two or more transactions are each waiting for a lock held by another, resulting in indefinite waiting.

### Example:

- T1 holds lock on A, requests lock on B.
- T2 holds lock on B, requests lock on A.

Neither can proceed.

### Diagram:



### 9.5.1 Deadlock Detection

Deadlocks are detected by constructing a **wait-for graph** (WFG):

- Nodes: transactions.
- Edge  $T_i \rightarrow T_j$  if  $T_i$  is waiting for a lock held by  $T_j$ .
- A cycle indicates a deadlock.

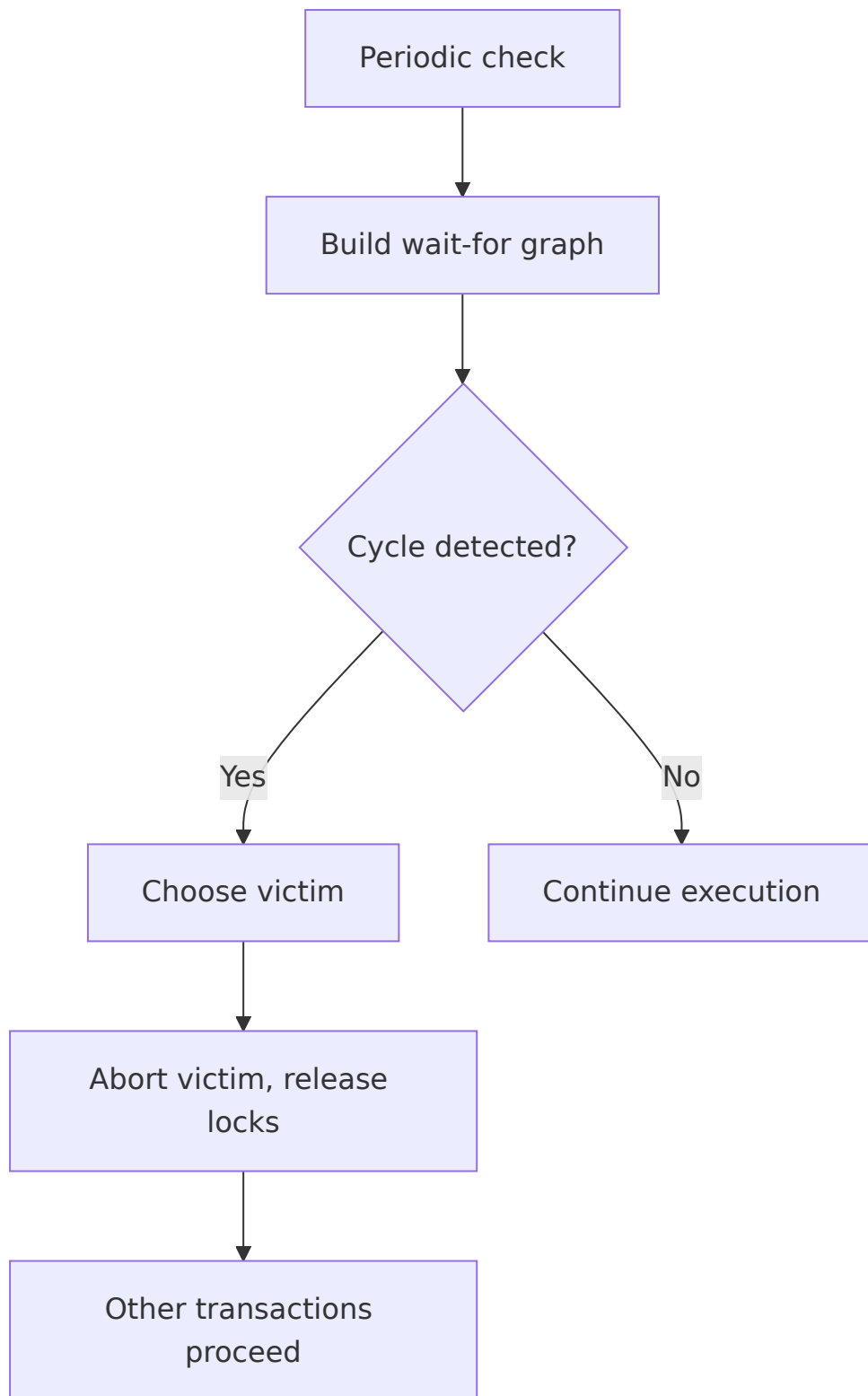
The system periodically checks the WFG. When a deadlock is detected, one transaction (victim) is aborted (rolled back) to break the cycle.

### Algorithm:

1. Build WFG.
2. If cycle exists, select victim (e.g., youngest, least work done, lowest priority).
3. Abort victim, release its locks, restart it later.

### Diagram:





## 9.5.2 Deadlock Prevention

Prevention ensures that deadlocks never occur by imposing rules on lock acquisition. Common prevention schemes:

- **Lock ordering (partial order):** All transactions acquire locks in a global order (e.g., by data item address). If a transaction needs a lock on a later item, it must release all earlier locks and restart. This is conservative but deadlock-free.

- **Wait-die (non-preemptive):** Each transaction is assigned a timestamp (older = smaller).  
If T1 requests a lock held by T2:
  - If T1 is older (smaller timestamp) than T2, T1 waits.
  - If T1 is younger, T1 dies (aborts and restarts with same timestamp).
- **Wound-wait (preemptive):**
  - If T1 is older than T2, T1 wounds T2 (T2 aborts and restarts).
  - If T1 is younger, T1 waits.

**Comparison table:**

| Scheme     | Older request lock held by younger | Younger request lock held by older |
|------------|------------------------------------|------------------------------------|
| Wait-die   | Wait                               | Die (abort)                        |
| Wound-wait | Wound (abort younger)              | Wait                               |

## Timeout-Based Prevention

A transaction waits only for a specified time. If timeout expires, the transaction aborts and restarts. Simple but may cause unnecessary restarts.

## 9.6 Summary

Concurrency control ensures serializable execution of concurrent transactions. Key concepts:

- **Schedules:** Serial (no interleaving) and non-serial (interleaved).
- **Serializability:** Conflict serializability (tested via precedence graph) and view serializability (more general but harder).
- **Lock-based protocols:** Shared/exclusive locks; compatibility matrix.
- **Two-phase locking (2PL):** Growing and shrinking phases; guarantees conflict serializability; strict 2PL prevents cascading aborts.
- **Deadlocks:**
  - Detection via wait-for graph; victim selection and abort.
  - Prevention via lock ordering, wait-die, wound-wait, or timeouts.

These mechanisms are implemented in all major database systems to support concurrent transaction processing while maintaining data integrity.

# Chapter 10: Recovery System

Recovery management ensures that the database remains consistent and durable despite various types of failures, including transaction failures, system crashes, and media failures. This chapter describes failure classifications, the write-ahead logging protocol, undo and redo operations, and the use of checkpoints to improve recovery efficiency.

## 10.1 Types of Failures

Database systems can experience several categories of failures. Recovery mechanisms are designed to handle each type appropriately.

### 10.1.1 Transaction Failures

A transaction may fail due to logical errors (e.g., division by zero, constraint violation) or system errors (e.g., deadlock detection leading to abort). When a transaction fails, it must be rolled back (undone) to preserve atomicity.

#### Causes:

- Application logic errors.
- Integrity constraint violations (e.g., foreign key, check constraint).
- Deadlock victim selection.
- User-issued ROLLBACK.

### 10.1.2 System Crashes (Soft Failures)

A system crash occurs when the database system stops unexpectedly due to power failure, operating system panic, or hardware fault. Main memory contents are lost, but stable storage (disk) is preserved. The recovery system must restore the database to a consistent state by undoing incomplete transactions and redoing committed transactions whose updates may not have reached disk.

#### Causes:

- Power outage.
- Operating system crash.
- Database software bug.

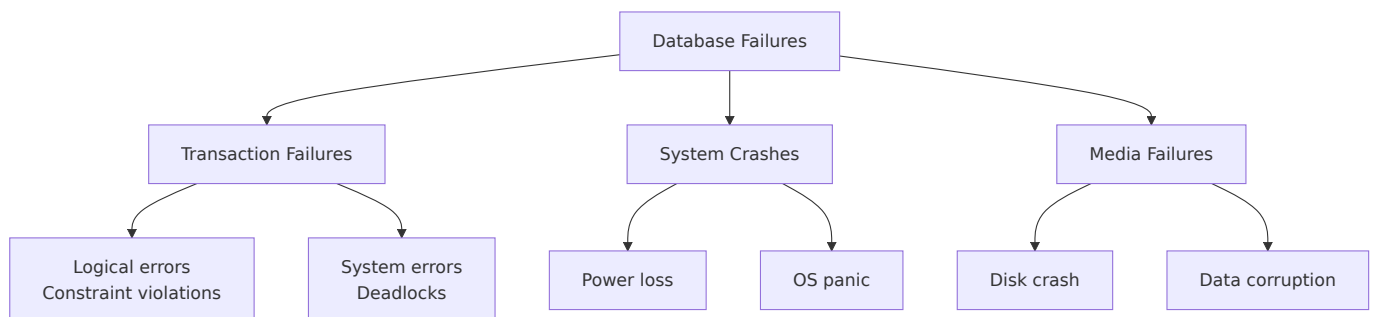
### 10.1.3 Media Failures (Hard Failures)

A media failure involves permanent damage to stable storage, such as a head crash on a disk or corruption of a disk block. Recovery from media failures requires archived backups and transaction logs to restore lost data.

#### Causes:

- Disk head crash.
- Bad sectors.
- Accidental file deletion.

### 10.1.4 Failure Classification Diagram



## 10.2 Log-Based Recovery

Log-based recovery uses a persistent log (also called journal) stored on stable storage. The log records every update operation performed by transactions. In the event of a failure, the log is used to undo incomplete transactions and redo committed ones.

### 10.2.1 Log Structure

Each log record typically contains:

- **Transaction identifier (TID)**
- **Data item identifier (X)**
- **Old value (before image)**
- **New value (after image)**
- **Operation type (START, UPDATE, COMMIT, ABORT)**

Common log record types:

- `<START T>` – transaction T has begun.
- `<WRITE T, X, old_value, new_value>` – T wrote new\_value to X; old\_value was the previous value.

- `<COMMIT T>` – T has committed successfully.
- `<ABORT T>` – T has been aborted.

## 10.2.2 Write-Ahead Logging (WAL) Protocol

WAL is a fundamental rule that ensures recoverability:

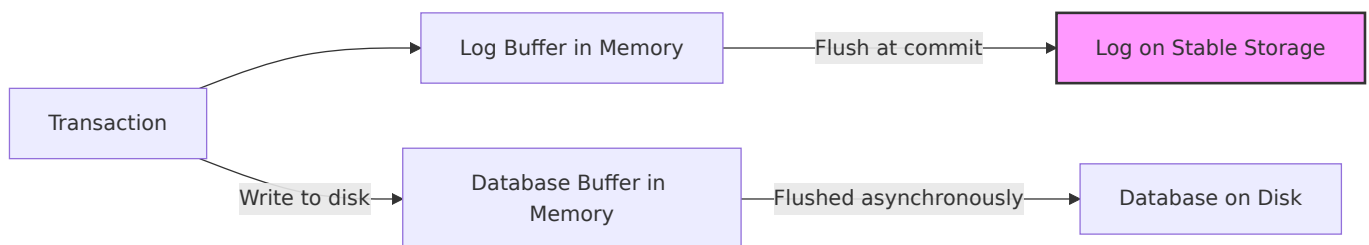
1. **Log before write:** Before a transaction writes a data item to disk, the corresponding `<WRITE>` log record must be flushed to stable storage.
2. **Commit log before commit:** Before a transaction's `<COMMIT>` record is acknowledged, all log records of that transaction must be flushed to stable storage.

WAL guarantees that if a crash occurs, the log contains enough information to recover the database.

## 10.2.3 Log Buffer and Forcing

The log is typically written to a memory buffer and flushed to disk at commit time or when the buffer fills. For performance, group commit can be used: multiple transactions' logs are flushed together.

**Diagram:**



## 10.3 Undo and Redo

Recovery after a system crash involves two basic operations: undo and redo. These operations use the log to restore the database to a consistent state.

### 10.3.1 Undo Operation

Undo reverts the effects of a transaction that did not commit (i.e., transactions that were active at the time of crash or aborted). For each write operation of the transaction, the old value (before image) is written back to the data item. Undo operations must be **idempotent** (applying the same undo multiple times has the same effect as once).

**Undo procedure** (for a transaction T):

- Read the log backward from the end.
- For each `<WRITE T, X, old, new>` encountered, write `old` to X (restore before image).
- Stop at `<START T>`.
- Finally, write an `<ABORT T>` record.

### 10.3.2 Redo Operation

Redo reapplies the effects of a committed transaction whose updates may not have been written to disk before the crash. For each write operation, the new value (after image) is written to the data item. Redo must also be idempotent.

**Redo procedure** (for a transaction T):

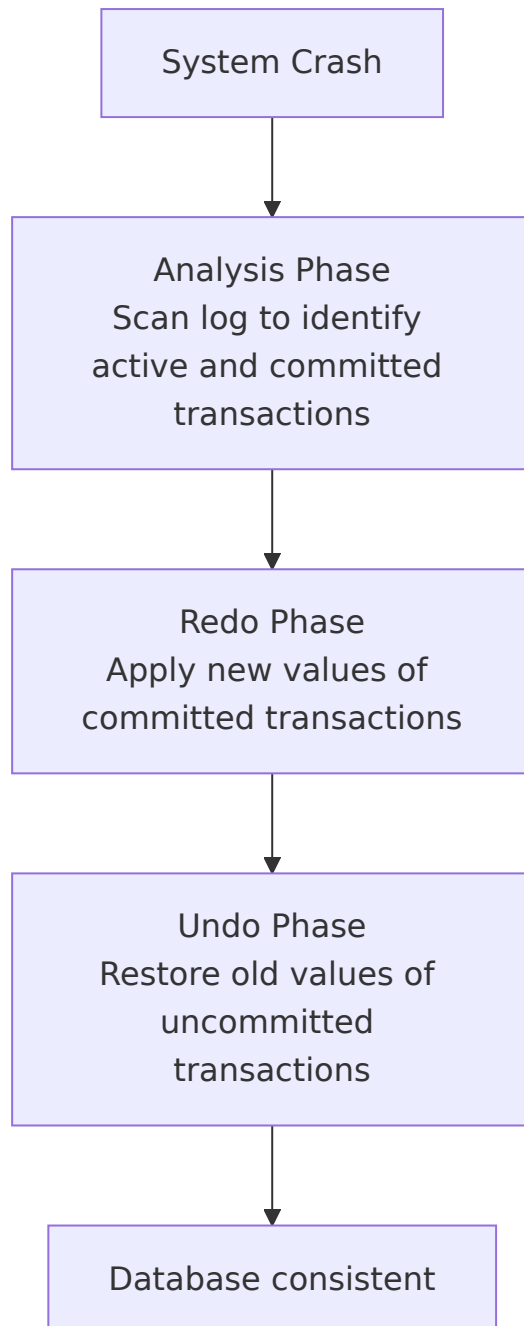
- Read the log forward from the beginning.
- For each `<WRITE T, X, old, new>` encountered, write `new` to X (apply after image).
- Redo is typically performed only after ensuring that the transaction's `<COMMIT>` appears in the log.

### 10.3.3 Recovery Algorithm (ARIES-like)

After a system crash, the recovery manager typically performs three phases:

1. **Analysis:** Determine which transactions were active at crash and which committed.
2. **Redo:** Redo all updates from committed transactions to ensure durability.
3. **Undo:** Undo all updates from transactions that did not commit.

**Diagram:**



### 10.3.4 Example Scenario

Consider three transactions at crash time:

- T1: committed, but some writes not on disk.
- T2: committed, all writes on disk.
- T3: active (no commit).

Recovery actions:

- Redo T1's writes (ensure durability).
- No redo needed for T2.



- Undo T3' s writes (rollback).

## 10.4 Checkpoints

A checkpoint is a point in time when the database system synchronizes all dirty buffers (modified pages in memory) to disk and records this fact in the log. Checkpoints reduce recovery time by limiting the portion of the log that must be processed.

### 10.4.1 Why Checkpoints?

Without checkpoints, after a crash the recovery system would need to scan the entire log since the beginning of time, which is impractical for large databases. Checkpoints establish a safe point: transactions that committed before the checkpoint are guaranteed to have all their updates on disk, so they need not be redone.

### 10.4.2 Checkpoint Procedure (Simple Version)

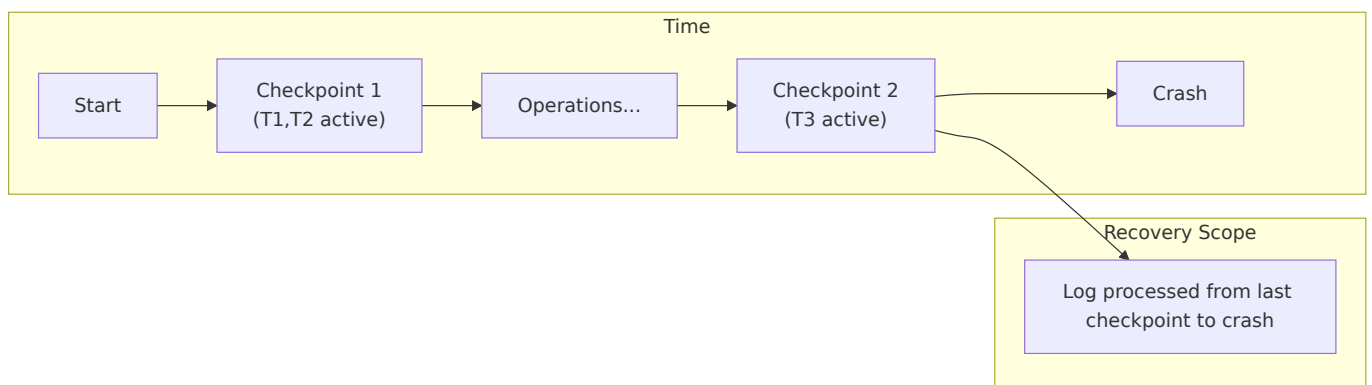
1. Write a `<CHECKPOINT>` log record.
2. Flush all log records up to this point to stable storage.
3. Flush all dirty database buffers to disk.
4. Write a `<CHECKPOINT_END>` record (or mark the checkpoint as complete).

After a crash, recovery only needs to process log records after the most recent successful checkpoint. Any transaction that committed before the checkpoint is already durable.

### 10.4.3 Fuzzy Checkpoints (Advanced)

Modern databases use fuzzy checkpoints, where the system does not stop all transactions during checkpointing. Instead, it records the current active transaction list and proceeds while writes continue. Recovery then uses the log to determine which transactions need redo/undo from the checkpoint onward.

### 10.4.4 Checkpoint Diagram



# 10.4.5 Recovery with Checkpoints

Recovery after a crash with checkpoints:

1. Locate the most recent `<CHECKPOINT>` record in the log.
2. From that point forward, scan the log to build lists of active and committed transactions.
3. Perform redo for all transactions that committed after the checkpoint (or that were active but had commits recorded in the log after the checkpoint).
4. Perform undo for all transactions that were active at crash time.

This greatly reduces the amount of log that must be scanned compared to scanning from the beginning.

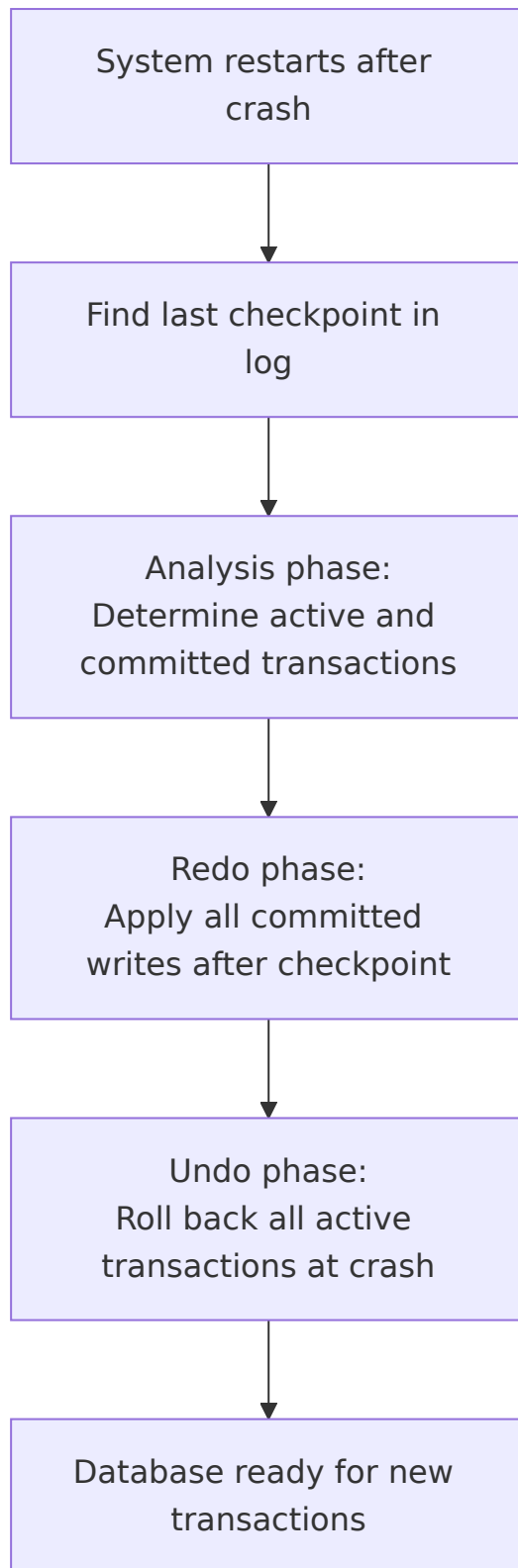
# 10.5 Summary of Recovery Techniques

| Failure Type        | Recovery Mechanism                                      |
|---------------------|---------------------------------------------------------|
| Transaction failure | Rollback using undo (from log, using before images)     |
| System crash        | Restart with analysis + redo + undo (ARIES)             |
| Media failure       | Restore from archive + apply redo from log since backup |

## Key Concepts Recap

- **Log:** Persistent record of all updates (write-ahead logging).
- **Undo:** Restore old values for uncommitted transactions.
- **Redo:** Apply new values for committed transactions (durability).
- **Checkpoint:** Reduces recovery time by bounding the log scan.

Final Recovery Algorithm Flowchart:



# Chapter 11: Indexing

Indexing is a data structure technique used to accelerate data retrieval operations in a database. Without indexes, a DBMS must perform a full table scan to locate rows satisfying a query condition. Indexes provide fast random access and efficient range scans by organizing key values in a searchable structure. This chapter introduces fundamental indexing concepts, classification of indexes, and the widely used B-tree and B+-tree structures.

## 11.1 Indexing Basics

An **index** is a auxiliary structure associated with a table that supports rapid lookup of rows based on the values of one or more columns (the **search key**). An index consists of **index entries** (key-pointer pairs) and is stored separately from the actual data.

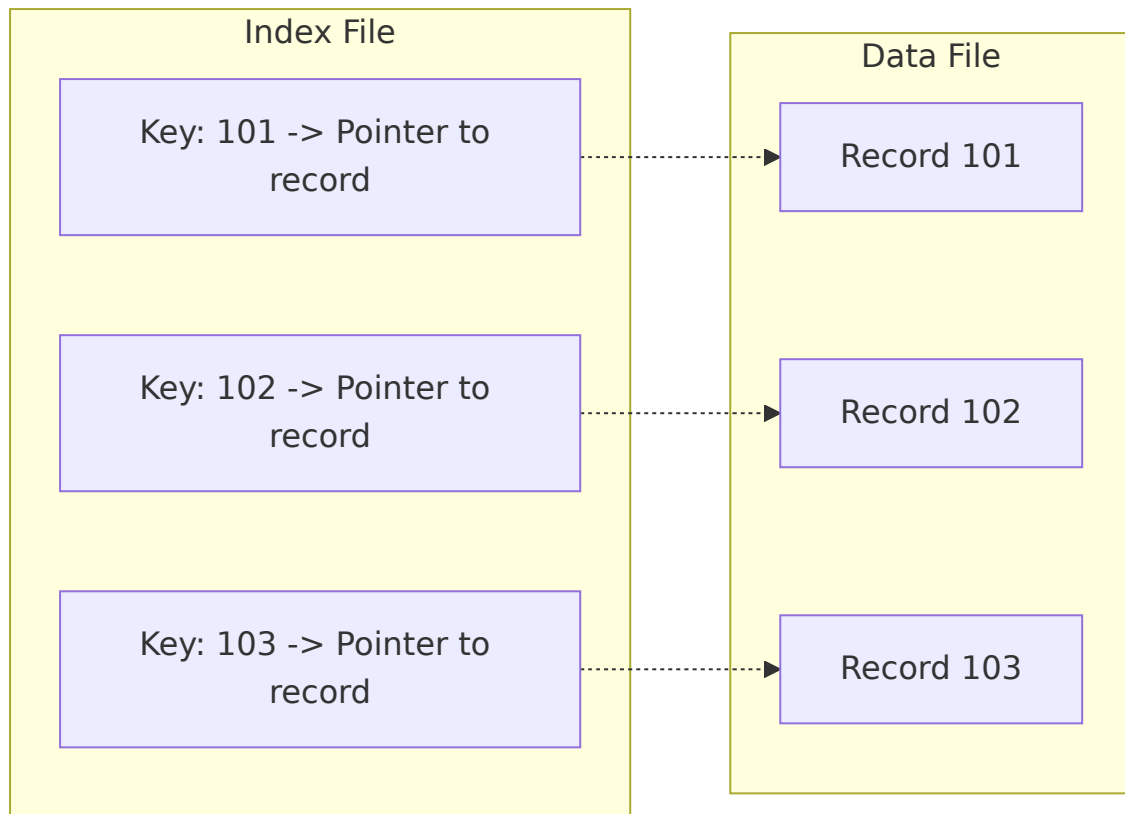
### 11.1.1 Why Indexing?

- **Speed:** Reduces the number of disk I/O operations from  $O(n)$  (full scan) to  $O(\log n)$  or  $O(1)$  for certain structures.
- **Constraints:** Enforces uniqueness (primary key, unique constraint).
- **Sorting:** Can provide ordered access without sorting the entire table.

### 11.1.2 Index Structure Components

- **Search key:** The attribute(s) on which the index is built.
- **Pointer:** Reference to the data record (e.g., row ID, block ID, or primary key value).
- **Ordering:** Index entries are typically stored in sorted order to support efficient searching.

### 11.1.3 Basic Diagram



## 11.2 Primary and Secondary Index

Indexes are classified based on the relationship between the index key and the physical ordering of data records.

### 11.2.1 Primary Index (Clustering Index)

A **primary index** is defined on an ordered file where the data records are physically sorted by the same key as the index. The key is usually the primary key of the table. There is at most one primary index per table because data can be sorted in only one order.

- **Characteristics:**
  - Data file is sorted by the primary key.
  - Index file contains entries for each block (sparse) or each record (dense).
  - Provides very fast lookup for equality and range queries on the primary key.
- **Example:** A table of employees sorted by `emp_id`; primary index on `emp_id`.

### 11.2.2 Secondary Index (Non-Clustering Index)

A **secondary index** is defined on a non-ordering attribute. The data file is not sorted by the secondary key. There can be many secondary indexes per table. Secondary indexes usually contain pointers to the actual data records (row IDs).

- **Characteristics:**

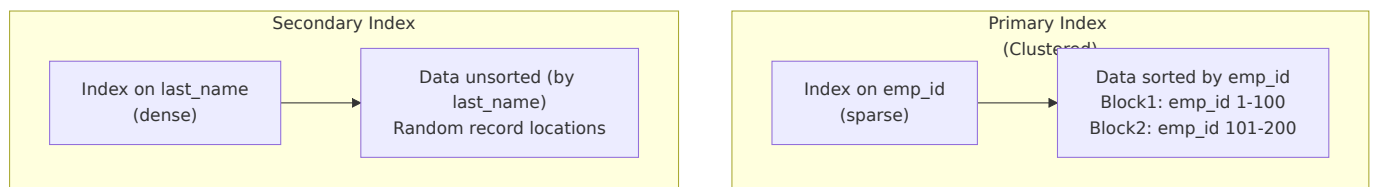
- Index entries are sorted by the secondary key, but data is not.
- Each index entry points to a data record (or a bucket of pointers if duplicate keys exist).
- Useful for accelerating queries on non-primary columns.

- **Example:** An index on `last_name` when the table is ordered by `emp_id`.

### 11.2.3 Comparison Table

| Feature                | Primary Index                      | Secondary Index                  |
|------------------------|------------------------------------|----------------------------------|
| Number per table       | At most one                        | Many                             |
| Data ordering          | Same as index key                  | Independent of index key         |
| Storage overhead       | Lower (can be sparse)              | Higher (usually dense)           |
| Range query efficiency | Very high (sequential data access) | Moderate (random pointer access) |
| Typical use            | Primary key lookups                | Non-key column searches          |

### 11.2.4 Diagram



## 11.3 Dense and Sparse Index

Indexes can also be classified by the number of index entries relative to the data records.

### 11.3.1 Dense Index

A **dense index** contains one index entry for every search key value (and thus for every data record). The index entry directly points to the data record or to the block containing it.

- **Advantages:** Direct lookup; no need to search within a block for the exact record (if index entry points to record).
- **Disadvantages:** Larger storage footprint; requires more updates on insert/delete.
- **Common in:** Secondary indexes, primary indexes on non-block boundaries.

## 11.3.2 Sparse Index

A **sparse index** contains one index entry for each block (or for a range of key values). The index points to the first record in each block. To locate a record, the index is searched to find the appropriate block, then a sequential scan inside the block is performed.

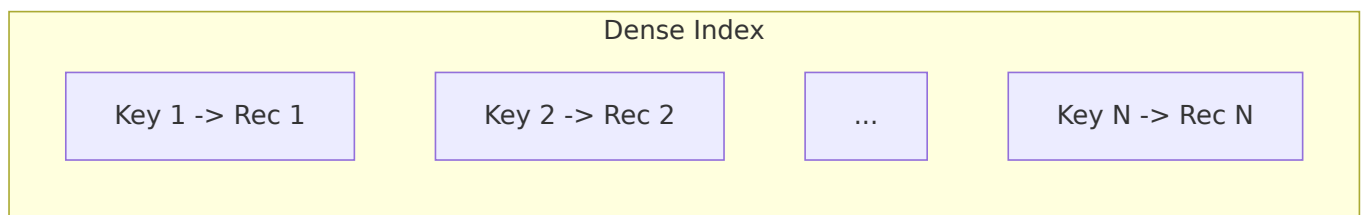
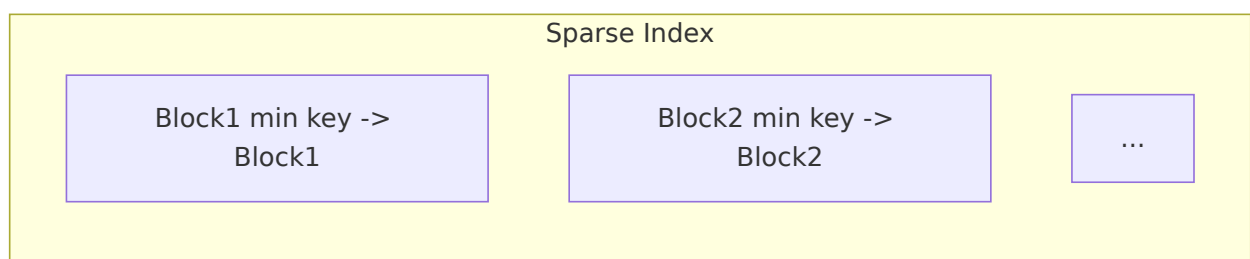
- **Advantages:** Smaller index size; less maintenance overhead.
- **Disadvantages:** Slightly slower for point queries (need to scan within block).
- **Common in:** Primary indexes on ordered files (clustered indexes).

## 11.3.3 Example Comparison

Assume a table with 10,000 records, 100 records per block → 100 blocks.

- **Dense index:** 10,000 index entries.
- **Sparse index:** 100 index entries (one per block).

## 11.3.4 Diagram



## 11.4 B-tree and B+-tree

B-trees and B+-trees are balanced search trees designed for storage systems where disk I/O dominates performance. They maintain a high fanout (many children per node) to keep the tree height low (typically 2-4 levels for millions of records).

### 11.4.1 B-tree

A **B-tree** of order  $m$  (where  $m$  is the maximum number of children per node) has the following properties:

- All leaves are at the same depth (balanced).
- Each internal node (except root) has between  $\lceil m/2 \rceil$  and  $m$  children.
- Each node contains keys and pointers to children and/or data records.
- In a B-tree, data records can be stored in both internal nodes and leaves.

#### **Characteristics:**

- Searching: traverse from root to leaf, possibly finding a record in an internal node.
- Insert/delete: maintain balance through node splits and merges.
- Less common in modern DBMS because storing data in internal nodes complicates range queries.

### **11.4.2 B+-tree**

The **B+-tree** is a variation that stores all data records (or pointers to data) in the leaf nodes only. Internal nodes store only key values to guide the search.

#### **Properties:**

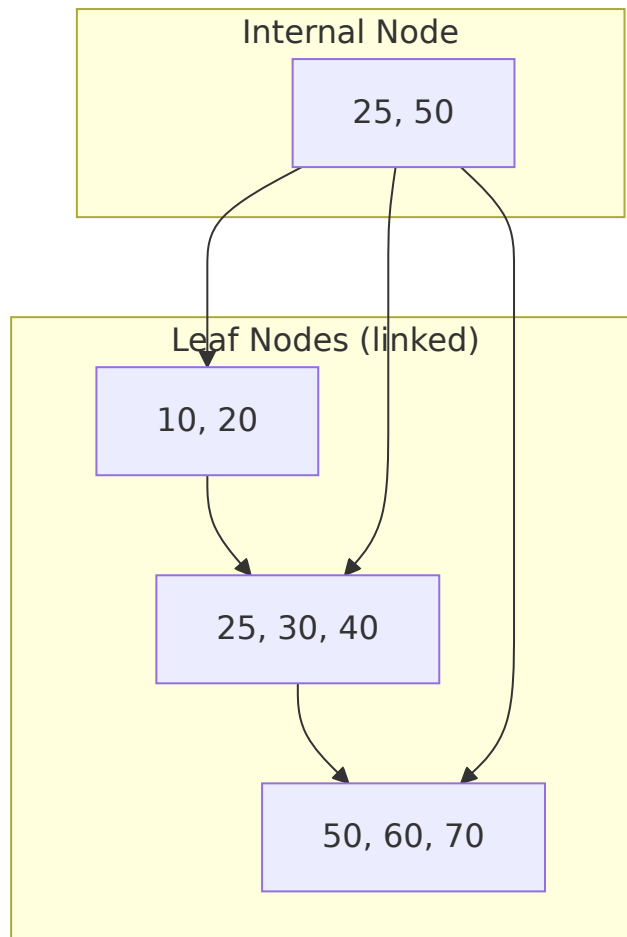
- Internal nodes: keys + child pointers; no data pointers.
- Leaf nodes: keys + data pointers (or actual data). Leaves are linked together in a sequential linked list.
- All leaves at same depth; the leaf level contains all key values in sorted order.

#### **Advantages over B-tree:**

- Efficient range queries: traverse to leftmost leaf then follow leaf-level pointers.
- Higher fanout in internal nodes (since no data pointers), leading to shorter tree height.
- More predictable I/O: data access always reaches leaf level.

#### **Diagram of B+-tree (order 4):**





### 11.4.3 B+-tree Operations

#### Search:

1. Start at root; compare search key with keys in the node.
2. Follow the appropriate child pointer until a leaf is reached.
3. Scan leaf to find exact key or range.

#### Insert:

1. Locate the leaf where the key belongs.
2. Insert the key into the leaf (maintain sorted order).
3. If leaf overflows (exceeds  $m-1$  keys), split into two leaves (each half). Promote the middle key to the parent.
4. If parent overflows, split recursively. If root splits, new root created and height increases.

#### Delete:

1. Locate and remove the key from leaf.
2. If leaf becomes too empty (less than  $\lceil m/2 \rceil$  keys), try to redistribute from a sibling (borrow).

3. If redistribution impossible, merge with a sibling and delete the separator key from parent.
4. If parent becomes too empty, recurse upward.

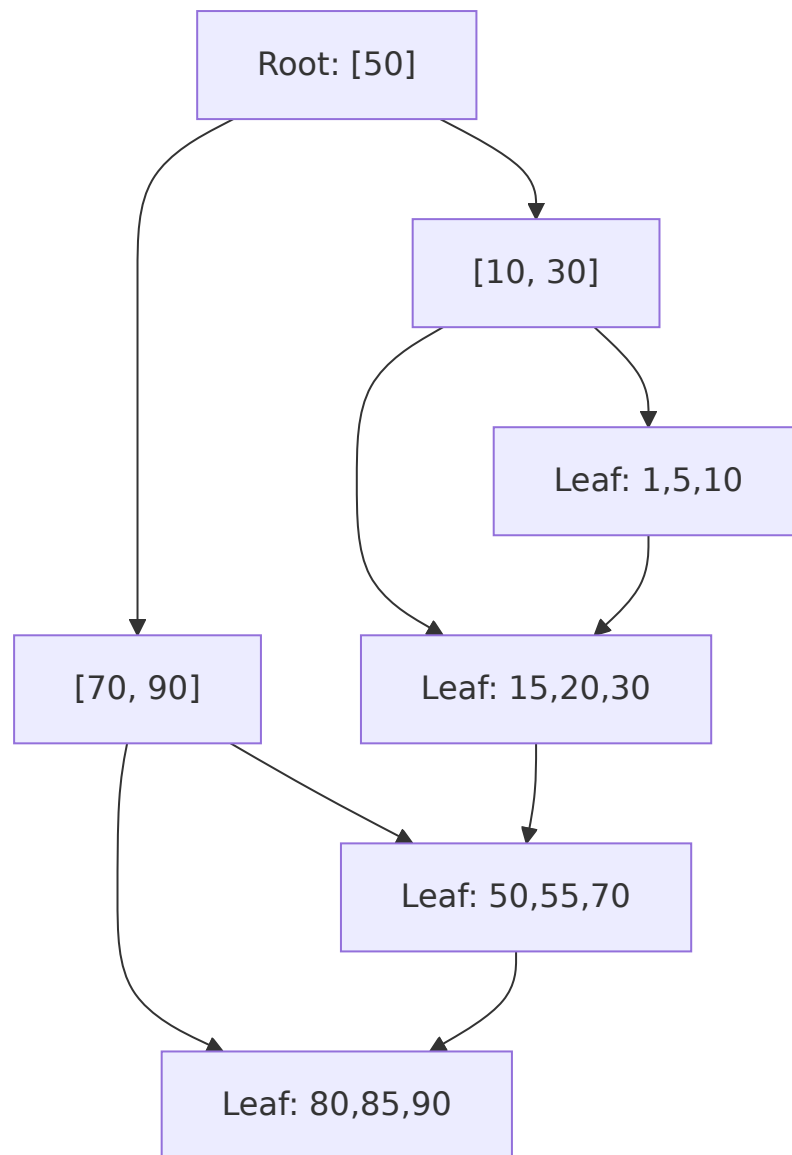
### 11.4.4 Comparison of B-tree and B+-tree

| Feature                | B-tree                            | B+-tree                                                |
|------------------------|-----------------------------------|--------------------------------------------------------|
| Data storage           | Internal nodes + leaves           | Only leaves                                            |
| Leaf linked list       | No (or optional)                  | Yes (sequential scan)                                  |
| Internal node size     | Larger (stores data pointers)     | Smaller (only keys + child ptrs)                       |
| Height for same data   | Slightly taller                   | Shorter (higher fanout)                                |
| Range query efficiency | Poor (must traverse up/down)      | Excellent (leaf chain)                                 |
| Common usage           | Legacy systems, some file systems | Almost all relational DBMS (MySQL, PostgreSQL, Oracle) |

### 11.4.5 Why B+-tree for Databases?

- **Balanced:** Guarantees  $O(\log n)$  I/O per operation.
- **High fanout:** Typically 100-500 children per node → tree height of 3-4 for billions of rows.
- **Range scans:** The leaf-level linked list allows efficient `BETWEEN`, `>`, `<` queries.
- **Concurrency:** Leaf-level locking is possible; B+-trees support high concurrency.

**B+-tree Structure Diagram (detailed):**



## 11.5 Summary

Indexing is critical for database performance. Key takeaways:

- **Indexing basics:** Indexes provide fast lookups at the cost of storage and update overhead.
- **Primary vs secondary:** Primary (clustered) indexes order the data; secondary (non-clustered) indexes provide alternative access paths.
- **Dense vs sparse:** Dense indexes have one entry per record; sparse indexes have one per block (smaller, but slower point lookups).
- **B-tree and B+-tree:**
  - B-trees store data in internal nodes; B+-trees store all data in leaves with leaf-level links.
  - B+-tree is the preferred index structure in modern DBMS due to efficient range queries, higher fanout, and predictable I/O.

Understanding these concepts allows database administrators and developers to choose appropriate indexes and optimize query performance.

# Chapter 12: File Organization and Hashing

File organization refers to the way data records are physically arranged on storage media (e.g., disk). The choice of file organization affects the efficiency of database operations such as insert, delete, update, and search. Hashing is a technique that maps keys directly to storage addresses, providing very fast equality searches. This chapter explains heap and sequential file organizations, as well as static and dynamic hashing schemes.

## 12.1 File Organization Basics

A database file is a collection of fixed-length or variable-length records. The file organization determines:

- How records are placed in blocks (pages) on disk.
- How records are located given a search key.
- Performance of sequential scans, point queries, and range queries.

Three fundamental operations are optimized differently by each organization:

- **Insert:** Add a new record.
- **Search (equality):** Find record with a given key.
- **Range search:** Find all records with keys in a range.
- **Delete/Update:** Modify or remove a record.

## 12.2 Heap File Organization

A **heap file** (also called unordered file) stores records in no particular order. New records are appended at the end of the file or inserted into a free space list. There is no sorting or ordering.

### 12.2.1 Characteristics

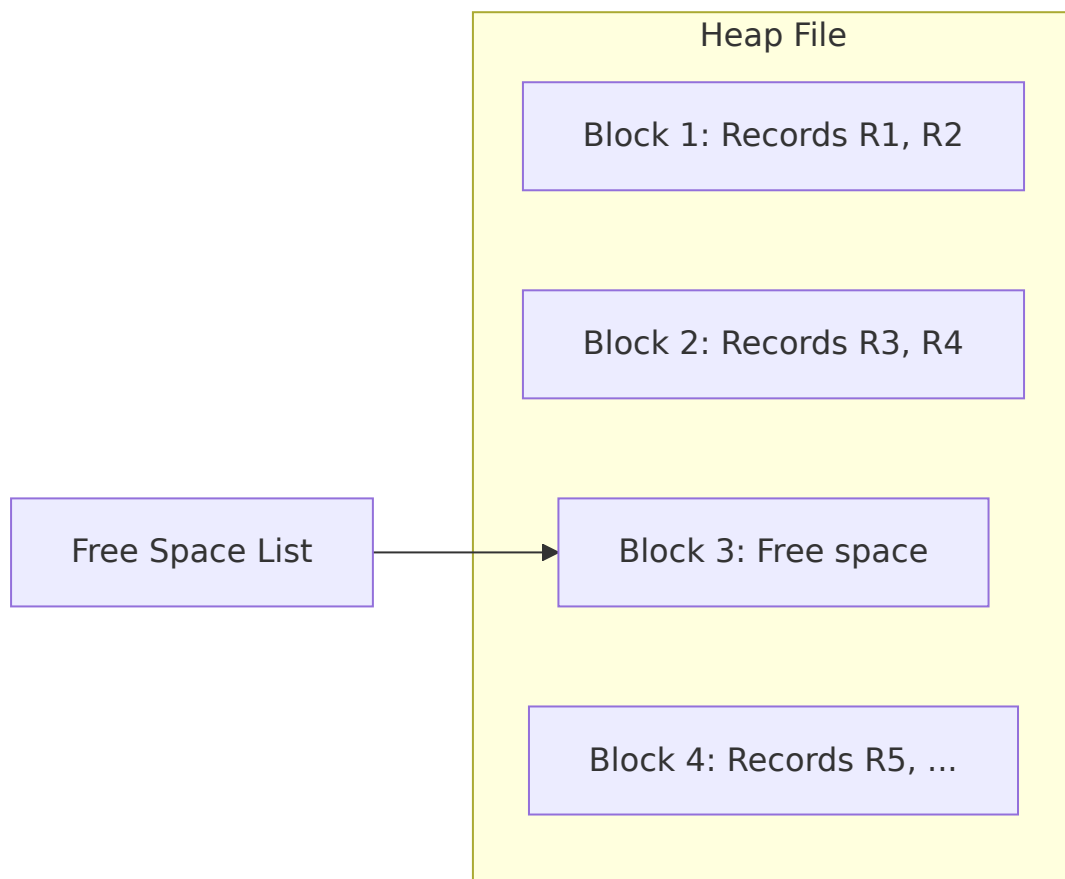
- **Insertion:** Very fast – just append or use a free slot.
- **Search (equality):** Slow – full file scan required ( $O(n)$  disk I/O).
- **Range search:** Same as full scan.
- **Delete:** Mark record as deleted (or compact later); can maintain a free list.

- **Update:** In-place if record size fixed; otherwise complicated.

### 12.2.2 Use Cases

- Temporary tables, log tables, or tables with very few queries.
- Bulk insert environments where retrieval is rare.

### 12.2.3 Heap File Structure with Free Space List



## 12.3 Sequential File Organization

A **sequential file** stores records in sorted order by a designated key (the ordering key). Records are physically arranged in ascending or descending key order. Insertion and deletion are costly because the order must be maintained.

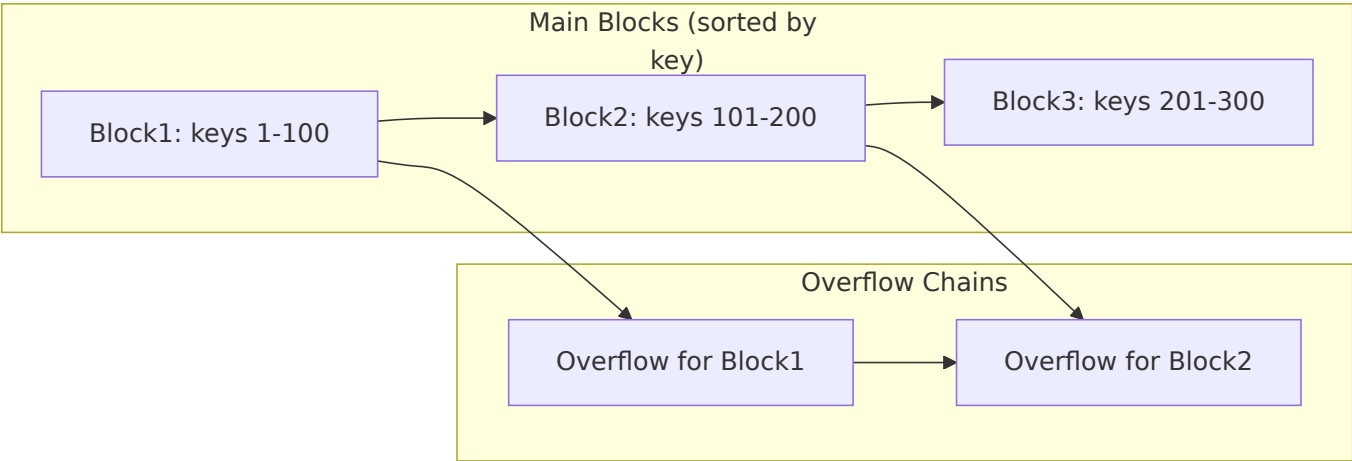
### 12.3.1 Characteristics

- **Insertion:** Requires finding correct position; may need to shift records or use overflow blocks.
- **Search (equality):** Fast using binary search on the file ( $O(\log n)$  block accesses).
- **Range search:** Very fast after locating start key; sequential read of subsequent blocks.
- **Delete:** Mark as deleted or compact; similar to insertion overhead.

## 12.3.2 Overflow Handling

When inserting a new record, if the target block is full, an **overflow block** is linked. Overflow chains degrade performance over time; periodic reorganization is needed.

## 12.3.3 Sequential File Structure



## 12.3.4 Comparison Table

| Operation           | Heap File               | Sequential File           |
|---------------------|-------------------------|---------------------------|
| Insert              | Fast (append)           | Slow (maintain order)     |
| Equality search     | Slow (scan)             | Fast (binary search)      |
| Range search        | Slow (scan)             | Very fast (sequential)    |
| Delete              | Moderate (mark)         | Moderate (mark + compact) |
| Storage utilization | High (few wasted slots) | Lower (overflow blocks)   |

## 12.4 Hashing

Hashing maps a search key directly to a disk block address using a **hash function**  $h(\text{key})$ . Hashing is ideal for equality queries ( `WHERE key = value` ) but inefficient for range queries.

### 12.4.1 Basic Hashing Concepts

- **Hash function:**  $h(\text{key}) \rightarrow \text{bucket number}$  (bucket = one or more disk blocks).
- **Buckets:** Logical storage units; each bucket can hold multiple records (typically one block).

- **Collision:** Two different keys map to the same bucket. Handled by overflow chaining or open addressing.

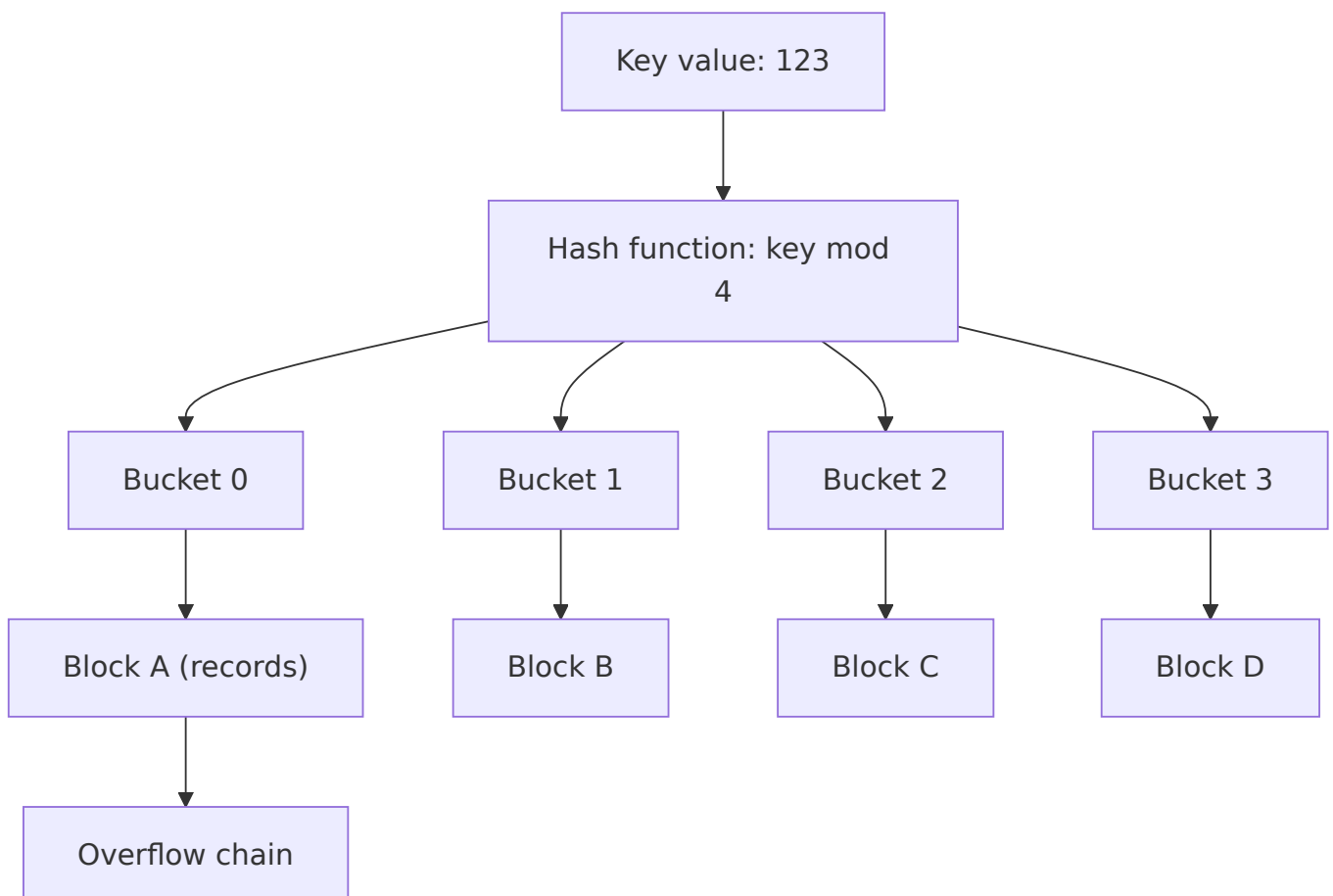
## 12.4.2 Static Hashing

In **static hashing**, the number of buckets is fixed and does not change over time. The hash function distributes keys among these buckets.

### Characteristics:

- **Simple:** Fixed bucket count  $N$ . Hash function:  $h(\text{key}) = \text{key} \bmod N$ .
- **Insert:** Compute bucket, insert; if bucket full, add overflow block.
- **Search:** Compute bucket, then search within bucket (and overflow chain).
- **Problems:**
  - If database grows, buckets become overloaded  $\rightarrow$  long overflow chains  $\rightarrow$  performance degrades.
  - If database shrinks, many empty buckets waste space.
- **Use cases:** Small, stable tables or applications with predictable size.

### Static Hashing Diagram:





## 12.4.3 Dynamic Hashing

Dynamic hashing (also called **extendible hashing**) allows the number of buckets to grow and shrink as the database size changes. It uses a directory of pointers to buckets and a hash function that produces a long sequence of bits (e.g., 32-bit hash). Only the first  $i$  bits are used, where  $i$  grows when a bucket overflows.

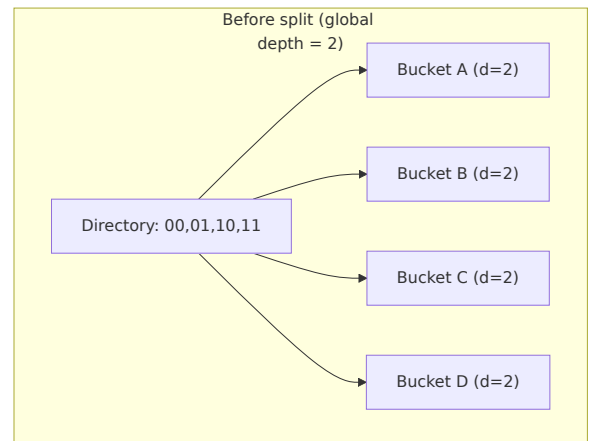
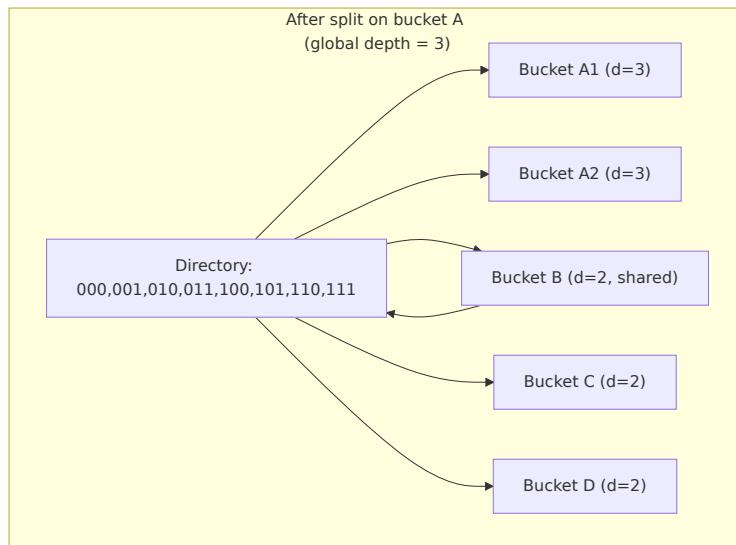
### Extendible Hashing Algorithm:

- **Directory:** Array of size  $2^i$  ( $i$  is the global depth). Each directory entry points to a bucket.
- **Bucket:** Has local depth  $d \leq i$ . A bucket holds up to  $B$  records.
- **Insert:**
  1. Hash key to get binary hash value.
  2. Use first  $i$  bits to index directory.
  3. Follow pointer to bucket.
  4. If bucket not full, insert.
  5. If bucket full:
    - If local depth  $d$  equals global depth  $i$ , double the directory ( $i \leftarrow i+1$ ).
    - Create a new bucket; redistribute records between old and new buckets using  $d+1$  bits.
    - Increment local depth of affected buckets.
    - Adjust directory pointers.
- **Search:** Same as insertion but no redistribution.
- **Delete:** May merge buckets and shrink directory if possible.

### Characteristics:

- **Dynamic growth:** No overflow chains; bucket splits maintain performance.
- **Directory overhead:** Directory can double in size; but for many databases, directory fits in memory.
- **Good for:** Growing databases, unpredictable size, hash-based indexes (e.g., in some NoSQL systems).

### Extendible Hashing Diagram (before and after split):



## 12.4.4 Comparison of Static vs Dynamic Hashing

| Feature                   | Static Hashing                      | Dynamic (Extendible) Hashing        |
|---------------------------|-------------------------------------|-------------------------------------|
| Bucket count              | Fixed                               | Grows/shrinks                       |
| Overflow handling         | Overflow chains (performance decay) | Bucket split (no chains)            |
| Directory                 | None (direct bucket access)         | Directory of pointers (size $2^i$ ) |
| Space utilization         | Can become poor (empty buckets)     | Good (splits on overflow)           |
| Performance over time     | Degrades with growth                | Stable (logical splits)             |
| Implementation complexity | Simple                              | More complex (directory management) |
| Use case                  | Small, static tables                | Large, dynamic databases            |

## 12.4.5 Hashing vs B+-tree

| Aspect         | Hash Index                      | B+-tree Index               |
|----------------|---------------------------------|-----------------------------|
| Equality query | $O(1)$ (ideal)                  | $O(\log n)$                 |
| Range query    | Not supported (needs full scan) | Very efficient (leaf chain) |
| Ordering       | No (keys scattered)             | Yes (sorted)                |

|                 |                             |                                |
|-----------------|-----------------------------|--------------------------------|
| Update overhead | Low (if no overflow)        | Moderate (node splits/merges)  |
| Typical use     | Hash join, equality lookups | General purpose (most indexes) |

## 12.5 Summary

File organization and hashing determine how data is physically stored and accessed. Key points:

- **Heap file:** Unordered; fastest inserts; slow searches; suitable for logs and temporary tables.
- **Sequential file:** Sorted by key; efficient range scans; costly updates; requires overflow handling.
- **Static hashing:** Fixed number of buckets; simple but suffers from performance degradation as database grows (overflow chains).
- **Dynamic hashing (extendible hashing):** Grows gracefully by splitting buckets; uses a directory; no overflow chains; ideal for unpredictable database size.

Choosing the right organization depends on workload: heap for insert-heavy, sequential for range queries, hashing for exact match lookups, and B+-trees for balanced mixed workloads.

# Chapter 13: Storage and Buffer Management

Efficient database management relies heavily on how data is stored on disk and how it is moved between disk and main memory. Disk I/O is orders of magnitude slower than memory access, making storage and buffer management critical for performance. This chapter describes the physical disk structure, buffer pool management, and the access techniques used to reduce I/O overhead.

## 13.1 Disk Storage Structure

Magnetic hard disk drives (HDDs) and solid-state drives (SSDs) are the primary persistent storage media in database systems. Understanding HDD geometry helps in optimizing data placement; SSDs have different characteristics but are often abstracted similarly at the file system level.

### 13.1.1 Physical Disk Components (HDD)

A typical HDD consists of:

- **Platters:** Circular magnetic disks stacked on a spindle.
- **Tracks:** Concentric circles on each platter surface.
- **Sectors:** The smallest addressable unit on a track (typically 512 bytes or 4 KB).
- **Blocks (Pages):** A logical unit of data transfer between disk and memory (usually 4 KB, 8 KB, or 16 KB). A block spans one or more consecutive sectors.
- **Heads:** Read/write heads attached to an arm that moves across tracks.
- **Cylinder:** All tracks at the same radius across all platters.

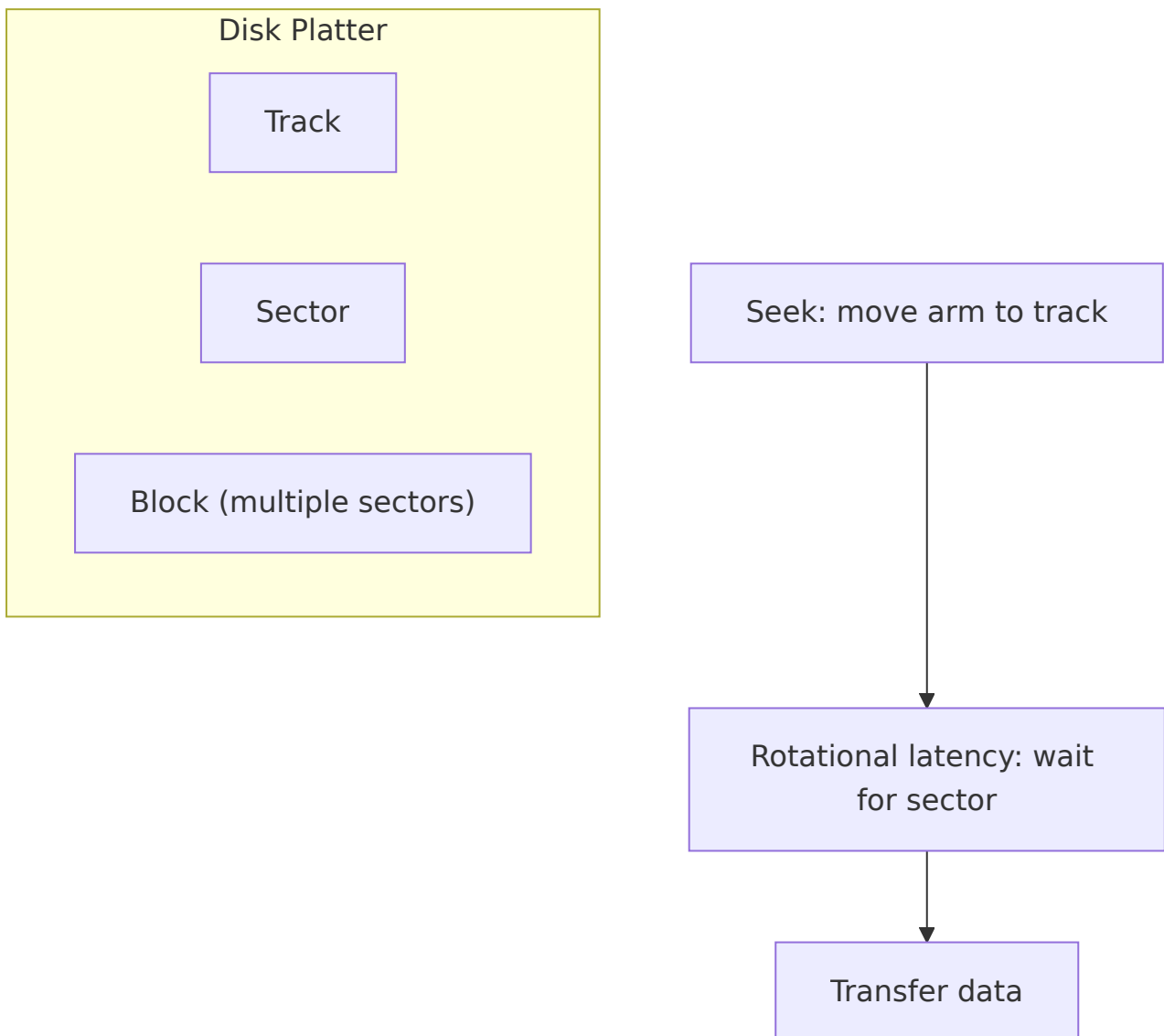
### 13.1.2 Disk Access Time Components

When reading or writing a block, three delays occur:

1. **Seek time:** Time to move the arm to the correct track. (Dominant cost, 5-10 ms)
2. **Rotational latency:** Time for the desired sector to rotate under the head (average half a rotation, 2-4 ms for 7200 RPM).
3. **Transfer time:** Time to read/write the data (negligible for a block, <0.1 ms).

Thus, sequential access (reading consecutive blocks) is much faster than random access because seek and rotational delays are minimized.

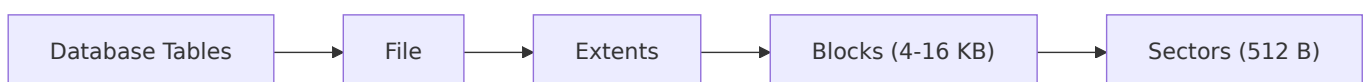
**Diagram:**



### 13.1.3 File Organization on Disk

The database management system (DBMS) maps logical database objects (tables, indexes) into files, which are then stored as blocks on disk. Each block has a unique address (disk block number). The DBMS' s storage manager is responsible for allocating, reading, and writing these blocks.

**Block Diagram:**



## 13.2 Buffer Management

The **buffer manager** is responsible for bringing disk blocks into main memory (the buffer pool) and managing them efficiently. It reduces disk I/O by keeping frequently used blocks in memory.

### 13.2.1 Buffer Pool

The buffer pool is an array of **frames**, each the size of a disk block. When a block is needed, the buffer manager:

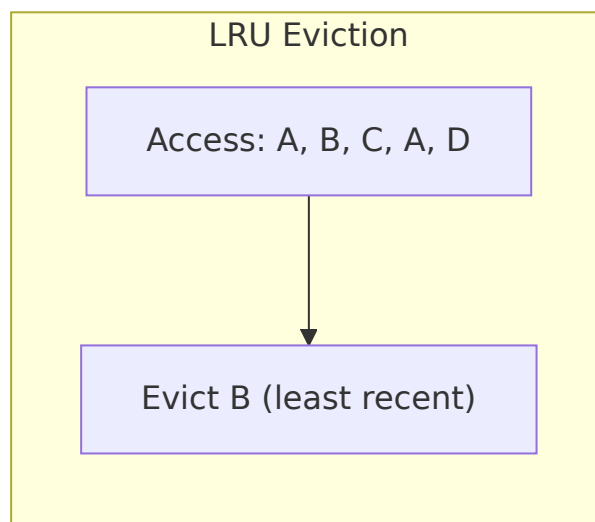
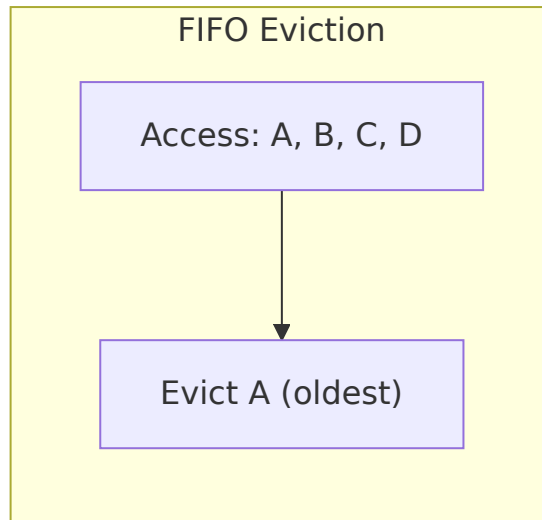
1. Checks if the block is already in the buffer pool (cache hit).
2. If not (cache miss), reads the block from disk into a free frame.
3. If no free frame is available, selects a victim frame to evict (using a replacement policy).
4. If the victim frame is dirty (modified), writes it back to disk before eviction.

### 13.2.2 Buffer Replacement Policies

The choice of replacement policy affects buffer hit rates. Common policies:

- **Least Recently Used (LRU)**: Evicts the block that has not been accessed for the longest time. Works well for repeated access patterns.
- **First-In-First-Out (FIFO)**: Evicts the oldest loaded block. Simple but may evict frequently used blocks.
- **Clock (Second Chance)**: An efficient approximation of LRU using a circular list and reference bits.
- **Most Recently Used (MRU)**: Evicts the most recently used block; useful for certain scan patterns.
- **Random**: Evicts a random block; surprisingly effective in some workloads.

**LRU vs FIFO Example Diagram:**



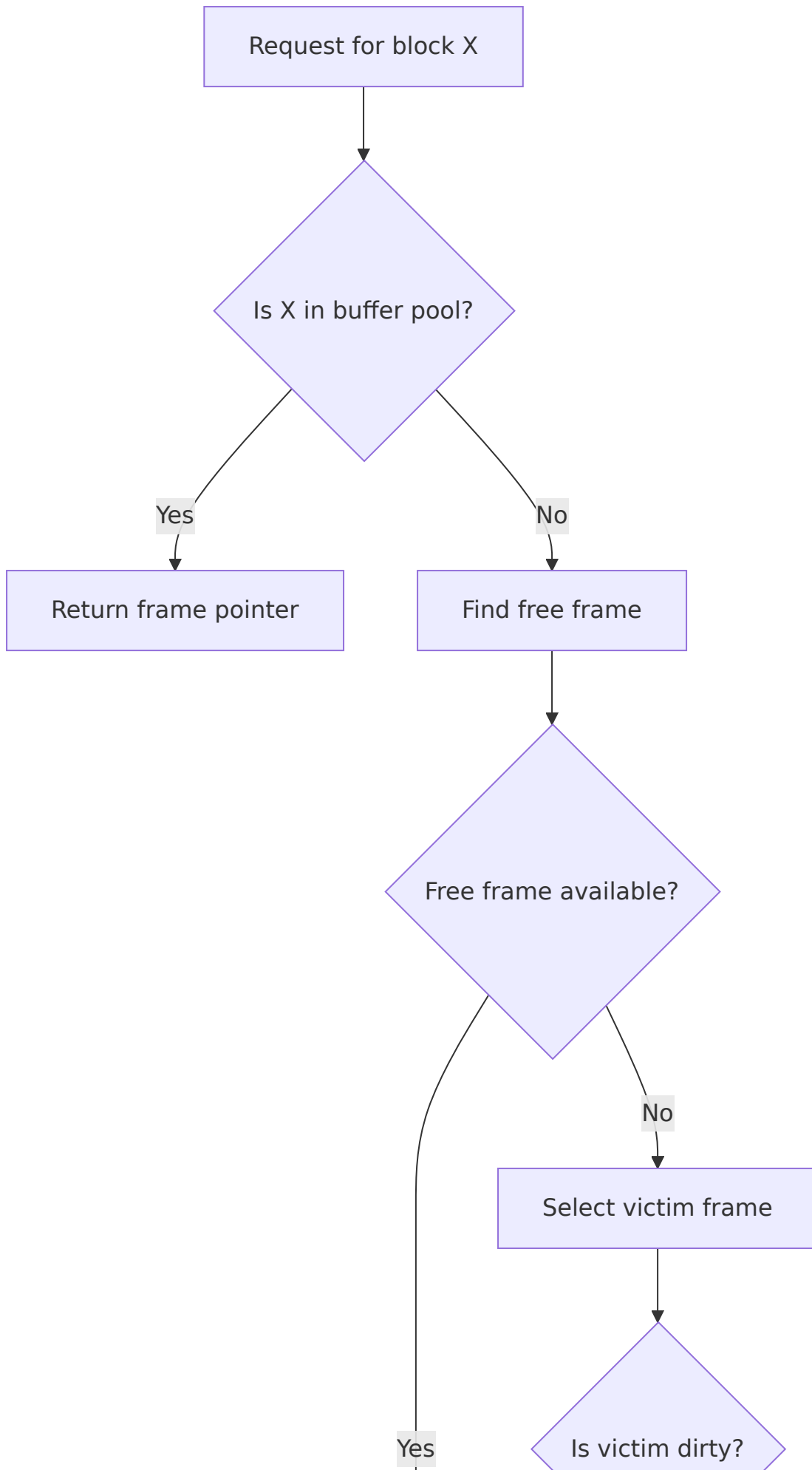
### 13.2.3 Dirty Blocks and Forced Writes

A **dirty block** is a buffer frame that has been modified but not yet written to disk. Before evicting a dirty block, the buffer manager must write it back. This write is called **forced write**. To improve performance, the DBMS may periodically flush dirty blocks using a background writer.

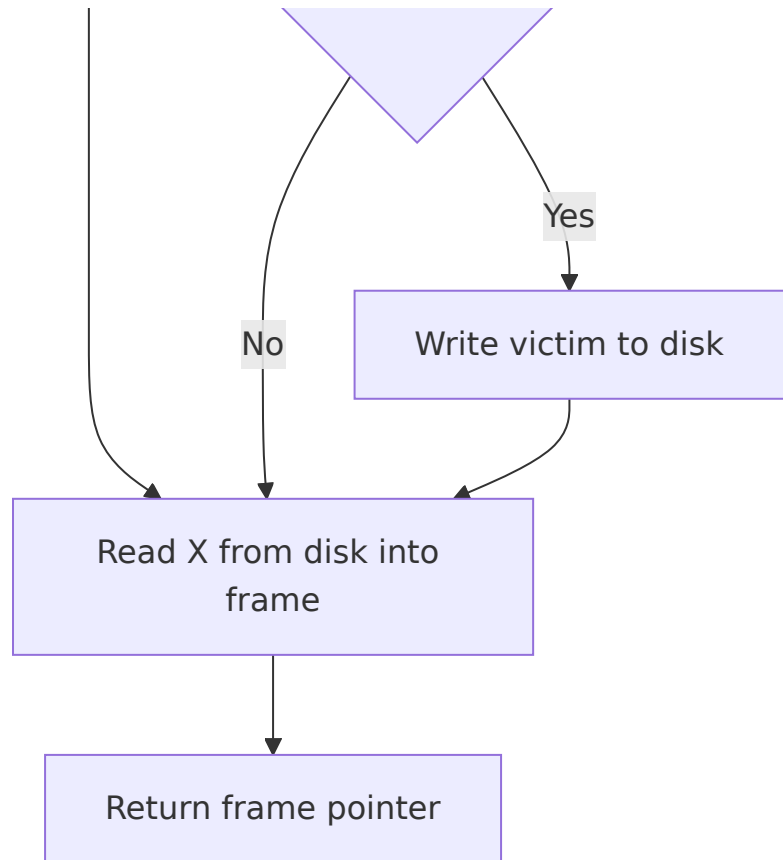
### 13.2.4 Buffer Manager Operations

- **Pin/Unpin:** A pinned block cannot be evicted. Used when a transaction is actively using a block.
- **Flush:** Write a dirty block to disk.
- **Prefetching:** Read blocks ahead of expected use to amortize seek time.

**Buffer Manager Flowchart:**







### 13.2.5 Buffer Management in Practice

- **Local vs Global policies:** Local policies manage buffers per transaction; global policies manage the entire pool.
- **Hysteresis:** Some DBMSs use two thresholds (low water, high water) to control flushing.
- **OS caching:** The DBMS often bypasses the operating system's file cache (using direct I/O) to avoid double caching and gain control over replacement.

## 13.3 Access Techniques

Access techniques are strategies to minimize disk I/O when retrieving data. They combine file organization, indexing, and buffer management.

### 13.3.1 Sequential Access

Sequential access reads consecutive blocks from disk. Because seek and rotational latency are incurred only once per large read, it is very efficient. Used for:

- Table scans (full table reads).
- Range scans on clustered indexes.
- Sequential file processing.

**Sequential vs Random Access Diagram:**

### Random Access

Block 1

Block 3

Block 2

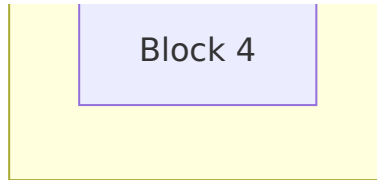
Block 4

### Sequential Access

Block 1

Block 2

Block 3



### 13.3.2 Indexed Access

Using an index (B+-tree, hash) to locate a record typically requires fewer block accesses than a full scan. For a B+-tree of height  $h$ , a point query reads:

- $h$  index blocks (from root to leaf).
- 1 data block (if not already in buffer).

For range queries, after locating the leftmost leaf, the leaf chain provides sequential access.

### 13.3.3 Prefetching (Read-Ahead)

The DBMS predicts which blocks will be needed soon and issues asynchronous read requests. Common patterns:

- **Sequential prefetch:** After reading block  $i$ , prefetch block  $i+1$ ,  $i+2$ , etc.
- **Index leaf prefetch:** Prefetch sibling leaves when scanning an index.

Prefetching hides latency but must be tuned to avoid wasting I/O bandwidth.

### 13.3.4 Double Buffering

Double buffering uses two buffers: one is being processed by the CPU while the other is being read from disk. This overlaps I/O with computation. Often used in external sorting and hash joins.

### 13.3.5 Columnar Storage Access

In column-oriented DBMSs, each column is stored separately. Queries that access only a few columns read only the relevant column files, reducing I/O. This is an advanced access technique that improves cache efficiency and compression.

## 13.4 Summary

Storage and buffer management form the foundation of database performance. Key concepts:

- **Disk storage:** Platters, tracks, sectors, blocks; access time dominated by seek and rotational latency.

- **Buffer management:** Buffer pool caches blocks in memory; replacement policies (LRU, Clock, etc.) choose victims; dirty blocks must be written back.
- **Access techniques:** Sequential access is fast; indexed access reduces block reads; prefetching and double buffering overlap I/O with computation.

Understanding these mechanisms allows database administrators and developers to configure memory, choose appropriate file organizations, and optimize queries for minimal I/O.